

Université de Strasbourg

École Doctorale 269: Mathématiques, Sciences de l'Information et de l'Ingénieur

Laboratoire ICube

Programmable and Self-adaptive Low-Power and Lossy Wireless Networks

THÈSE présentée par

Ahmad MAHMOD

Pour obtenir le grade de : **Docteur de l'université de Strasbourg**

Discipline : **Informatique**

Soutenue le : 14 September 2026

Jury de thèse

Directeurs :

Julien Montavont, *Maître de Conférences HDR, Université de Strasbourg*

Thomas Noël, *Professeur, Université de Strasbourg*

Rapporteurs :

Adrien van den Bossche, *Maître de Conférences HDR, Université de Toulouse Jean Jaurès*

Nathalie Mitton, *Directrice de Recherche, Inria Lille*

Examineurs :

Oana Teodora Iova, *Maître de Conférences, INSA Lyon*

Cristel Pelsser, *Professeure, Université catholique de Louvain*

Acknowledgments

I would like to express my deepest gratitude to everyone who contributed, in one way or another, to the completion of this PhD journey.

First and foremost, I would like to thank my supervisors, Julien Montavont and Thomas Noel, for their unwavering support throughout this journey. I am sincerely grateful for the fruitful discussions, thoughtful guidance, and constant encouragement. Working with you has been a true pleasure, both professionally and personally.

I would also like to thank the members of my PhD committee, Adrien van den Bossche, Oana Iova, Nathalie Mitton, and Cristel Pelsser, for kindly agreeing to evaluate this work and for the time they devoted to its assessment and discussion. Their insightful comments and constructive feedback have greatly contributed to improving the quality of this thesis.

I am also grateful to the members of the Network team, with whom I had the honor of working during this research journey. I would like to thank the colleagues for their kindness, sincerity, and support, and I wish them all the very best in their future endeavors. I also extend my best wishes to the students of the team for the successful completion of their projects and for bright and fulfilling careers ahead.

I cannot forget the friends who stood by me throughout my academic journey: Abdulrahman, Adeeb, Bashar, and Hani; later Issam and Mahmoud Ezzat; and, during my PhD, Samir, Thomas, Anjali, Kamel, and Ciro. Each of you played an important role in helping me reach this point. Thank you for your support, your presence, and the time we spent together—moments filled with love, encouragement, and laughter.

I would also like to express my gratitude to my childhood friends, Ali Bassam, Mohammed Kartish, Khaldoun, and Basel, whose friendship has remained a meaningful part of my life.

Finally, I would like to thank my family. I dedicate this achievement to the soul of my father, whom time did not allow to witness this moment. I also express my deepest gratitude to my mother for the endless love, care, and support she has given me throughout my life, leading up to this day. To my sisters, Wafaa, Samar, and Sawsan, and to my brother Ali: thank you for your constant support, your love, and for always being there for me.

Abstract

Low-Power and Lossy Wireless Networks (LLWNs) form a fundamental communication substrate for the Internet of Things (IoT), interconnecting constrained devices across diverse application domains such as environmental monitoring, smart infrastructure, healthcare, and industrial sensing. These networks operate under severe memory, processing, energy, and bandwidth limitations while relying on unreliable multi-hop wireless links. At the same time, LLWN deployments are exposed to evolving operating conditions, including topology changes, link-quality variations, heterogeneous application requirements, and changing traffic profiles. As a result, LLWNs must continuously balance communication reliability and resource efficiency in constrained and dynamic environments.

Despite this need for adaptability, LLWN devices have historically relied on monolithic network stacks in which protocol logic is tightly integrated into the firmware and fixed at compile time. Consequently, even minor modifications to the network stack often require large firmware updates, making protocol evolution costly, slow, and impractical while also causing network-state loss and risking connectivity disruptions. These limitations have led to static network-stack configurations that cannot efficiently adapt to evolving conditions, particularly under changing traffic patterns, thereby highlighting the need for networks that are both programmable and capable of dynamic runtime adaptation.

This thesis aims to advance the programmability and adaptability of LLWNs. First, it introduces a programmable modular network-stack architecture that decomposes protocol logic into lightweight virtual functions coordinated by a central orchestrator. By enabling fine-grained composition and over-the-air management of protocol components, this architecture supports fast, selective runtime updates while respecting the strict resource constraints of LLWN devices, thereby bridging the gap between programmability and efficiency. Second, the thesis enables flexible control plane deployments, in particular, hybrid control plane models that combine local in-network autonomy with optional global orchestration. These hybrid models are then used to support state-preserving runtime updates of the fine-grained network stack. This is necessary because decomposing the stack into independent modules creates interdependencies and shared state among protocol components, which may lead to post-update inconsistencies or require state reconstruction if this state is not handled explicitly. Third, the thesis develops a global, intelligent, traffic-aware framework for LLWN medium access control. This framework aims to make LLWNs proactively self-adaptive by integrating runtime monitoring, short-horizon prediction, recommendation, and online adaptation of the running configurations.

Overall, this thesis shows that LLWNs can evolve from rigid, statically configured network stacks toward programmable, state-aware, and self-adaptive designs capable of operating efficiently under dynamic and constrained conditions.

Contents

Acknowledgments	ii
Abstract	iii
List of Abbreviations	xi
1 Introduction	2
1.1 Economic Impact of LLWNs	3
1.2 Open Scientific Issues in LLWN	4
1.3 Thesis Organization and Contributions	6
1.4 List of Publications	8
2 Background	11
2.1 Low-Power and Lossy Wireless Networks	11
2.1.1 Characteristics of LLWNs	12
2.1.2 Applications and Traffic Regimes	13
2.2 Communication Protocol Stack for LLWNs	15
2.2.1 Protocol Stack Overview	15
2.2.2 Functional Decomposition of Network Operation	18
2.3 Medium Access Control in LLWNs	20
2.3.1 Contention-free MAC Protocols	20
2.3.2 Contention-based MAC Protocols	22
2.4 Conclusion	23
3 State of the Art	26
3.1 Software Evolution and Over-the-air Updates in LLWNs	26
3.1.1 Firmware Image Updates	27
3.1.2 Dynamic Code Update Frameworks	29
3.1.3 Conclusion	35
3.2 Control plane Deployment Models in LLWN	36
3.2.1 Distributed Control Plane Deployments in LLWNs	36
3.2.2 Centralized Control Plane Deployments in LLWNs	37
3.2.3 Hybrid Control Plane Deployments	40
3.2.4 Conclusion	43

3.3	MAC-Layer Adaptation in LLWNs	43
3.3.1	Adaptive MAC Protocols in LLWNs	44
3.3.2	Machine Learning for MAC-layer in LLWNs	46
3.3.3	Conclusion	50
3.4	Conclusion	50
4	A Modular Network Stack Design for LLWNs	53
4.1	Introduction	54
4.2	The FINE-STACK Design	55
4.2.1	Design Concepts	55
4.2.2	Network Stack	55
4.2.3	Complete Architecture	57
4.2.4	Update Process	58
4.3	Evaluation	59
4.3.1	Packet Delivery Ratio (PDR)	61
4.3.2	Update Size and Time	62
4.3.3	Post-update Convergence	64
4.3.4	Update Overhead	64
4.3.5	Security and Recovery Cost	66
4.3.6	Memory Footprint	67
4.3.7	Power Consumption	68
4.3.8	Execution Time	69
4.4	Conclusion	70
5	Hybrid Control Plane Deployments and State-Aware Runtime Updates for LLWNs	72
5.1	Introduction	72
5.2	FINE-STACK with Hybrid Control Plane Deployment	73
5.2.1	Network Stack for Different Control Plane Deployments	73
5.2.2	Architectural Support for Hybrid Deployment	75
5.3	Evaluation	76
5.3.1	Update Size and Time	77
5.3.2	Control Plane Deployments Performance Comparison	80
5.4	Conclusion	82
6	AI-Enabled Traffic-Aware Closed-Loop Orchestration for LLWNs	85
6.1	Introduction	85
6.2	Methodology and Framework	87
6.2.1	Dataset Collection	88
6.2.2	Dataset Construction	90
6.2.3	Offline Training and Model Design	93
6.2.4	Model Families and Performance Comparison	95

6.2.5	Online Closed-Loop Management	97
6.3	Evaluation	98
6.3.1	Prediction Model Performance	99
6.3.2	Forecasting-based Recommendation Performance	101
6.3.3	Adaptability to Data Profiles	103
6.3.4	Scalability over a Large Heterogeneous Topology	105
6.4	Conclusion	106
7	Conclusion and Perspectives	108
7.1	Thesis Contributions	108
7.2	Limitations and Perspectives	110
7.2.1	Modular Network Stack	110
7.2.2	Update Management	111
7.2.3	Update Automation	113
7.3	Concluding Perspective	113
	Bibliography	116

List of Figures

1.1	Number of connected IoT devices between 2020 and 2035 [6]	3
1.2	Thesis contributions	7
2.1	Representative LLWN consisting of constrained nodes and a gateway	12
2.2	LLWN network protocol stack	15
2.3	Comparison of RPL non-storing and storing modes of operation	17
2.4	Control Plane Deployments: (a) Distributed, (b) Centralized	18
2.5	TSCH MAC protocol operation	21
2.6	Duty-cycled MAC protocol operation	22
3.1	Firmware size comparison for different operating systems.	27
3.2	Module-based update process	30
3.3	Strict binding module-based approach	31
3.4	Dynamic binding module-based approach	32
3.5	Femto-Container operation and integration in the operating system [88] . .	34
3.6	Convergence time in RPL over hop distance in a 7-node line topology. . . .	37
3.7	SDN-TSCH operation [95]	39
3.8	Hybrid control plane deployment	40
3.9	SOHS dual-stack operation [106]	42
3.10	GoMacH frame structure	45
3.11	Per-link prediction framework [116]	47
3.12	Prediction-based adaptive duty-cycle for solar energy-harvesting networks [122]	49
4.1	FINE-STACK VF-based network stack in an LLWN device	56
4.2	FINE-STACK architecture. The orchestrator manages the VF-based network stack through the Network Manager and Management Server, while each device maintains a VF Pool and Management Agent to validate, install, and coordinate updates across constrained nodes.	57
4.3	Update process. Two-phase update procedure (install then activate) ensuring network-wide consistency: nodes first authenticate and store the VF in the pool, then activate it synchronously after enough nodes acknowledge installation.	58
4.4	Native and FINE-STACK stacks and update methodology	60

4.5	Topology on SLICES-FR testbed. It includes 25 M3 nodes, one node serves as the Border Router (BR), providing connectivity to the orchestrator. . . .	61
4.6	Packet Delivery Ratio (PDR) comparison between FINE-STACK and native implementation. FINE-STACK restores PDR by updating the network stack at runtime.	62
4.7	Update time comparison between FINE-STACK and native implementation. FINE-STACK reduces the update time significantly by 94.4%	63
4.8	Post-update convergence time. While the native implementation requires more than 60 s to reconverge, FINE-STACK avoids any reconvergence delay	64
4.9	Update and recovery: (a) Update phases, (b) Recovery cost at each phase .	66
4.10	Memory footprint: (a) Flash memory, (b) RAM memory	67
4.11	Power Consumption	68
4.12	Execution Time	69
5.1	VF composition in the FINE-STACK network stack to realize different control plane deployments: (a) Distributed, (b) Centralized, and (c) Hybrid.	74
5.2	Proposed hybrid deployment in FINE-STACK.	75
5.3	Topology on SLICES-FR. The network includes 46 M3 nodes; one node serves as the Border Router (BR), providing connectivity to the orchestrator and acting as the RPL root in the distributed deployment.	76
5.4	CDF of node update times across control plane deployments. Centralized mode is fastest with compact rule-only VFs, distributed mode is slower due to larger logic-bearing VFs, and hybrid mode is slowest because it installs logic-bearing VFs and a State VF.	79
5.5	CDF of node convergence time after updates. Distributed mode shows long tails due to control plane re-exchange, while centralized and hybrid modes converge rapidly through rule installation or state restoration.	80
5.6	Maintenance recovery time across control plane deployments. Recovery increases with load, with centralized control degrading most due to congested multi-hop rule dissemination. Distributed and hybrid modes recover faster through local fallback.	81
6.1	PDR under sporadic and bursty traffic regimes	86
6.2	Closed-loop control framework. It consists of two phases: Offline training and Online closed-loop control.	87
6.3	GoMacH configuration parameters.	88
6.4	Controlled topology used to evaluate the framework	98
6.5	Measured versus predicted trajectories for representative nodes: (a) PDR and (b) duty cycle.	99
6.6	Impact of the calibration interval k on forecasting error. The error increases as k becomes larger, but remains within an acceptable range.	100

6.7	PDR comparison under three scenarios: (a) no recommendation and no forecasting, (b) recommendation without forecasting, and (c) recommendation with forecasting.	102
6.8	PDR comparison under mixed traffic profile: (a) without TDMA and (b) with dynamic TDMA.	103
6.9	Global average PDR in the 50-node heterogeneous topology under both closed-loop dynamic control and static configuration.	105

List of Tables

3.1	Summary comparison of dynamic code update approaches in LLWNs. . . .	35
4.1	Update size comparison between FINE-STACK and native implementation. FINE-STACK reduces the update size by 94.9%	63
4.2	Update overhead cost. The cost in FINE-STACK is multiplied by the number of updated VFs.	65
5.1	Update size among control plane deployments.	78
6.1	Dataset collection parameters.	89
6.2	Cross-validation MAE (%) of the prediction and recommendation models. .	96
6.3	Traffic profiles used in the evaluation.	98
6.4	Comparison of performance under No TDMA, Always-on TDMA, and Dynamic TDMA.	104

List of Abbreviations

6LoWPAN	IPv6 over Low-Power Wireless Personal Area Networks
6TiSCH	IPv6 over Time-Slotted Channel Hopping
AI	Artificial Intelligence
API	Application Programming Interface
ARQ	Automatic Repeat Request
BR	Border Router
CoAP	Constrained Application Protocol
CP	Contention Period
CSMA/CA	Carrier Sense Multiple Access with Collision Avoidance
DAO	Destination Advertisement Object
DAO-ACK	Destination Advertisement Object Acknowledgment
DC	Duty Cycle
DIO	DODAG Information Object
DIS	DODAG Information Solicitation
DODAG	Destination-Oriented Directed Acyclic Graph
FC	Femto-Container
GBRT	Gradient Boosting Regression Trees
HGBT	Histogram-Based Gradient Boosting Trees
IEEE	Institute of Electrical and Electronics Engineers
IETF	Internet Engineering Task Force
IoT	Internet of Things

IP	Internet Protocol
IPv6	Internet Protocol Version 6
KNN	K-Nearest Neighbors
LASSO	Least Absolute Shrinkage and Selection Operator
LLN	Low-Power and Lossy Network
LLWN	Low-Power and Lossy Wireless Network
MAC	Medium Access Control
MAE	Mean Absolute Error
ML	Machine Learning
OS	Operating System
OTA	Over-the-Air
PDR	Packet Delivery Ratio
QoS	Quality of Service
RAM	Random Access Memory
RIOT	RIOT Operating System
ROM	Read-Only Memory
RPL	Routing Protocol for Low-Power and Lossy Networks
RSSI	Received Signal Strength Indicator
RTOS	Real-Time Operating System
SDN	Software-Defined Networking
SF	Superframe
SUIT	Software Updates for the Internet of Things
SVR	Support Vector Regression
TDMA	Time Division Multiple Access
TSCH	Time-Slotted Channel Hopping
UDP	User Datagram Protocol
VF	Virtual Function

VM	Virtual Machine
WSN	Wireless Sensor Network
XGBoost	Extreme Gradient Boosting

Introduction

Chapter contents

1.1	Economic Impact of LLWNs	3
1.2	Open Scientific Issues in LLWN	4
1.3	Thesis Organization and Contributions	6
1.4	List of Publications	8

The Internet of Things (IoT) has extended Internet connectivity beyond traditional computing devices and mobile smartphones to embedded sensing and actuating devices that interconnect wirelessly. This new category of devices has enabled a wide range of application domains driven by the need to monitor diverse environments, detect changes, and react to them. At the same time, these devices exhibit specific characteristics imposed by their typically low cost and large-scale deployment [1]. A fundamental characteristic of these devices is their power constraint, as they are often battery-powered and must operate for extended periods. In addition, they are constrained in memory and processing capacity because of their limited power budget and small physical size. These limitations, in turn, make their wireless communication inherently constrained in terms of throughput and transmission range. In this context, the network must operate under severe resource constraints while still providing reliable communication, energy efficiency, and long-term autonomy [2].

Low-Power and Lossy Wireless Networks (LLWNs), or more generally Low-Power and Lossy Networks (LLNs), represent the communication infrastructure of such deployments. The IETF defines LLNs as networks where both routers and the links interconnecting them are constrained. LLN routers typically operate with limited resources, while their links are characterized by low data rates, instability, and high loss rates. These networks may range from a few dozen to thousands of nodes and commonly rely on multi-hop communication,

where nodes cooperate to deliver traffic toward a destination, such as a sink or border router. As a result, each node can act both as a source of its own data and as a forwarder for packets generated by neighboring nodes [3].

LLWNs are used in a wide range of application domains, beginning with environmental monitoring and early disaster response, such as forest-fire protection and flood detection. They also play an important role in industrial automation by supporting production optimization, and in healthcare by enabling continuous patient monitoring and rapid intervention [4]. Smart cities represent one of the fastest-growing application domains, supporting automated and efficient management of parking, streets, waste, energy, buildings, and homes [5].

1.1 Economic Impact of LLWNs

The economic importance of LLWNs is linked to the broader growth of IoT and connected embedded systems. Figure 1.1 reports that the number of connected IoT devices reached 18.6 billion in 2024 and is projected to increase to 55 billion by 2035 [6]. This growth reflects the increasing deployment of connected objects in industrial, urban, environmental, agricultural, and healthcare systems.

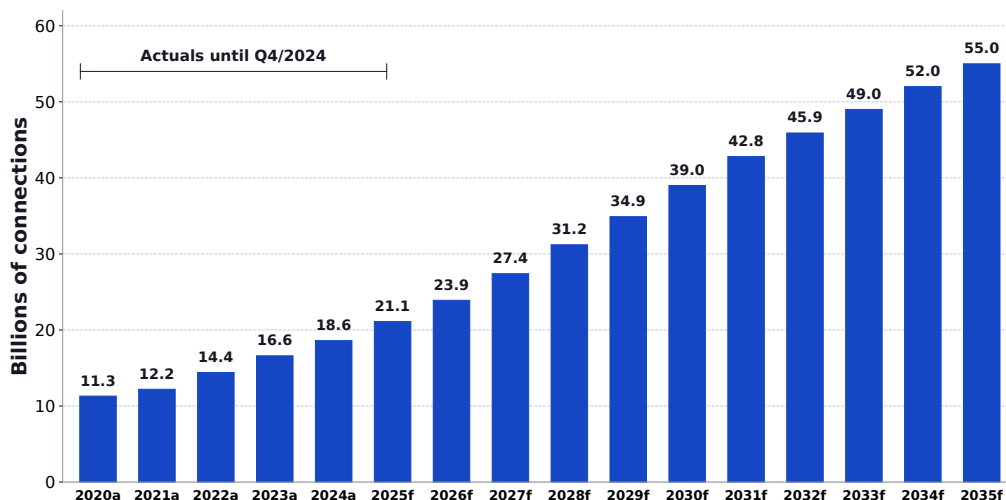


Figure 1.1: Number of connected IoT devices between 2020 and 2035 [6]

The economic potential of IoT is also substantial. McKinsey estimates that IoT could enable between USD 5.5 trillion and USD 12.6 trillion in global economic value by 2030, depending on the ability of organizations and industries to capture value from connected systems [7]. A large part of this potential relies on the deployment of sensors and actuators in physical environments, where LLWNs provide low-cost and energy-efficient communication for distributed monitoring and control. More specialized markets further illustrate the relevance of LLWN technologies. The global sensors and actuators market was valued at USD 13.5 billion in 2024 and is estimated to grow at an annual rate of 17.5% between 2025

and 2034 [8].

These figures show that LLWN-like deployments are not simply a research topic, but key enablers of the global technology economy, supporting industrial digitalization, smart infrastructure, predictive maintenance, energy optimization, and large-scale environmental sensing. The deployment of thousands of constrained devices becomes economically meaningful only if they can remain operational for years, adapt to evolving requirements, and preserve reliable communication under changing conditions. Therefore, improving the ability of LLWNs to evolve after deployment is a fundamental requirement for both technical and economic sustainability.

1.2 Open Scientific Issues in LLWN

The wireless nature of LLWNs makes communication inherently dynamic. This dynamicity arises from environmental changes, mutual interference among devices, power depletion, and mobility requirements in some applications, all of which cause topology variations and affect communication performance, making communication links unstable and prone to high packet loss [9]. In addition, LLWN applications exhibit various Quality of Service (QoS) requirements, and each application can generate variable traffic rates depending on environmental events [10]. These dynamic environmental conditions, combined with the diversity of QoS requirements and traffic patterns, require network protocols to be reconfigured. As a result, most software updates in LLWNs target the network stack to enhance performance [11]. Despite their economic and social importance, LLWNs remain difficult to maintain and optimize after deployment. Their design is often rigid and hardware-centric, while operating conditions, traffic demands, and resource availability change continuously [12].

LLWNs face three main research issues in network stack updates that motivate this thesis: efficient runtime network updates, control and decision-making placement, and resource-constrained performance optimization.

Efficient runtime network updates. First, efficient network stack reconfiguration is a major challenge because LLWN devices are highly constrained in memory, processing capacity, energy supply, and communication throughput. These constraints have historically encouraged compact and tightly integrated network stacks, in which protocol logic is bound at compile time [13]. However, such designs become rigid once deployed. Even minor protocol changes, such as modifying routing behavior, adding a reliability mechanism, or tuning a Medium Access Control (MAC)-layer function, may require replacing entire integrated software components, including those that have not been modified. LLWNs usually rely on Over-the-Air (OTA) mechanisms to deliver such updates remotely, avoiding physical access to deployed devices and reducing operational and maintenance costs [14]. However, conventional OTA updates become inefficient in LLWNs when they involve

transmitting large binaries over throughput-limited, lossy, and often multi-hop links that must also carry application traffic [15]. In addition, such updates are semantically blind, as they are performed without explicitly reasoning about the role of the updated protocol functions, their dependencies on other stack components, or the runtime state that must be preserved. This lack of semantic awareness may lead to inconsistent protocol behavior, connectivity interruptions, and temporary service disruption. Consequently, conventional updates can increase service downtime, energy consumption, network congestion, and the probability of update failure.

Control and decision-making placement. Second, the placement of the control plane, which is responsible for packet-forwarding decisions, remains an open question in LLWNs. Early LLWN deployments adopted a distributed control plane approach in which each node hosts its own local decision-making and forwarding functionalities. Nodes exchange control information to construct and maintain local decisions, which makes them autonomous and more robust to failures and errors. However, this deployment suffers from slow convergence, local sub-optimality, and costly traditional updates when the control logic must be modified across many nodes [16]. More recent approaches move decision-making to a central entity while leaving only forwarding functionalities on the nodes [17]. This approach improves the programmability of the control plane by updating only the central entity hosting the control logic, provides a global network view, and enables network-wide optimization. Its main limitation, however, is its dependence on a reliable control path between the central control plane and the constrained nodes. In LLWNs, this path is itself lossy, throughput-limited, and often multi-hop, which can delay the dissemination of decisions. Therefore, no single control plane deployment model remains optimal under all network conditions [18].

Resource-constrained performance optimization. Third, performance optimization under resource constraints is difficult because LLWNs must constantly balance reliability and latency against energy efficiency under changing operating conditions. Radio transmission and reception represent major sources of power consumption in LLWN devices [19], making periodic sleeping phases necessary to extend device lifetime. Increasing reliability and reducing latency generally require longer radio activity periods. In contrast, aggressive sleeping saves energy, but may degrade reliability and increase latency when traffic increases. This trade-off becomes even more complex because LLWN conditions are not stationary [20]. Application traffic may alternate between sporadic monitoring and intensive event-driven phases, while link quality and forwarding roles in the multi-hop topology may also evolve over time. A static network configuration may therefore be efficient under certain conditions, but inadequate under others [21]. This creates the need for adaptive mechanisms capable of reconfiguring network operation while respecting the strict resource constraints of LLWN devices.

Together, these three issues show that LLWNs require more than isolated protocol-

level improvements. They call for programmable and adaptive network solutions capable of reconfiguring the network stack at runtime, ranging from parameter tuning to the replacement, composition, or switching of protocol functions. To be effective in constrained deployments, such architectures must support efficient runtime OTA updates, flexible control plane deployment, and continuous performance optimization while respecting these constraints.

These issues motivate the central problem addressed in this thesis: **how to make LLWNs programmable, adaptable, and self-reconfigurable while respecting the strict resource constraints of embedded wireless devices?** This central problem is decomposed into three research questions, which are addressed through a progressive set of contributions presented in the next section.

1.3 Thesis Organization and Contributions

The remainder of this thesis is organized as follows.

Chapter 2 presents the background required for this work. It introduces LLWNs by explaining their main constraints and application domains, together with the different traffic regimes they generate. The chapter then describes the main protocol-stack components used in LLWNs, with particular emphasis on the protocols studied in this thesis. Finally, it discusses medium-access mechanisms in LLWNs and compares their respective strengths and limitations.

Chapter 3 reviews the state of the art along three main directions. First, it examines existing network-update solutions and discusses their limitations and lack of suitability for LLWN constraints. Second, it reviews the available control plane deployments and compares their strengths and limitations, thereby motivating hybrid deployments. Hybrid deployments are then analyzed through their different realizations in wired networks, wireless networks, and LLWNs. Finally, the chapter studies adaptive wireless networks, with a particular focus on the MAC layer and machine-learning-based solutions for automating wireless network management.

Chapter 4 presents the first scientific contribution of the thesis by addressing the following questions: **How can the network stack of constrained LLWN devices be made programmable without violating their memory, energy, and processing constraints? To what extent can a modular network-stack design make updates selective in order to reduce update size, update time, and failure probability?** The chapter proposes a new LLWN architecture based on a network stack built from lightweight virtual functions, providing modularity at the network-function level. It also evaluates the achieved gains in terms of update size and update time while ensuring compliance with LLWN resource constraints in memory, power consumption, and processing overhead.

Chapter 5 focuses on the following questions: **How can LLWNs support flexible control plane deployment models, including distributed, centralized, and hybrid control? How can runtime updates to LLWN network stacks be made semantically aware and state-preserving?** This chapter constitutes the second scientific contribution of the thesis. It extends the proposed architecture to support enhanced versions of distributed and centralized control plane deployments and introduces a hybrid deployment that addresses the limitations of both approaches while enabling stateful network updates. It then demonstrates the superiority of hybrid deployments through different evaluation scenarios comparing them with fully distributed and fully centralized control plane deployments.

Chapter 6 presents the third scientific contribution of the thesis by addressing the following question: **How can LLWNs become self-adaptive and provide proactive reconfiguration of the network stack?** This chapter develops a closed-loop framework for MAC-layer adaptation in multi-hop LLWNs. The framework combines runtime monitoring, short-horizon forecasting of future network performance, and online reconfiguration of the network stack to maintain the required performance. It responds proactively to changes in network conditions and traffic regimes while preserving the required QoS and minimizing power consumption.

Chapter 7 concludes the thesis by summarizing the main findings, discussing the limitations, and outlining future research directions.

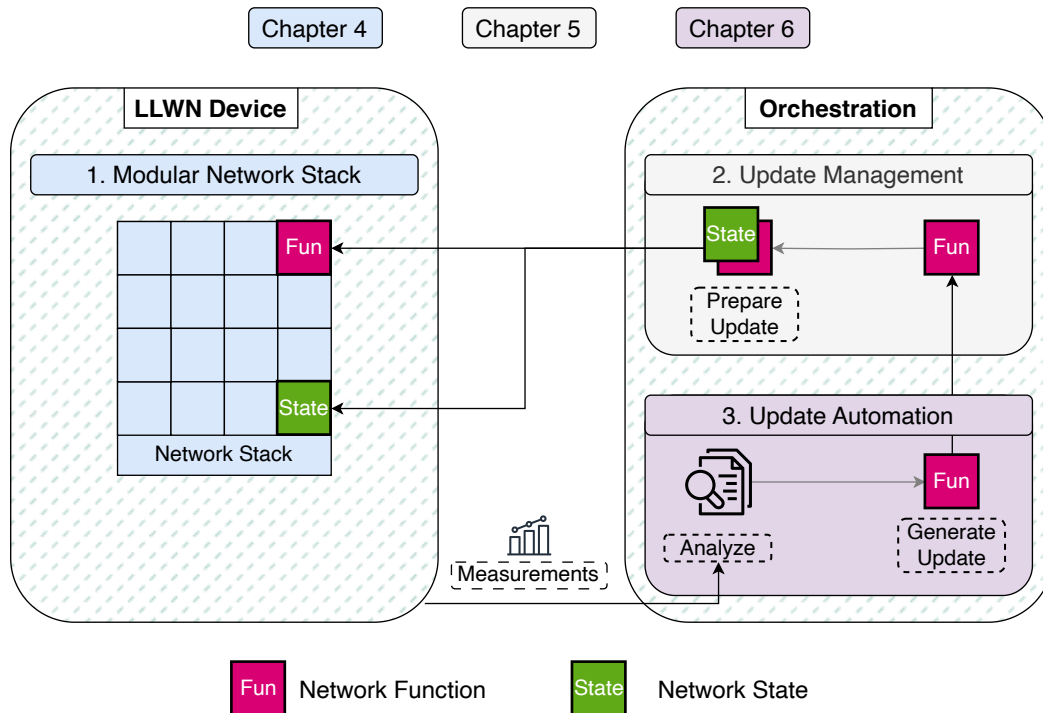


Figure 1.2: Thesis contributions

All three contributions are evaluated through real-world experimental scenarios, thereby validating the proposed solutions under realistic LLWN operating conditions and across diverse network topologies and traffic regimes.

In conclusion, the contributions of this thesis are distributed between LLWN devices and the network orchestration, as illustrated in Figure 1.2. The first contribution focuses on enabling devices to accept selective updates through a modular design at the function level. This design enables frequent and fast updates with a high probability of success in order to maintain performance or achieve a new performance level. The second contribution addresses orchestration by introducing semantic management capabilities to support modular updates, including update preparation, state augmentation, and distributed deployment across devices via hybrid control plane architectures. The third contribution extends orchestration with intelligence to automate reconfiguration decisions according to the required performance level and the current operating conditions. Taken together, these three contributions provide a closed-loop framework for automating network updates. The loop begins with collecting performance metrics and environmental conditions, analyzing them, and triggering network updates accordingly. These updates are then prepared for distribution and can be enriched with state information to ensure transparent installation in the modular network stacks of the devices.

1.4 List of Publications

The work conducted during this thesis has led to the following publications and manuscripts.

International Journals

- ◇ **Ahmad Mahmood**, Julien Montavont, and Thomas Noel. “FINE-STACK: Flexible Architecture for Low-Power Lossy IoT Networks with Programmable Network STACK and Semantic Runtime Updates.” *Elsevier Ad Hoc Networks*, volume 192, 2026.
- ◇ **Ahmad Mahmood**, Julien Montavont, and Thomas Noel. “PENSIL: Programmable Network Stack for Low-Power Lossy IoT Networks Using Lightweight Virtualization.” *Elsevier Internet of Things*, volume 35, 2026.

International Conferences

- ◇ **Ahmad Mahmood**, Julien Montavont, and Thomas Noel. “AI-Enabled Traffic-Aware Closed-Loop Framework for MAC-Layer Control in Multi-Hop IoT Networks.” *The International Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems (MSWiM)*, Accepted, 2026.
- ◇ **Ahmad Mahmood**, Julien Montavont, and Thomas Noel. “Programmable Solutions for Low-Power Lossy Wireless Networks: A Study of SDN and Femto Containers.”

The International Conference on Advanced Information Networking and Applications (AINA), pp. 46–57, 2025.

National Conferences

- ◇ **Ahmad Mahmud**, Julien Montavont, and Thomas Noel. “Divide and Conquer: A Semantic and Modular Reconfiguration of the Network Stack for Constrained Wireless Networks.” *CoRes 2026–11es Rencontres Francophones sur la Conception de Protocoles, l’Évaluation de Performances et l’Expérimentation des Réseaux de Communication*, 2026. **Best Paper Award**.
- ◇ **Ahmad Mahmud**, Julien Montavont, and Thomas Noel. “Menu or à la carte? An Architecture for Programming the Data Plane of Constrained Wireless Networks.” *CoRes 2024–9èmes Rencontres Francophones sur la Conception de Protocoles, l’Évaluation de Performance et l’Expérimentation des Réseaux de Communication*, 2024.

Background

Chapter contents

2.1	Low-Power and Lossy Wireless Networks	11
2.1.1	Characteristics of LLWNs	12
2.1.2	Applications and Traffic Regimes	13
2.2	Communication Protocol Stack for LLWNs	15
2.2.1	Protocol Stack Overview	15
2.2.2	Functional Decomposition of Network Operation	18
2.3	Medium Access Control in LLWNs	20
2.3.1	Contention-free MAC Protocols	20
2.3.2	Contention-based MAC Protocols	22
2.4	Conclusion	23

2.1 Low-Power and Lossy Wireless Networks

Low-power and lossy wireless networks (LLWNs) are a fundamental communication substrate for the Internet of Things (IoT). Figure 2.1 illustrates a representative LLWN composed of constrained wireless nodes, which act as sensors or actuators, and a border router, or sink, responsible for connecting the network to the Internet. These nodes communicate over lossy and throughput-limited wireless links and often rely on multi-hop transmission, where intermediate nodes forward packets toward the sink. LLWNs are deployed in domains where conventional communication infrastructure is costly, energy-demanding, or difficult to install, including environmental monitoring, building automation, smart cities, industrial supervision, healthcare, and critical monitoring [22]. In these

settings, the network must operate for long periods with little human intervention while supporting sensing, reporting, actuation, and control. Unlike conventional wireless networks, which often prioritize connectivity and throughput, LLWNs must provide reliable communication under severe constraints in energy, memory, processing capacity, and link quality.

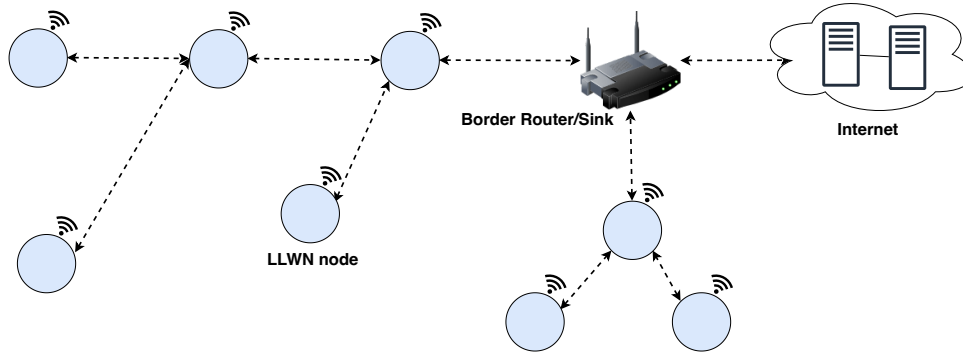


Figure 2.1: Representative LLWN consisting of constrained nodes and a gateway

2.1.1 Characteristics of LLWNs

A first defining property of LLWNs is device constraints. Nodes are typically low-cost, small, and battery-powered, and are equipped with low-power radios and microcontrollers that provide limited processing power, RAM, and Flash memory. These limitations directly shape protocol design: large routing tables, complex algorithms, heavy security mechanisms, and memory-intensive protocol implementations may be impractical or may shorten network lifetime [23]. Limited buffering also becomes critical under retransmissions or intensive traffic, while limited processing capacity restricts the complexity of runtime decision-making and packet processing. Consequently, LLWN protocols must remain lightweight in both code size and execution cost [24].

A second property of LLWNs is low-rate, short-range communication. Many LLWNs rely on bandwidth-constrained links that offer modest bit rates, small frame sizes, and low transmission power. These characteristics make the direct use of conventional Internet protocols challenging because such protocols assume larger frame sizes than LLWN links can natively support. Adaptation mechanisms such as 6LoWPAN are therefore required to enable Internet Protocol (IP)-based communication over LLWNs through header compression, fragmentation, and adaptation-layer forwarding. However, this also means that even basic packet delivery requires careful management of protocol overhead, buffering, fragmentation, and retransmission costs [23].

Because low-power radios have limited communication range, LLWNs commonly rely on multi-hop forwarding to cover larger areas without increasing transmission power. While multi-hop communication extends coverage at low energy cost, it also introduces routing maintenance overhead, forwarding load, and topology-dependent bottlenecks [25]. As a

result, nodes play different roles depending on their position in the routing topology. Leaf nodes mainly generate and transmit their own data, whereas intermediate forwarding nodes, especially those located close to the sink or located at the end of a transmission path, must relay both their own traffic and the traffic generated by other nodes. These unequal forwarding responsibilities lead to significant differences in reliability, latency, queue occupancy, and energy consumption across the network [26]. As a result, LLWN performance cannot be assessed solely through global averages, since such averages hide the effects of node role, hop distance, forwarding burden, and local link quality.

LLWNs commonly operate in shared frequency bands and are characterized by lossy and unstable links. Low-power wireless communication is affected by interference, fading, obstacles, asymmetric links, and external traffic in the shared spectrum [10]. As a result, link quality may vary over time, causing transmission failures, busy-channel events, retransmissions, and packet drops. These effects reduce reliability and consume energy even when no useful data is successfully delivered [24]. Similarly, LLWN topology is dynamic: even when nodes remain physically fixed, logical routes evolve as routing protocols recompute next hops based on changing link metrics, requiring continuous adaptation while limiting control overhead [27].

These characteristics create a persistent reliability–energy trade-off. Reliability requires mechanisms such as channel listening, acknowledgments, and retransmissions, whereas energy efficiency requires limiting radio activity. Since the radio is one of the dominant energy consumers in low-power nodes [19], increasing communication opportunities can shorten network lifetime, while aggressive sleeping can increase latency, collisions, queue drops, and packet loss [26]. The same constraints also affect software evolution: deployed nodes may require updates to fix faults, patch vulnerabilities, adapt to new requirements, or respond to changing conditions. However, the limited bandwidth, restricted memory, lossy links, and the risk of state loss make conventional OTA firmware replacement costly [13].

2.1.2 Applications and Traffic Regimes

LLWNs support a broad range of Internet of Things (IoT) applications in which constrained embedded devices observe, report, and sometimes react to physical-world phenomena. These applications are deployed in domains such as environmental monitoring, smart buildings and cities, industrial supervision, and healthcare, where wired infrastructure is often costly, difficult to maintain, or physically impractical, and where devices must operate for long periods under strict energy, memory, and processing constraints [28, 29]. Although these domains share similar communication constraints, they differ significantly in their sensing objectives, reporting frequency, and performance requirements, including latency, reliability, and tolerance to packet loss. As a result, LLWN traffic cannot be represented by a single uniform model. Instead, it includes periodic low-rate reports, such as temperature, energy, or environmental measurements sent at regular intervals; sporadic event-triggered messages, such as motion detection, access-control notifications, or occasional alarms; and bursty

traffic, where several packets are generated within a short time due to correlated events such as fires, floods, industrial faults, emergency situations, or infrastructure failures [30, 31, 32, 33].

These examples show that LLWN traffic is shaped both by the monitored phenomenon and by the application objective. **Periodic low-rate** sensing is a common baseline regime. In this case, nodes generate small packets at regular intervals, making packet arrivals predictable and channel access easier to configure. When transmissions are distributed over time, contention remains moderate, queues are less likely to overflow, and low-power MAC protocols can keep the radio asleep for long periods between scheduled or expected transmissions. **Sporadic traffic**, by contrast, is generated irregularly and only when occasional events occur. Although it is often low-rate over the long term, it is less predictable than periodic traffic because packet generation does not follow a fixed interval. In this traffic regime, MAC protocols can also keep the radio asleep for long periods, but they must remain responsive to irregular event-triggered transmissions. **Bursty traffic** is more difficult to handle because many packets are generated within short high-intensity intervals separated by idle periods. Although the long-term average rate may remain low, the instantaneous load can exceed the service capacity of a low-power medium-access scheme. Queues then build up, especially at intermediate forwarding nodes, and limited buffers may lead to packet drops before transmission. As a result, a communication protocol configuration selected for average traffic may become unreliable during bursts, even if it performs well under periodic or sporadic operation [30].

The coexistence of periodic, sporadic, and bursty regimes is therefore central to LLWN protocol design. Periodic traffic favors predictable access and efficient low-power operation, while sporadic traffic requires responsiveness to irregular event-triggered transmissions without wasting energy during long idle periods. Bursty traffic, in contrast, requires temporary increases in service capacity and improved channel access mechanisms to limit collisions under high contention [31]. A worst-case configuration may preserve reliability during bursts, but waste energy during normal periodic or sporadic operation. Conversely, an energy-optimized configuration may be efficient under periodic or sporadic low-rate traffic, but fail when sudden events generate correlated packet arrivals. The appropriate operating point is therefore application-dependent, traffic-dependent, topology-dependent, and time-varying [34].

In summary, LLWNs support a wide range of application domains that differ not only in what they monitor, but also in when and how they generate traffic. Some applications produce regular periodic low-rate reports, others generate sporadic event-triggered messages after long silent periods, and some produce bursty traffic when events cause many packets to be generated within a short time. These behaviors can also coexist within the same deployment, where traffic patterns evolve over time, making traffic-aware communication protocol tuning essential for maintaining reliability and energy efficiency.

2.2 Communication Protocol Stack for LLWNs

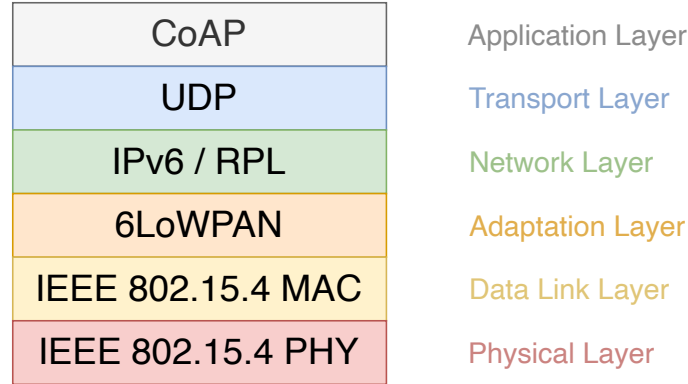


Figure 2.2: LLWN network protocol stack

LLWNs can rely on different protocol families and ecosystems. In this thesis, the focus is placed on the IP-based protocol stack proposed and supported by the IETF for constrained low-power networks. This stack follows the layered Internet architecture, but each layer must be adapted to the constraints of low-power devices, small frame sizes, low data rates, lossy links, and multi-hop routing. A commonly adopted standardized stack, illustrated in Figure 2.2, combines IEEE 802.15.4 at the physical and MAC layers, 6LoWPAN as an adaptation layer, Internet Protocol Version 6 (IPv6) at the network layer, Routing Protocol for Low-Power and Lossy Networks (RPL) for routing, User Datagram Protocol (UDP) at the transport layer, and Constrained Application Protocol (CoAP) at the application layer [35, 36, 37]. However, this representation should be understood as a simplified view of the broader IETF ecosystem, which may also include additional sublayers or protocols for schedule management, end-to-end security, and network joining. This stack provides the baseline architecture on which this thesis examines programmability, runtime updates, and adaptive MAC-layer control.

2.2.1 Protocol Stack Overview

This subsection introduces the protocols commonly adopted in LLWNs at each layer of the communication stack.

2.2.1.1 Physical and MAC Layers

At the physical and data-link layers, IEEE 802.15.4 [38] provides low-power, low-cost, and short-range communication for low-rate wireless personal area networks. It defines radio characteristics, frequency bands, modulation schemes, data rates, frame formats, acknowledgments, and retransmission mechanisms. IEEE 802.15.4 includes several MAC operation modes. Contention-based access relies on CSMA/CA, where nodes sense the

medium, back off, and transmit when the channel becomes available. This mode is simple and widely supported, but it remains sensitive to contention, collisions, and retransmissions, especially when several nodes transmit simultaneously. More recent IEEE 802.15.4 evolutions introduced deterministic time-slotted operation, notably TSCH, which combines time synchronization, channel hopping, and scheduled transmissions.

2.2.1.2 Adaptation and Network Layers

The small IEEE 802.15.4 frame size is incompatible with the assumptions of native IPv6 packets. IPv6 over Low-Power Wireless Personal Area Networks (6LoWPAN) [39] addresses this mismatch by introducing an adaptation layer between IEEE 802.15.4 and IPv6. It enables IPv6 over constrained links through header compression, fragmentation, and reassembly. On top of 6LoWPAN, IPv6 provides addressing, interoperability, and end-to-end network-layer semantics, allowing constrained devices to integrate with existing IP infrastructure [40]. However, this interoperability comes with additional constraints at the adaptation layer. When packets cannot fit into a single IEEE 802.15.4 frame, 6LoWPAN fragmentation increases transmission overhead, loss sensitivity, and reassembly complexity.

2.2.1.3 Routing in LLWN

Routing is provided primarily by RPL [41]. RPL organizes the network as a Destination-Oriented Directed Acyclic Graph (DODAG) rooted at a sink or border router, which serves as a collection point and often provides connectivity to external IPv6 networks [42]. The DODAG is built through the exchange of control messages. The root initiates topology formation by disseminating DODAG Information Object (DIO) messages that advertise the DODAG information. Upon receiving DIO messages from neighboring nodes, a node computes its rank, identifies one or more candidate parents, and selects a preferred parent according to the objective function. In RPL, the rank is a scalar value that represents a node's relative position within the DODAG with respect to the root. Rank increases as nodes move farther away from the root according to the routing metric being used, and it helps prevent routing loops by ensuring that packets are forwarded toward nodes with lower ranks. The objective function defines how ranks are computed and how parents are selected. It specifies the routing metrics and constraints to consider, such as hop count, link quality, latency, reliability, or energy consumption, thereby determining the criteria used to build and optimize routing paths. Through this process, upward routes are progressively established from all nodes toward the root. Nodes may also send DODAG Information Solicitation (DIS) messages to request DIO information from neighbors, for example when joining the network or when local routing information becomes stale.

Once the DODAG is formed, upward routing is naturally supported by forwarding packets from each node to its preferred parent until they reach the root. This traffic direction is common in LLWN monitoring applications, where sensors report measurements

to a sink. Downward routing, from the root to the internal nodes, requires additional route construction. For this purpose, RPL uses Destination Advertisement Object (DAO) messages. A node sends DAO information to advertise reachability to itself and to destinations in its sub-DODAG, depending on the operating mode. DAO messages may be acknowledged through Destination Advertisement Object Acknowledgment (DAO-ACK) messages, which help to confirm the installation of the downward routing state [41]. Thus, while DIO messages primarily support DODAG formation and maintenance, DAO messages enable the establishment of downward routes.

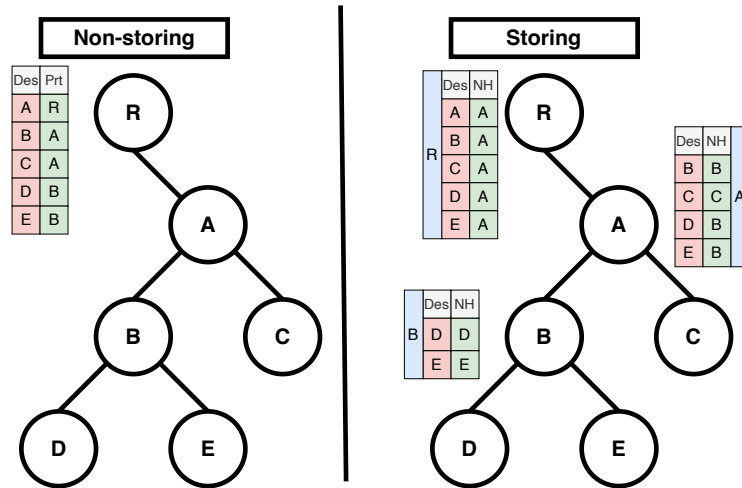


Figure 2.3: Comparison of RPL non-storing and storing modes of operation

RPL defines two main modes for downward routing, namely non-storing and storing, as illustrated in Figure 2.3. In non-storing mode, intermediate nodes do not maintain downward routing tables for their descendants. Instead, DAO messages are sent upward to the root, which becomes responsible for storing downward topology information. When the root needs to send a packet to a node, it includes the route in the packet header using source routing. This reduces memory requirements on intermediate nodes and simplifies their forwarding state, but it increases the responsibility of the root and may introduce additional header overhead, especially over long multi-hop paths. In addition, the packet path may become non-optimal, since downward traffic must always traverse the root so that the source route can be inserted [42].

In storing mode, downward routing state is distributed across the network. Nodes advertise reachability information through DAO messages, and intermediate routers may store routing entries for destinations located in their sub-DODAG. Downward packets can then be forwarded hop by hop using local routing tables [41]. This reduces source-routing overhead, but may create memory pressure on constrained intermediate nodes, especially those close to the root or those with many descendants, as they must aggregate and maintain multiple routes. The choice between storing and non-storing modes therefore reflects a trade-off among distributed memory usage, scalability, root complexity, packet

overhead, and path length.

In addition to these upward and downward routing operations, RPL also supports point-to-point (P2P) communication between nodes within the LLWN to establish more direct routes between communicating nodes when needed, reducing path length and latency for specific communication patterns.

2.2.1.4 Transport and Application Layer

At the transport layer, LLWNs commonly use UDP because it avoids connection establishment, congestion-control state, retransmission buffers, and complex reliability mechanisms, making it well suited to small independent messages [43]. UDP delegates reliability to other layers, so packet loss must be handled at the link layer, the application layer, or through protocol-specific mechanisms. At the application layer, CoAP provides a compact REST-like model over UDP, with methods such as GET, POST, PUT, and DELETE, together with optional lightweight reliability based on confirmable messages and acknowledgments [44]. This makes CoAP suitable for sensing, configuration, and management operations.

Taken together, IEEE 802.15.4, 6LoWPAN, IPv6, RPL, UDP, and CoAP form a protocol stack that enables Internet interoperability while accommodating the constraints of low-power wireless networks. Moreover, this communication protocol stack is often compiled into monolithic firmware, so modifying a routing behavior, a transport reliability mechanism, or a MAC parameter may require rebuilding and redistributing a large binary.

2.2.2 Functional Decomposition of Network Operation

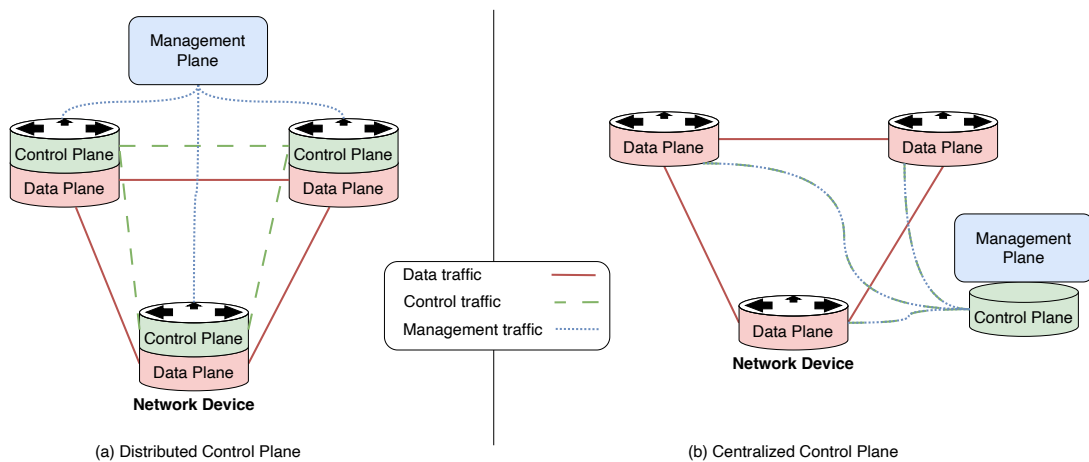


Figure 2.4: Control Plane Deployments: (a) Distributed, (b) Centralized

General network architectures, and therefore LLWN protocol architectures, can be analyzed through three planes: the data plane, the control plane, and the management plane. This separation clarifies which functions process packets, which functions decide

how packets are forwarded or scheduled, and which functions supervise, configure, and update the network [45].

The **data plane** directly handles data packets. It receives frames from the radio, parses headers, performs local packet processing, follows the outgoing interface or next hop selected by the control plane, and transmits packets. In LLWNs, this includes IEEE 802.15.4 frame handling, 6LoWPAN adaptation, and IPv6 forwarding. Because nodes are constrained, data-plane operations must remain simple, fast, and memory-efficient; even small increases in packet-processing complexity can affect latency, energy consumption, and feasibility on constrained devices [46].

The **control plane** determines how the data plane should behave. It constructs and maintains routes, selects next hops, computes scheduling decisions, allocates resources, and defines policy rules. Depending on its placement, two traditional deployment models can be distinguished: distributed and centralized control planes, as illustrated in Figure 2.4. In conventional distributed LLWNs, control plane logic is embedded in each node. RPL is a representative example of a distributed routing control plane: nodes exchange control messages, compute ranks, select next hops, and maintain routing state. Contention-based MAC protocols also include local control decisions, such as when to transmit, back off, or retry. This distributed placement provides autonomy and resilience, but it limits global optimization because nodes act mainly on the basis of local information and exchange control information over the same unreliable links used by the data plane [47].

In a centralized control plane deployment, the control plane is separated from the data plane by placing decision-making in a logically centralized controller that installs rules in the nodes [48]. Centralization improves programmability by concentrating control modifications in a single entity and it enables global optimization, but it is difficult to realize efficiently in LLWNs because state collection and rule dissemination must traverse low-power, lossy, multi-hop links. A purely centralized control plane may therefore become fragile under congestion, interference, or node failures [49].

The **management plane** observes, configures, updates, and orchestrates the network. Whereas the control plane determines how the network behaves, the management plane evaluates whether this behavior satisfies operational objectives. It includes monitoring, telemetry collection, configuration management, software update delivery, fault handling, policy enforcement, and runtime orchestration [50]. This plane is essential in long-lived deployments where applications, environments, and security requirements evolve. In centralized control plane deployments, the management plane usually coexists centrally with the control plane, and both use the same backhaul link with the devices to transmit control and management data, as illustrated in Figure 2.4.

This plane-based view also exposes the limitations of conventional monolithic stacks, in which data plane processing, control plane logic, and management functions are compiled into a single firmware image. Even a small change may require rebuilding and redistributing the whole stack, without any semantic distinction between parameter tuning, packet-

processing modification, routing update, or management-logic replacement.

2.3 Medium Access Control in LLWNs

The MAC layer plays a central role in LLWN performance because it directly determines when nodes can access the wireless medium, how long radios remain active, and how effectively traffic can be served under constrained energy budgets. MAC protocols must carefully balance communication opportunities with sleeping periods to reduce energy use while preserving acceptable reliability and latency [51]. Broadly, LLWN MAC protocols can be classified into two main families according to their access mechanism: *contention-free* protocols, which organize transmissions through explicit scheduling and *contention-based* protocols, which rely on probabilistic channel access. The first category provides more predictable access to the medium and stronger service guarantees, whereas the second reduces coordination overhead and remains lightweight.

2.3.1 Contention-free MAC Protocols

Contention-free MAC protocols provide predictable transmission opportunities by organizing medium access in time, frequency, or both. Time Division Multiple Access (TDMA) is the classical example in which time is divided into slots and transmissions are allocated to avoid conflicts [52]. In LLWNs, this approach reduces collisions while also decreasing latency variability and enabling operation close to the maximum achievable service capacity [53].

In TDMA-based access, a node transmits and receives only during its allocated slot and remains silent or sleeps otherwise. This can improve energy efficiency by almost eliminating idle listening and retransmissions.

TDMA is particularly well suited to predictable traffic because the scheduler can allocate transmission slots according to expected packet arrivals. It can also support bursty traffic by temporarily increasing the number of allocated transmission slots during burst periods. Event-driven traffic may create short bursts that contention-based MAC protocols cannot drain efficiently. ATMA, for example, combines advertisement and reservation to support bursty or periodic traffic, reporting energy reductions of up to 75% compared to well-known duty-cycled contention-based MAC protocols while maintaining comparable delivery ratio and latency [54]. Such results illustrate the value of scheduled access when many packets must be served over a short interval.

TSCH, introduced in IEEE 802.15.4, extends contention-free access by combining time slots with channel hopping [38]. As illustrated in Figure 2.5, a scheduled node is assigned a specific time slot on a specific frequency channel. The time-slotted structure provides predictable transmission opportunities, while channel hopping improves robustness against narrow-band interference and multipath fading. This combination is particularly attractive in industrial LLWNs, where reliability, latency, and energy efficiency are critical. IPv6

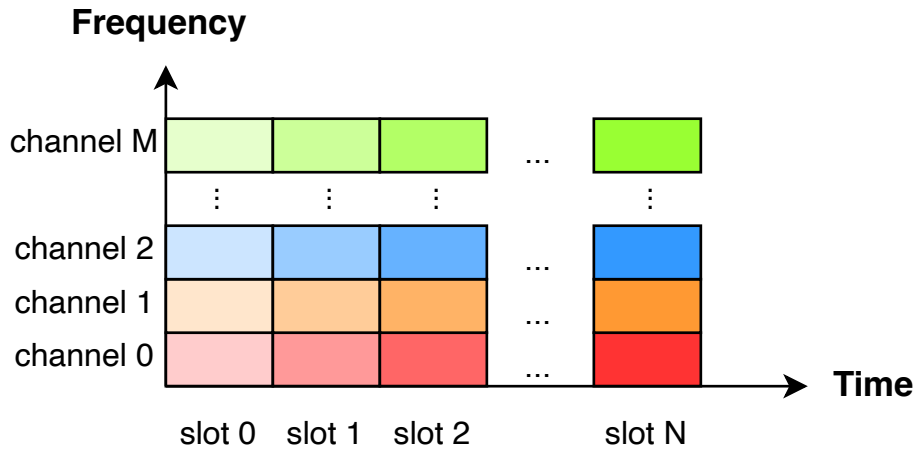


Figure 2.5: TSCH MAC protocol operation

over Time-Slotted Channel Hopping (6TiSCH) further integrates TSCH with IPv6-based deterministic wireless networking [55].

The central challenge in TSCH-like operation is schedule construction and maintenance. A schedule specifies which nodes transmit or receive in each cell, usually identified by a slot offset and a channel offset. A good schedule reduces collisions and idle listening while providing sufficient opportunities for critical traffic. However, IEEE 802.15.4 defines the TSCH mechanism but not how schedules should be generated or adapted [56]. Centralized scheduling can exploit global topology and traffic information, but it requires collecting state and distributing schedules. Distributed scheduling improves autonomy, but may be less optimal and may add negotiation overhead. Weight-based distributed TDMA scheduling illustrates how routing information can be exploited to reduce latency and schedule length, highlighting the close relationship among scheduling, routing, and forwarding load [57].

Contention-free MAC protocols are powerful, but they are not free of cost. Their benefits depend on the ability to construct and maintain a schedule that matches traffic demand. For predictable periodic traffic, this task is relatively straightforward because transmission opportunities can be planned according to known packet-generation intervals. However, under bursty traffic, the schedule must be dynamically adapted so that nodes can reserve additional slots during temporary traffic peaks and release them once the burst ends. Mechanisms such as ATMA illustrate this direction by enabling reservation-based adaptation, but such adaptations are not instantaneous, as they require negotiation and schedule dissemination, which introduces latency and increases overhead.

2.3.2 Contention-based MAC Protocols

Duty-cycle contention-based MAC protocols are a cornerstone of low-power communication in LLWNs. As illustrated in Figure 2.6, these protocols reduce energy consumption by alternating between active periods during which nodes transmit, receive, or listen, and sleep periods, during which the radio is turned off. During the active phase, medium access is typically contention-based, for example, through IEEE 802.15.4 CSMA/CA. This design is motivated by the high energy cost of radio activity and the waste caused by idle listening when a node is neither sending nor receiving data [58]. The fundamental trade-off is that sleeping saves energy, but it also reduces communication opportunities, which can increase latency and degrade reliability.

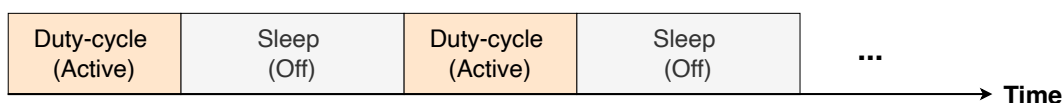


Figure 2.6: Duty-cycled MAC protocol operation

Classical duty-cycled MAC protocols illustrate different strategies for reducing idle listening while maintaining communication opportunities. S-MAC [59] introduced periodic listen/sleep schedules, in which neighboring nodes coordinate common active periods to reduce unnecessary radio activity. T-MAC [60] improved this principle by dynamically shortening the active period when no channel activity is detected, thereby adapting the listening duration to traffic conditions. Protocols also differ in how transmitters and receivers coordinate wake-up times. In synchronous designs, neighboring nodes align their schedules, which reduces the need for long wake-up signaling but introduces synchronization overhead. In asynchronous designs, nodes avoid maintaining a common schedule and instead rely on preamble: a sender transmits a preamble, or a sequence of short preambles, long enough to overlap with the receiver’s periodic channel check. B-MAC [61] follows this low-power listening approach using long preambles, allowing receivers to detect pending transmissions when they wake up. However, long preambles increase energy consumption at the sender and may cause overhearing at non-target receivers. X-MAC [62] addresses this limitation by replacing the long preamble with a sequence of short, addressed preambles, allowing the intended receiver to respond early and allowing non-target nodes to return to sleep. Receiver-initiated protocols such as RI-MAC [63] further reduce sender-side preamble by letting the receiver announce its availability through a beacon, after which waiting senders transmit their data. ContikiMAC [64] further enhances performance by replacing preamble sequences with the data packets themselves. It also includes mechanisms to support consecutive transmissions once a receiver is awake, which improves performance when multiple packets must be delivered in a short interval.

In most duty-cycled protocols, the active period includes a contention interval based on carrier sensing and backoff. The duration of this interval strongly affects performance. A longer contention period gives nodes more time to access the medium, exchange ac-

knowledge, and drain queued packets, thereby increasing service capacity but also increasing the duty cycle. A shorter contention period saves energy, but may be insufficient to serve queued traffic when several nodes contend simultaneously. This expresses the fundamental delay–energy trade-off: longer sleep intervals reduce energy consumption but increase delivery delay and may adversely affect packet delivery performance [65].

Duty cycling is particularly effective under non-intensive traffic because packet arrivals are distant and can often be served with short active periods. However, multi-hop forwarding creates non-uniform load even when traffic generation at the sources is homogeneous. Some nodes mainly generate and transmit their own application packets, while others also act as relays and must receive, buffer, and forward packets generated by their neighbors. As a result, a common duty-cycle configuration may be sufficient for lightly loaded source nodes, but insufficient for forwarding or aggregation nodes.

The limitations of contention-based duty cycling become more pronounced under dynamic or bursty traffic. If many packets arrive within a short interval, the active period may be too short to serve them, leading to queue buildup, buffer drops, busy-channel events, collisions, backoffs, and retransmissions. A configuration that is energy-efficient during low-rate operation can therefore provide insufficient service capacity during traffic bursts, potentially leading to packet loss. Adaptive mechanisms, such as activity-dependent active periods in T-MAC [60], overhearing reduction in X-MAC [62], and consecutive-transmission adaptation in ContikiMAC, partially alleviate this problem. Nevertheless, due to contention-based medium access, such adaptation does not fully eliminate contention, forwarding bottlenecks, or queue buildup under network-wide bursty traffic, making these protocols less suitable for intensive data traffic.

Overall, contention-based and contention-free MAC techniques each exhibit distinct strengths and limitations, and neither is universally superior in terms of both performance and power consumption. This observation motivates the exploration of hybrid MAC designs that combine both access strategies. In such designs, duty-cycled contention-based access is often more efficient under sporadic and low-rate traffic, where a small number of packets can usually be served during periodic wake-up periods without maintaining a continuous transmission schedule. Conversely, contention-free techniques can increase service capacity during bursty periods by allocating additional transmission slots.

2.4 Conclusion

This chapter provided a global overview of LLWNs and their traffic regimes and introduced the main concepts used throughout the remainder of the thesis. It first presented the network stack and the separation of network functionality into data, control, and management planes. It also explained how, in conventional LLWN operating systems, these protocols and planes are typically compiled into a single monolithic firmware image. This design makes network-stack reconfiguration costly, since even minor modifications may require

transmitting a complete firmware image over constrained LLWN links. The chapter also discussed different MAC-layer access techniques and showed that their performance depends strongly on the traffic regime. In particular, contention-based and contention-free techniques exhibit complementary strengths under sporadic, periodic, and bursty traffic, motivating the need for hybrid and adaptive MAC designs.

Building on these observations, the next chapter reviews the state of the art related to selective update mechanisms for LLWNs. It also discusses hybrid control plane architectures in wireless networks and LLWNs that can provide better performance compared to fully distributed or fully centralized control plane deployments. Finally, it reviews adaptive and hybrid MAC protocols and machine-learning-based solutions for MAC-layer reconfiguration and network performance forecasting.

State of the Art

Chapter contents

3.1	Software Evolution and Over-the-air Updates in LLWNs	26
3.1.1	Firmware Image Updates	27
3.1.2	Dynamic Code Update Frameworks	29
3.1.3	Conclusion	35
3.2	Control plane Deployment Models in LLWN	36
3.2.1	Distributed Control Plane Deployments in LLWNs	36
3.2.2	Centralized Control Plane Deployments in LLWNs	37
3.2.3	Hybrid Control Plane Deployments	40
3.2.4	Conclusion	43
3.3	MAC-Layer Adaptation in LLWNs	43
3.3.1	Adaptive MAC Protocols in LLWNs	44
3.3.2	Machine Learning for MAC-layer in LLWNs	46
3.3.3	Conclusion	50
3.4	Conclusion	50

3.1 Software Evolution and Over-the-air Updates in LLWNs

LLWNs are typically deployed with the expectation that they will operate for long periods with little human intervention. This long operational lifetime creates a corresponding need for long-term maintainability. A protocol stack that is adequate at deployment time may become insufficient after months or years of operation. In addition, the evolution of

applications and security requirements also motivates runtime updates [11]. As a result, software evolution cannot be treated as an exceptional maintenance operation; it becomes a regular requirement for maintaining the network throughout its operational lifetime.

In many applications, replacing or physically accessing nodes is expensive, difficult, or even impossible after deployment [14]. Over-the-Air (OTA) update mechanisms have been identified as a key enabler of IoT maintainability, since they allow deployed nodes to receive software corrections and improvements remotely through the network without physical access [11]. This is particularly important in LLWNs, where the cost of manual maintenance can exceed the cost of the devices themselves.

The central concern of this thesis is the maintenance of the network stack. LLWNs operate in dynamic and uncertain environments, where changing link conditions and application requirements may require reconfiguration of the network stack in order to continue operating close to optimal performance. For example, routing paths may need to be updated, or MAC parameters may need to be retuned. Existing studies on constrained IP stacks show that network-stack architecture strongly affects the ability to evolve protocol behavior after deployment [13]. Therefore, update mechanisms should not be limited to applications; they should also support the evolution of lower-layer communication functions.

3.1.1 Firmware Image Updates

The conventional OTA update model in many constrained IoT and LLWN systems is full-image firmware replacement. In this approach, the updated software is compiled into a complete firmware image, which is then transmitted to the target devices and installed as a replacement for the currently running image [66]. Its main advantage is simplicity: the device does not need to interpret the update semantics, but only receives, verifies, stores, and boots into a new binary image. However, this simplicity comes at a high cost in LLWNs [67].

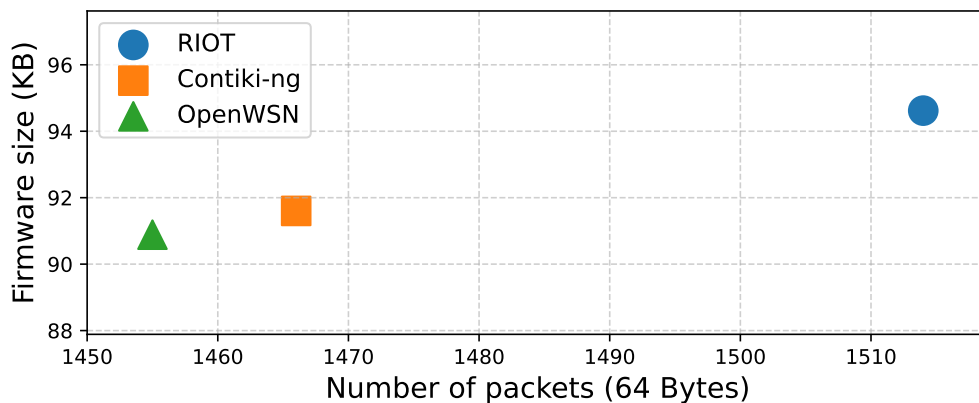


Figure 3.1: Firmware size comparison for different operating systems.

The first limitation is update size. Even a minor modification, such as changing a

protocol parameter, may require transmitting the entire firmware image. In constrained IoT systems, this image can reach tens or hundreds of kilobytes, which is large relative to the throughput of low-power wireless links [13]. Fig. 3.1 compares the firmware size of a network stack that includes CoAP, UDP, IPv6-RPL, 6LoWPAN, and IEEE 802.15.4 across three major LLWN operating systems: RIOT [68], Contiki-NG [69], and OpenWSN [70]. All systems exceed 90 kB, which, when segmented into 64-byte CoAP payloads typically used for LLWN update delivery [44], results in thousands of packets traversing the network. Due to the limited frame size, large firmware images must be fragmented into several packets. These packets must traverse the same low-power and often multi-hop network used by application and control traffic. This additional traffic can increase contention, collisions, queue occupancy, and retransmissions, especially when several nodes are updated simultaneously. Consequently, the update process itself may temporarily degrade the communication service it is intended to improve [71].

Full-image updates also lead to long update times. Such long update durations can cause temporary network partitioning because nodes may operate with different configurations before the update converges across the network. In addition, excessively long update times may cause the new configuration to become obsolete with respect to newly emerging conditions [67]. Energy cost is another major concern. Transmitting, receiving, verifying, and storing a full firmware image requires radio activity, processing, and memory access, all of which consume energy. Ruckebusch et al. show that the energy consumption of OTA updates depends strongly on the update technology, radio access, and amount of transmitted data [14]. In power-constrained LLWNs, the cost of frequent large updates can therefore significantly reduce network lifetime.

A further drawback is the mandatory reboot that is typically required after installation. Rebooting is disruptive in LLWNs because protocol state is often stored in volatile memory. After reboot, all protocol-state information, such as routing tables and queue states, is lost. The node must then restart the network stack and rejoin the network, which introduces reconvergence delay, additional control traffic, and interrupts the service until the state is rebuilt [11].

A refinement of the full-image update model is available: instead of transmitting the complete firmware image, this refinement computes and disseminates only the difference between the currently installed image and the new version. When the modification is small, this can significantly reduce the number of bytes transmitted over the air, thereby reducing update time, energy consumption, and congestion compared to complete image replacement. Sorensen et al. [72] show that incremental firmware updates are a practical way to reduce the size of OTA updates and installation time for embedded IoT fleets. They also highlight an important trade-off: while such approaches save control traffic overhead, they can increase temporary storage requirements and still depend on the device having the expected base firmware version. However, incremental updates remain largely binary-level mechanisms where the update is still expressed as a modification of the firmware image.

In conclusion, full-image firmware replacement is simple and robust from a software-packaging perspective, but it is poorly aligned with LLWN constraints. A mandatory reboot can erase volatile network state, leading to full stack reconvergence and temporary service interruption. Incremental updates reduce update size by transmitting only the binary differences between old and new images, but they remain largely image-oriented. These limitations motivate modular designs, which decompose the system image into independent components that can be updated separately. In these designs, only the necessary part of the system is modified while unrelated components and network runtime state can be preserved by moving update granularity from the binary level to the function level.

3.1.2 Dynamic Code Update Frameworks

The limitations of image-based updates have motivated more modular software designs. These designs can be broadly grouped into three families: scripting-based approaches, module-based approaches, and virtual-machine-based approaches.

3.1.2.1 Scripting-Based Approaches

Scripting-based approaches constitute an early form of runtime software evolution for constrained IoT devices. Their main idea is to separate the application logic from the native firmware image by embedding a script interpreter on the device. Updates are then performed by transmitting a new script, which can be executed at runtime without recompiling or replacing the complete firmware. Compared with full-image replacement, this model provides a more flexible and application-oriented form of reprogramming: the sensing, actuation, and control logic can evolve after deployment, while the underlying operating system and communication stack remain unchanged.

Several works have explored this direction by adapting high-level languages, particularly JavaScript, to constrained IoT platforms. Ultra-lightweight engines such as JerryScript [73] aim to make JavaScript execution feasible on small embedded devices by reducing the memory footprint of the interpreter. Other approaches integrate scripting into broader development and deployment frameworks. For example, Baccelli et al. [74] combine graphical programming, automatic JavaScript generation, and OTA script deployment for RIOT Operating System (RIOT)-based devices, where scripts can be updated through a CoAP-accessible runtime container. WebletScript [75] extends the same principle to distributed IoT applications by partitioning a JavaScript application across end devices, gateways, and cloud components.

Taken together, these works show that scripting is effective when the objective is to simplify application-level development and deployment by reducing the update size. However, they also reveal an important boundary of the scripting model for LLWNs. First, interpretation introduces a persistent runtime cost in memory and processing, which is difficult to justify on highly constrained nodes. Second, interpreters typically run in user

space and cannot directly implement or modify kernel-level networking functions. Consequently, scripting improves software flexibility but remains too application-oriented and too costly to serve as the main mechanism for fine-grained network-stack programmability in LLWNs.

3.1.2.2 Module-Based Approaches

Module-based approaches aim to reduce the cost of software evolution by decomposing firmware into separate modules. As a result, only affected modules are transmitted and updated. This can reduce update size, transmission time, and memory requirements. Module-based systems generally execute native code, and therefore avoid the execution overhead of script interpreters required in scripting approaches.

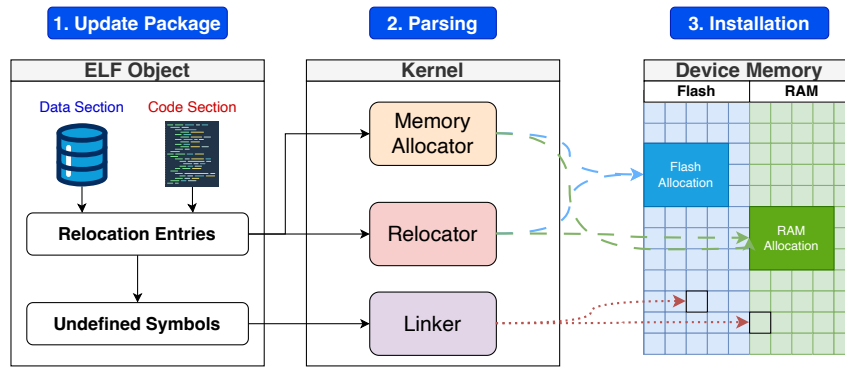


Figure 3.2: Module-based update process

In these approaches, the firmware is decomposed into separate modules or components with different roles. A module can represent application logic, a network protocol, a driver, or any service in the system. A subset of modules remains static and non-updatable, forming the kernel or core runtime support that manages and executes the remaining updatable modules that can be installed, replaced, or removed after deployment.

Each updatable module is compiled as an Executable and Linkable Format (ELF) object. Each ELF object contains the compiled code and associated data of the corresponding module, as illustrated in Figure 3.2. During module installation, the kernel typically allocates Flash memory for the code section and RAM for the data section. The ELF object also includes relocation entries that indicate the locations in the code or data that must be adjusted once the module is placed in memory. These entries allow the operating system to relocate address references according to the actual physical addresses assigned to the module. In addition, an ELF object may depend on functions or data provided by the kernel or by other modules; such dependencies appear as undefined symbols. The operating system therefore relies on a runtime linker to resolve these symbols and bind them to the corresponding physical addresses. Consequently, the flexibility of post-deployment updates depends not only on the ability to load new modules, but also on how flexibly the system

can bind updated modules to the functionality already present in the kernel or in other modules. This dependency-resolution mechanism defines the binding model used by the module-based update system.

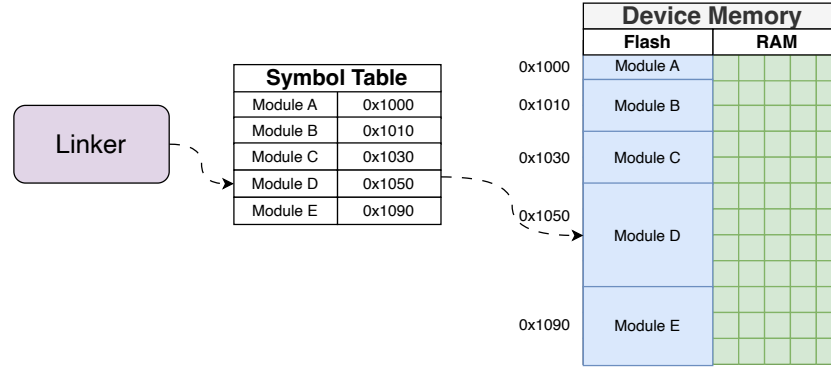


Figure 3.3: Strict binding module-based approach

A first class of module-based updates relies on *strict binding*. In this model, the base firmware exposes a predefined symbol table generated at compile time, as illustrated in Figure 3.3. This symbol table provides a proxy for system modules hardcoded in the base kernel and cannot be modified after deployment. Contiki [76] is a representative example of this approach. This model reduces dissemination cost and avoids rebooting, but the update scope remains narrow because the kernel, drivers, and most network-stack services remain part of the static core.

Dynamic TinyOS [77] partially relaxes this limitation by preserving selected module boundaries across compilation. Instead of compiling the whole application and operating system into a single monolithic binary, it allows developers to define boundaries between modules or groups of modules that can be updated separately. However, it still follows a strict-binding philosophy because the possible update boundaries must be defined before deployment. In practice, strict binding limits updates mainly to top-level or explicitly isolated modules, rather than enabling arbitrary modification of low-level kernel, driver, or network-stack functions.

A second class of module-based systems adopts a more dynamic module-oriented design. Instead of binding updated modules directly to fixed physical addresses, these systems introduce an indirection layer, such as function-control blocks, or dynamic linking tables. As illustrated in Figure 3.4, when a module is installed or replaced, the indirection structure is updated to associate the fixed module identifier with the new physical address of the corresponding implementation. Existing modules can therefore continue to invoke the same module identifier, while the runtime redirects the call to the updated implementation.

SOS [78] is one of the earliest examples of this model. It organizes software as dynamically loaded position-independent modules. Functions exported by modules are represented through Function Control Blocks (FCBs), which record the function identifier, prototype

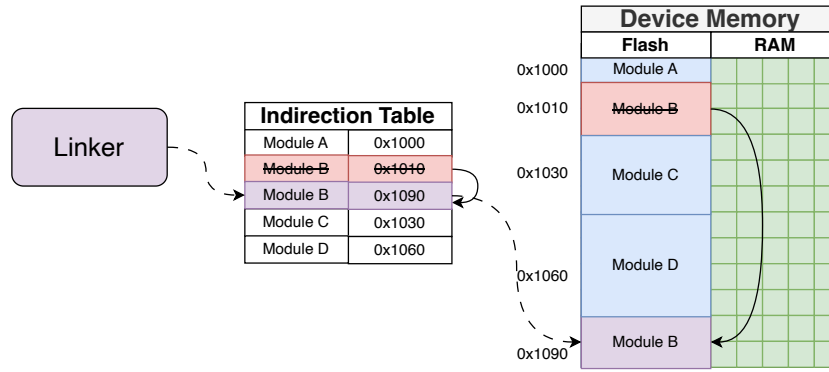


Figure 3.4: Dynamic binding module-based approach

information, and current implementation address. When a module is updated, SOS can update the corresponding FCB and automatically redirect other modules that use it to the new implementation. In addition, Enix [79] and REMOWARE [80] extend this dynamic binding approach by introducing runtime redirection of function calls. Both systems support the loading, replacement, and reconfiguration of software modules while maintaining stable references to their functionality. Lorian [81] further strengthens this model by assigning each module its own *jump table*, which is updated whenever the module is updated. After each update, the kernel instantiates the jump table of the updated module to refresh the physical addresses of the corresponding functions or data.

Compared with strict binding, the use of indirection makes updates more flexible because dependent modules do not need to be relinked directly against the physical addresses of the updated module. However, although these systems improve the programmability and reconfigurability of LLWN applications, they are not specifically designed as network-stack update architectures. Their ability to update MAC, routing, or other protocol functions depends on whether these functions have been explicitly modeled as dynamic modules and whether their interfaces remain compatible across updates.

GITAR [82] explicitly extends dynamic updates to both application and network modules. Unlike earlier systems that focus mainly on application-level reconfiguration, GITAR defines network-level programmability as the ability to add or modify code that implements communication functionality from the MAC layer up to the transport layer. To support this, it introduces a clear boundary between a static system level and a dynamic module level, placing network protocols and applications above this boundary as dynamically updateable modules. This makes GITAR one of the closest prior works to network-stack programmability in IoT systems. GITAR shows that communication-stack functionality can also be treated as updateable software modules. Nevertheless, its granularity remains coarse. Network protocols are still treated as relatively large modules, while their internal protocol semantics remain largely hidden. As a result, GITAR does not fully expose the internal protocol state and structures to the update orchestrator. This

limits its ability to support precise and state-preserving updates of the network stack. There is a need to understand what is updated and how it should be updated to avoid post-update inconsistencies and lengthy reconvergence that can result from losing the stored state or the need to build a new state for specific updates.

Overall, module-based update approaches provide a flexible mechanism for evolving IoT software at runtime by replacing or reconfiguring individual modules instead of redeploying complete system images. These systems reduce update size, minimize service disruption by avoiding reboots, and enable incremental adaptation of both application and networking functionality. However, updates are generally performed at the granularity of modules, which often remain relatively large. In addition, this granularity is too coarse to capture the semantics of individual protocol functions needed to identify dependencies, update ordering, and state consistency during updates. For example, a routing protocol may still be updated as a single large module rather than being decomposed into smaller functions such as control-message handling, next-hop selection, route installation, or source-route construction. As a result, while module-based approaches significantly advance runtime adaptability, they provide limited support for fine-grained, semantic, and state-aware evolution of network-stack functionality.

3.1.2.3 Virtual-Machine Approaches

Virtual-machine-based approaches execute updated functionality inside an isolated runtime environment that interacts with modules through predefined Application Programming Interfaces (APIs) rather than linking it directly to the native firmware. In this model, the device firmware includes a Virtual Machine (VM) interpreter or engine, and new VM functionality is deployed as bytecode or portable code that is interpreted or executed by this runtime. Compared with module-based approaches, VMs provide stronger isolation between the updated logic and the operating system, which can improve safety and portability. They also simplify deployment because, in principle, the same bytecode can run on different hardware platforms as long as the required VM engine is available. However, this flexibility comes at the cost of an additional memory footprint, execution overhead, and runtime complexity, which must be carefully considered in LLWNs.

Early VM-based approaches for LLWNs focused mainly on application-level reprogramming. Maté [83] is one of the earliest examples; instead of replacing native code, it disseminates compact bytecode capsules that execute on top of the VM engine. However, Maté provides a very restricted programming model and is mainly designed for simple application tasks. Darjeeling [84] extends the idea by providing a richer Java-compatible virtual machine for constrained devices. It supports a higher-level programming model than tiny capsule-based systems and allows developers to write code in a more portable language. More recently, WebAssembly [85] has been proposed as a portable and safer execution format for constrained devices. WebAssembly provides a compact binary format, a sandboxed execution model, and language portability.

In general, the use of VMs significantly reduces update size compared to full firmware. However, this type of virtual machine still requires non-negligible memory and processing resources. To reduce the overhead of conventional VMs, lightweight VM approaches have emerged. These approaches are often inspired by the Berkeley Packet Filter (BPF) model, in which small programs are verified and executed within a restricted virtual machine. The advantage of this model is that bytecode programs are compact and the runtime can remain small. rBPF [86] and μ BPF [87] adapt this idea to IoT constrained devices, showing that minimal BPF-like virtual machines can run on constrained devices and support fine-grained, on-demand code updates. However, BPF-like systems are still usually limited by the *hooks* exposed by the underlying runtime or operating system. Hooks are specific checkpoints installed in different system modules to observe specific events and trigger the corresponding VM when those events occur.

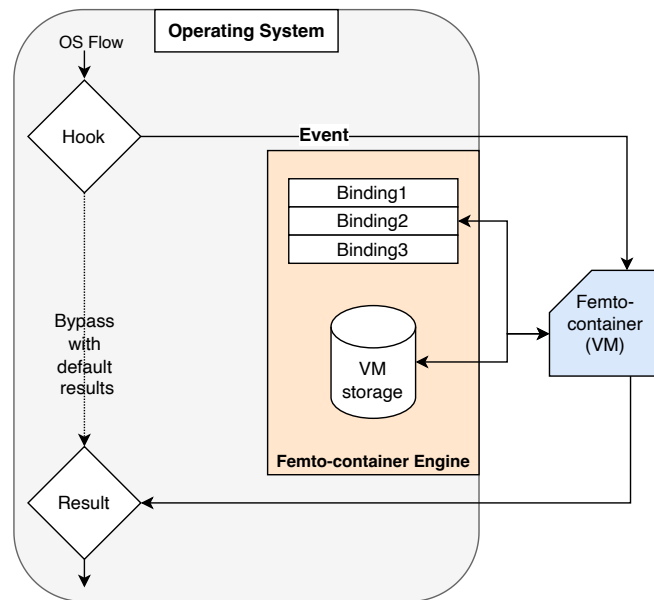


Figure 3.5: Femto-Container operation and integration in the operating system [88]

Femto-Containers (FCs) [88] represent one of the most relevant lightweight VM approaches for LLWNS. They provide isolated execution environments for small software functions on constrained devices. Their design follows the lightweight BPF-inspired model, but adapts it to real-time-operating-system (RTOS)-based constrained devices. As illustrated in Figure 3.5, the VMs (or FCs) are event-driven: a VM executes only when triggered by a specific event or hook, rather than running continuously as a separate process. This makes the approach particularly suitable for low-power devices. FCs also provide fault isolation and interact with the operating system only through predefined bindings, allowing small functions to be deployed and updated without compromising the rest of the firmware.

The main strength of FCs is that they provide a practical substrate for fine-grained runtime extensibility. A function can be implemented as an independent VM and attached

Table 3.1: Summary comparison of dynamic code update approaches in LLWNs.

Approach	Examples	Granularity	Strengths	Network-programmability limitations
Scripting-based	JerryScript [73] RIOTjs [74] WebletScript [75]	Script level	Flexible and portable. Easy application-level updates.	Interpreter overhead. Application-oriented. Limited access to low-level protocols.
Module-based	Contiki [76] TinyOS [77] SOS [78] Enix [79] RemoWare [80] Lorien [81] GITAR [82]	Module level	Native execution. Smaller updates than full images. Supports dynamic loading.	Binding and dependency complexity. Coarse modules. Limited semantic visibility.
VM-based	Maté [83] Darjeeling [84] WebAssembly [85] rBPF [86] μ BPF [87] FCs [88]	Function level	Isolation and portability. Safe extensibility. Fine granularity with lightweight VMs.	Conventional VMs are costly. Lightweight VMs require hooks. Need bindings and semantic management.

to a hook in the operating system or application. This enables selective updates: instead of transmitting a complete firmware image or replacing a large module, only the small VM implementing the modified function needs to be updated. However, FCs alone do not define a complete programmable network stack. They provide the execution mechanism, but the system designer must still determine where hooks should be placed, what each container should represent, and how containers should exchange data with the operating system. Like other approaches, FCs were developed mainly to operate at the application layer.

In general, virtual-machine-based approaches provide an important foundation for safe and portable runtime updates in constrained IoT systems. Their main limitation is that conventional VMs are too costly for many LLWN devices, while lightweight VMs provide only the execution substrate rather than a complete semantic-management framework. The remaining challenge is therefore architectural: lightweight virtualized functions must be extended to lower-layer network functions and integrated with semantics-aware management rather than being limited to application-layer logic.

3.1.3 Conclusion

LLWNs must evolve after deployment, especially at the network-stack level, to maintain performance, address vulnerabilities, and extend network lifetime. Although image-based updates are simple to implement, their large size increases transmission time, network congestion, energy consumption, and failure probability over lossy multi-hop links. Moreover, the reboot typically required after installation causes state loss and service interruption.

Dynamic code update frameworks address these limitations through three main families, summarized in Table 3.1. Scripting-based approaches simplify application-level updates, but

interpreter overhead and limited access to protocol internals restrict their suitability for fine-grained network-stack reconfiguration. Module-based approaches improve efficiency through native execution and smaller update units than full firmware images; yet, binding, relocation, dependency management, and coarse protocol modules limit their flexibility. VM-based approaches provide stronger isolation and portability, while lightweight runtimes such as BPF-inspired VMs and Femto Containers reduce traditional VM overhead. However, they remain only execution substrates unless combined with well-placed hooks, operating-system bindings, and semantic orchestration able to manage protocol functions, dependencies, and state. Lightweight virtualization therefore represents the most promising direction for safe, selective, and fine-grained LLWN network-stack updates, provided that it is supported by an appropriate semantic management framework.

3.2 Control plane Deployment Models in LLWN

The control plane computes and maintains the state required for network operation, including routing information, transmission schedules, and resource allocation policies. In LLWNs, the placement of these control plane functions is particularly important because devices operate under limited energy, memory, and processing capacity, as well as unreliable multi-hop wireless connectivity. Control decisions may therefore be executed locally on the nodes, delegated to a central controller, or shared between both. This section reviews the main control plane deployment models in LLWNs: distributed deployments, centralized deployments, and hybrid deployments.

3.2.1 Distributed Control Plane Deployments in LLWNs

Historically, LLWNs have relied mainly on distributed control plane deployments because constrained devices operate over low-rate, lossy, and often multi-hop wireless links. Kleinrock's work [89] showed that distributed packet switching and decentralized resource sharing could utilize network resources more efficiently than centralized approaches, especially under dynamic traffic conditions. In this model, each node executes part of the control logic locally and exchanges control messages with its neighbors to build and maintain the network.

At the routing layer, RPL is the representative distributed control plane protocol for IPv6-based LLWNs. The control plane is distributed because topology formation and maintenance rely on local message exchanges and local decisions. However, this distributed nature may also lead to slow convergence and locally suboptimal parent selection [41].

At the MAC layer, IEEE 802.15.4 also reflects the early distributed design philosophy of LLWNs. In its contention-based operation, nodes access the shared medium using CSMA/CA, relying on local carrier sensing to decide whether they can access the medium. This makes IEEE 802.15.4 simple and lightweight, but it also makes performance sensitive

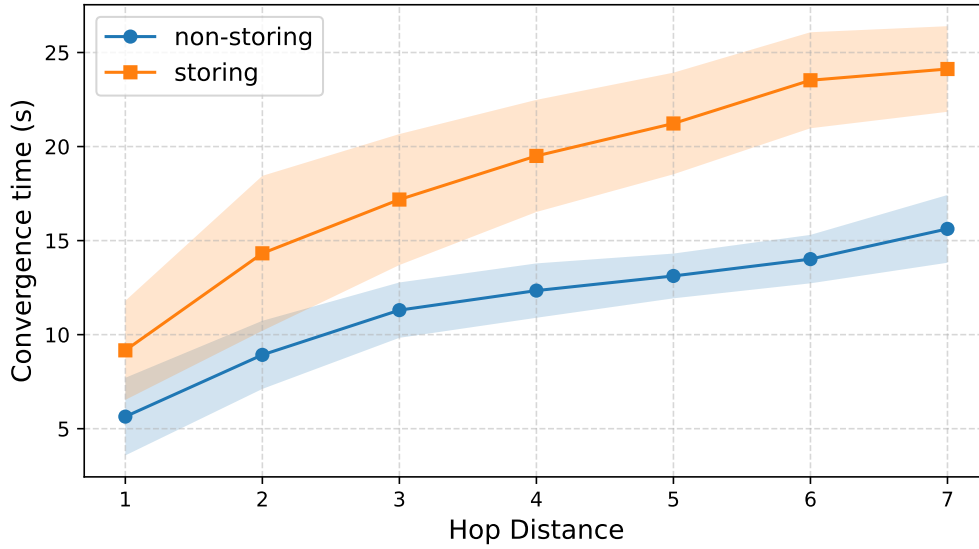


Figure 3.6: Convergence time in RPL over hop distance in a 7-node line topology.

to contention and medium-access conflicts.

Overall, distributed control plane deployments offer several advantages in LLWNs. They provide local autonomy and allow nodes to continue operating under intermittent connectivity. However, distributed control has limited global visibility. Each node makes decisions based on local information, which can lead to suboptimal routing, uneven forwarding load, inefficient scheduling, and slow adaptation [90]. Moreover, distributing the control plane among nodes makes control-logic modification difficult because, in conventional LLWN stacks, routing and MAC functions are usually compiled into the firmware. Thus, even small changes to the control plane may require a firmware update, reboot, and protocol convergence. Convergence time results from the need to exchange control information locally again, leading to temporary service interruptions. Figure 3.6 shows the convergence time required for nodes to register with the root in a seven-node linear RPL topology, based on an experiment conducted on the SLICES-FR¹ testbed. Both storing and non-storing modes exhibit increasing convergence times as hop distance from the root grows. More generally, convergence time is also expected to increase with the number of nodes due to increased overhead.

3.2.2 Centralized Control Plane Deployments in LLWNs

The limitations of purely distributed control have motivated the introduction of Software-Defined Networking (SDN) principles into IoT and LLWN environments. SDN separates the control plane from the data plane and places decision-making logic in a logically centralized controller. The controller maintains a broader network view and installs forwarding, routing, scheduling, or configuration rules in constrained devices. Surveys

¹<https://slices-fr.eu/>

on software-defined LLWNs emphasize that this separation can improve programmability, network management, reconfiguration, and support for heterogeneous applications [91]. However, they also show that SDN cannot be transferred directly from wired networks to LLWNs because the control/management channel is itself wireless, lossy, multi-hop, and energy-constrained.

A first family of centralized LLWN approaches focuses on routing and forwarding programmability. SDN-WISE [92] introduces a stateful SDN solution for wireless sensor networks, where sensor nodes are programmable as finite-state machines and can execute local stateful actions. This statefulness is important because a naive traditional SDN model would generate excessive control/management traffic in constrained LLWNs. SDN-WISE therefore centralizes network control while attempting to reduce the amount of control information exchanged between nodes and the controller.

Baddeley et al. [17] propose μ SDN to adapt SDN to LLWNs. The work identifies SDN control overhead as a major barrier in LLWNs, where control traffic may suffer from jitter, loss, and contention while also interfering with application traffic. To address this issue, μ SDN introduces compact control messages that avoid fragmentation, control-message throttling, and memory-efficient flow-table structures.

Samadi et al. [93] propose IERMIoT, an SDN-based routing approach in which the controller is responsible for topology discovery, cluster formation, cluster-head selection, and route computation. The approach uses a genetic algorithm to determine the number and distribution of clusters, while considering criteria such as residual energy, node centrality, and distance when selecting cluster heads. It also introduces a mechanism that allows mobile nodes to remain attached to their current cluster head as long as connectivity is maintained, thereby reducing unnecessary re-clustering and control-message overhead.

A second family of centralized LLWN approaches focuses on MAC scheduling and radio-resource management. This is particularly important in industrial LLWNs, where IEEE 802.15.4 TSCH and 6TiSCH provide scheduled time-frequency resources. In such networks, the control problem is not limited to route selection; it also includes transmission-cell allocation and critical-flow protection. Baddeley et al. [94] propose allocating specific time slots for control/management traffic, thereby reducing interference between this traffic and application data. This addresses one of the main weaknesses of centralized LLWN control: the control plane must be reliable before it can reliably manage the rest of the network.

Veisi et al. propose SDN-TSCH, a centralized scheduling framework for industrial LLWNs [95]. In SDN-TSCH, the controller is responsible for selecting time sources, maintaining a control plane tree with dedicated resources, and installing data flows while guaranteeing flow isolation, as illustrated in Figure 3.7. Different cells are allocated for different traffic types. For example, to send telemetry from a node to the controller, the node and the intermediate nodes use their dedicated cells toward the controller to relay the data until it reaches the controller. The key contribution is therefore the adaptation of

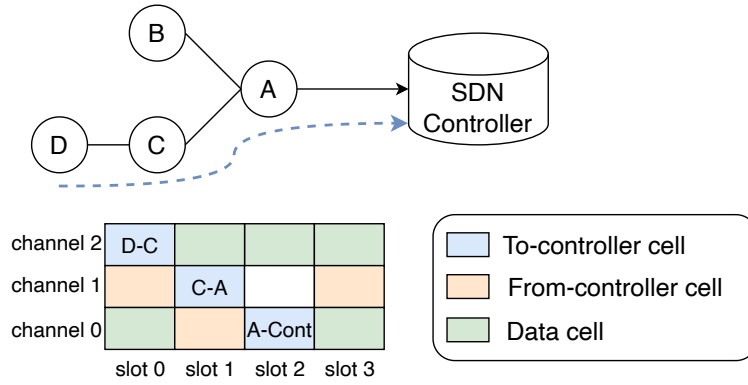


Figure 3.7: SDN-TSCH operation [95]

centralized control to TSCH networks, where MAC scheduling becomes part of the control plane decision space.

Scalability is another major challenge for centralized SDN in LLWNs. Orozco-Santos et al. [96] show that, as the network grows, SDN control and management traffic accumulates near the sink, reducing the throughput available for data traffic. To mitigate this bottleneck, they propose using multiple virtual sinks operating on different radio channels, allowing simultaneous packet reception over several TSCH channels and increasing capacity in the most congested network region. This work shows that centralized MAC and routing management can improve QoS only when the architecture also controls the overhead and bottlenecks introduced by the controller or management plane.

Overall, centralized control plane deployments bring important benefits to LLWNs, including global visibility, programmable routing, traffic engineering, flow isolation, scheduling optimization, and easier reconfiguration. However, the literature also shows recurring limitations. First, the control/management channel is fragile because it relies on the same low-power, lossy, and multi-hop wireless substrate as data traffic. Second, telemetry collection and rule installation increase overhead and may congest forwarding nodes close to the sink. Third, centralized decisions may become stale when topology or link quality changes faster than the controller can react [18].

Finally, distributed control plane deployments provide autonomy and robustness under constrained wireless conditions, while centralized SDN-based deployments improve programmability, global optimization, and runtime management. Nevertheless, neither model is sufficient alone: distributed control lacks global optimization and is difficult to update, although it scales naturally, while centralized control depends on a reliable and low-overhead management path. This tension motivates hybrid control plane deployments, discussed in the next subsection, where local autonomy and global orchestration are combined.

3.2.3 Hybrid Control Plane Deployments

Hybrid control plane deployments were first explored in wired networks. Their objective was not to immediately replace legacy routing, but to introduce SDN programmability while preserving the robustness, stability, and operational maturity of distributed routing protocols. In this context, hybrid SDN is motivated by practical deployment constraints: network operators already rely on large legacy infrastructures that are stable and well understood, while a sudden migration to full SDN may be costly, risky, or operationally unrealistic [97].

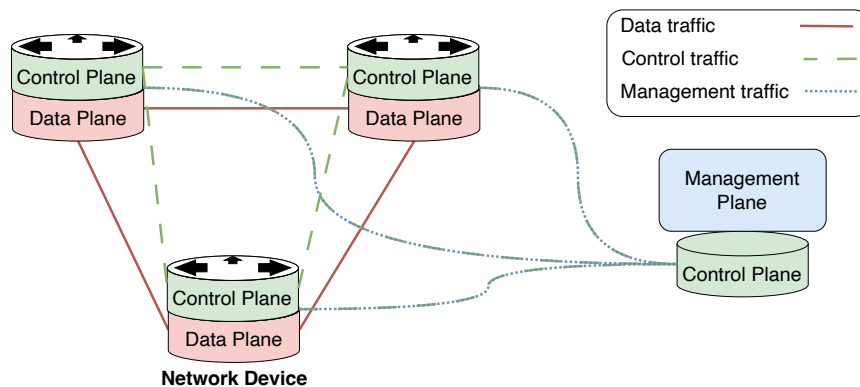


Figure 3.8: Hybrid control plane deployment

As illustrated in Figure 3.8, hybrid deployments maintain the local control plane alongside the data plane. Nodes continue to exchange control information with their neighbors to build local control state and maintain local forwarding decisions. At the same time, they interact with a global control and management plane that provides broader network visibility and optimization. Collaboration between the local and global control planes can take different forms depending on how responsibilities are divided between distributed protocols and centralized logic, and on the level of coordination achieved between them. Vissicchio et al. [98] show that running multiple uncoordinated control planes can create forwarding anomalies, even when each control plane is correct in isolation.

Tilmans and Vissicchio propose an explicit separation between centralized and local decisions [99] without an explicit collaboration. During normal operation, the SDN controller installs the primary forwarding rules, and these rules determine the active traffic paths. In parallel, local agents run the Interior Gateway Protocol (IGP) routing protocol to compute backup routes, but this IGP state is not used to compete with the controller during normal operation. Instead, it is kept as a local recovery mechanism. OpenRouteFlow [100] upgrades legacy routers with agents that export routing and flow information to a central controller. It keeps distributed routing protocols active and uses their state to build the central controller's view. This prevents conflicting decisions because the controller relies on the router's existing routing state rather than imposing an independent one. The agent also translates controller commands into supported router operations, allowing local routing to

maintain connectivity while the controller provides higher-level control.

In general, wired-network solutions for hybrid control planes assume stable links, powerful devices, and reliable control channels. In LLWNs, the same ideas must be adapted to low-power devices, lossy multi-hop communication, limited memory, and intermittent connectivity. In particular, hybrid deployments require lightweight control, low-overhead telemetry, state preservation, and semantic runtime updates. This gap motivates the later discussion of hybrid control plane deployments in wireless networks and, more specifically, in LLWNs.

3.2.3.1 Hybrid Control in Wireless Networks

In wired networks, hybridization is often motivated by incremental migration from legacy routing to SDN. In wireless networks, hybridization is also motivated by operational necessity: links are unstable, topology may change, interference affects connectivity, and the controller may not always maintain a fresh or reliable view of the data plane.

Abolhasan et al. [101] propose an architecture in which the controller does not directly install complete end-to-end routes that would override local decisions. Instead, it collects link-state and channel information, computes abstract routing weights, and broadcasts these pre-processed values to the nodes. The nodes then use the controller-provided weights to compute routes locally. CATMAN [102] follows a similar coordination principle but implements it through fog nodes. Normal nodes maintain local neighborhood information and can route in a distributed manner when no fog node is available. When fog nodes are present, they extend the controller's network visibility by collecting local information, which helps the controller inject forwarding decisions. Here, local routing is used as the default mechanism for neighborhood-level operation and fallback connectivity, while global control is applied through updates when broader network knowledge is available.

Other works adopt a different hybrid model in which the SDN controller interacts directly with existing wireless or mobile control mechanisms, rather than replacing or adapting the legacy data plane. The objective is to introduce programmability while preserving the operational role of legacy components. Silva et al. [103] follow this approach in a hybrid SDN mobile-core architecture for 4G and 5G systems. The controller introduces dynamic traffic steering, offloading, and slice instantiation, while existing functions continue to manage standardized attachment, authentication, and mobility procedures.

Duo et al. [104] propose an SDN-based handover scheme for cellular/IEEE 802.11p hybrid vehicular networks, where the controller maintains a global topology view and prepares handover decisions, while local access technologies support connectivity and data continuity during mobility. Chen et al. [105] apply SDN to hybrid LiFi/WiFi networks, where the controller selects access points and installs flow rules, while the underlying LiFi and WiFi links remain responsible for access-specific communication. In both cases, the controller improves coordination across heterogeneous technologies, but local mechanisms

still handle the constraints of each wireless medium.

3.2.3.2 Hybrid control in LLWNs

In LLWNs, hybrid control plane deployment is more constrained than in general wireless networks. The centralized component may introduce high control overhead, while the distributed component makes runtime reconfiguration difficult because its coarse design requires updating a large number of nodes.

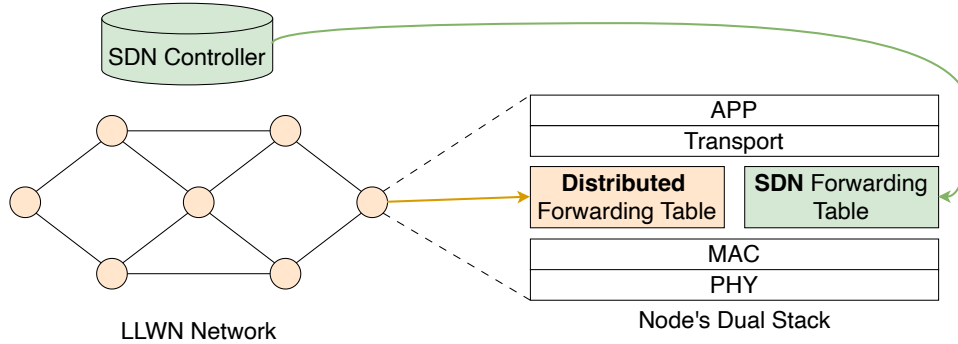


Figure 3.9: SOHS dual-stack operation [106]

The most directly relevant LLWN-oriented work is SOHS, proposed by Bonanni et al. [106]. SOHS can be classified as a dual-stack hybrid control plane architecture because it keeps the traditional distributed control stack active in the background while adding an SDN stack above it for centralized optimization. As illustrated in Figure 3.9, each node maintains two forwarding structures: a basic forwarding table built locally through distributed neighbor discovery and topology-control messages, and an SDN forwarding table populated by the controller. When a packet arrives, the node first checks the SDN table; if a matching SDN rule exists, the packet follows the controller-optimized path. If no SDN rule is available, or if the controller becomes unreachable, the node falls back to the distributed forwarding table.

Hybridization in SOHS is therefore achieved through priority-based coexistence rather than replacement. The distributed stack is responsible for maintaining baseline connectivity, neighbor awareness, and resilience, while the SDN controller is used to optimize selected end-to-end paths according to application requirements. This design extends the autonomy of distributed control with the global knowledge provided by centralized control when available.

However, this dual-stack design can also increase memory footprint, implementation complexity, and coordination cost on constrained devices. More importantly, hybridization operates mainly at the level of coexistence and fallback between control modes. It allows the network to switch between centralized and distributed forwarding behavior, but does not provide deep collaboration between the two control planes. The controller does not semantically manage the internal state of the distributed routing protocol itself. For

example, if the distributed routing component is updated or reconfigured, the routing state may be lost, while the controller does not provide mechanisms to preserve, migrate, or restore the local state at the nodes. Therefore, SOHS improves the resilience of SDN-based LLWNs but does not fully address the coordinated and state-preserving runtime evolution of the network stack.

3.2.4 Conclusion

Control plane deployment in LLWNs has evolved from fully distributed protocols toward centralized and, more recently, hybrid models. Distributed approaches such as RPL and IEEE 802.15.4 CSMA/CA provide autonomy and robustness under constrained wireless conditions, but they suffer from limited global visibility, slow adaptation, and difficult runtime updates. Centralized SDN-based approaches improve programmability, optimization, and management, but they depend on a reliable and low-overhead management plane, which is difficult to guarantee in lossy multi-hop LLWNs.

Hybrid deployments therefore emerge as a necessary compromise: they preserve local control for resilience while exploiting centralized orchestration for global decision-making and reconfiguration. Such deployments can be realized in different ways, including the coexistence of centralized and distributed control planes or the merging of both planes. However, existing hybrid solutions for LLWNs remain limited compared with those proposed for other wireless or wired networks and often rely on dual-stack designs. These designs provide limited collaboration between the two planes, thereby restricting the transparent runtime evolution of the network stack that could be achieved through closer and more simultaneous coordination between local and global control.

3.3 MAC-Layer Adaptation in LLWNs

The previous sections have shown that LLWNs operate under severe resource constraints and highly variable wireless conditions. In this context, the MAC layer is a natural point for adaptation because it determines when nodes wake up, how they access the channel, and how retransmissions are handled. These decisions directly affect reliability, latency, throughput, and energy consumption. Improving LLWN performance therefore requires not only efficient MAC protocols but also mechanisms that allow their behavior to evolve according to traffic load, network state, and application requirements.

This section reviews MAC-layer adaptation from two complementary perspectives. First, it discusses adaptive MAC protocols. Second, it reviews machine-learning-based approaches that predict communication performance and/or adapt MAC behavior.

3.3.1 Adaptive MAC Protocols in LLWNs

Adaptive MAC protocol designs are motivated by the need to serve variable traffic regimes, and they can be broadly classified into three families according to their dominant access model: contention-based, contention-free, and hybrid protocols.

A first family keeps contention-based access as the main communication mechanism, but adapts the timing or contention parameters to reduce collisions and idle listening. ContikiMAC [64] represents an important duty-cycled baseline in this direction. Phase-lock optimization, which aligns senders' and receivers' wake-up, enables a sender to learn the approximate wake-up schedule of a receiver based on previous successful communications. During a burst, phase-lock information can help reduce transmission overhead for nodes that communicate frequently. However, the adaptation remains implicit, as the protocol does not increase its duty cycle, allocate additional transmission opportunities, or modify contention behavior when bursts occur. As a result, when multiple nodes simultaneously generate event-driven traffic, the contention, queue occupancy, and retransmissions may increase significantly.

Kim et al. propose a dynamic duty-cycle MAC protocol [107]. In this protocol, the receiver monitors traffic congestion and dynamically adapts its duty-cycle ratio and acknowledges other nodes. Under high traffic conditions, the protocol shortens sleep periods and spreads transmission attempts among competing nodes to reduce collisions. When traffic decreases, it extends sleep periods to save energy, avoid unnecessary waiting, and improve channel utilization. Sender nodes use the information in acknowledgment messages from receivers to adapt their contention configuration. This improves delay and energy consumption compared with a fixed duty cycle, but the adaptation remains local and reactive to observed congestion.

A second family relies mainly on scheduled or contention-free access protocols that adapt the deterministic resource allocation. For example, recent adaptive TSCH-MAC works allocate time slots according to contextual information such as node position and data intensity [108]. Nodes that have intensive data receive longer or more frequent active durations compared to other nodes. This provides deterministic access and improves reliability and energy efficiency. However, this adaptability depends on synchronization, periodic metadata reporting, centralized schedule computation, and schedule dissemination, which increase overhead.

The previous two families improve adaptability by modifying only one dominant access model. Although adaptation can enhance the performance of both access models, relying on a single model always introduces limitations. Contention-based protocols suffer from degraded performance when traffic becomes dense or bursty, whereas contention-free protocols introduce scheduling overhead and may waste resources when traffic is sparse. A third family therefore combines both access models within the same protocol. These approaches seek to exploit the flexibility of contention-based access under light traffic and

the reliability of scheduling under dense or bursty traffic.

Al-Janabi and Al-Raweshidy propose a hybrid TDMA-CSMA/CA MAC protocol with dynamic scheduling [109]. TDMA schedules slots for groups of devices, and devices within each group then contend using CSMA/CA. In this design, adaptation operates at two levels. First, the base station partitions nodes into groups and assigns TDMA slots to each group, ensuring that groups access the channel in an organized manner and reducing inter-group collisions. Second, the protocol continuously monitors network conditions such as traffic load, collision rate, and channel reliability and adjusts the duration of sleep and wake-up periods accordingly. This adaptation helps balance energy efficiency and channel utilization, although it remains reactive since adjustments are triggered only after congestion, collisions, or channel degradation are detected.

Application-specific hybrid protocols follow similar principles. MC-HYMAC [110] integrates contention-based access (CSMA/CA) with scheduled access (TDMA) and exploits multiple channels to separate control signaling from data transmission within a single framework for Wireless Body Area Networks (WBANs). This separation helps reduce interference and collisions, as control messages do not compete with data packets on the same channel. In addition, this protocol dynamically adjusts transmission power, selects channels based on current conditions, and adapts contention intervals and time-slot assignments according to network load and traffic patterns. However, it is tailored to WBAN scenarios, which limits its direct applicability to more general LLWN deployments.

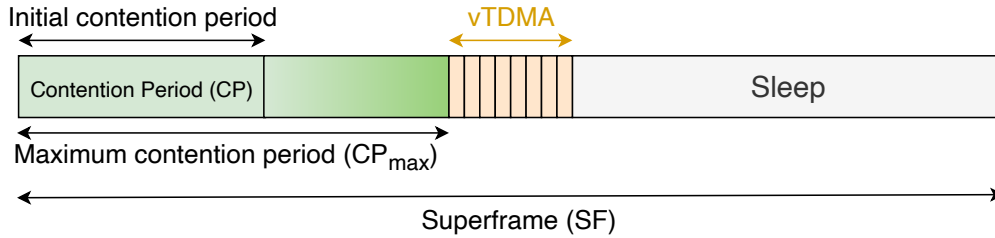


Figure 3.10: GoMacH frame structure

GoMacH [111], the MAC substrate used in this thesis, also belongs to the hybrid MAC family. As shown in Fig. 3.10, GoMacH organizes time into repeated superframes, denoted as SF . Each superframe contains an active communication part followed by a sleeping period used to reduce radio energy consumption. The active part combines contention-based access, dynamically allocated variable TDMA access, and multi-channel operation to further improve parallelism and reduce interference. At the beginning of the active period, nodes wake up during a contention period CP , where senders use CSMA/CA to transmit packets to the receiver. GoMacH becomes adaptive when traffic increases. Each sender embeds its current queue length in the MAC header of the packet transmitted during the contention period. If no sender reports pending packets, the receiver can end the active period and nodes return to sleep. If pending traffic is detected, the receiver extends the

active period, up to CP_{max} , and prepares a variable TDMA phase. The receiver then broadcasts a scheduling beacon that assigns transmission slots to the senders according to their reported queue lengths, subject to the maximum number of slots that can fit within the superframe.

In this way, GoMacH combines the low overhead of contention-based access under light traffic with the service capacity benefits of scheduled transmissions under bursty traffic. Its adaptation is therefore local and traffic-driven, and the behavior of GoMacH remains governed by predefined parameters such as CP , CP_{max} , SF , and the maximum number of allocated slots, which motivates the need for an external control loop able to reconfigure these parameters dynamically and according to broader network conditions.

Overall, the adaptive MAC literature shows that adaptation is essential for LLWNs, but also reveals important limitations in how existing protocols adapt. These mechanisms improve MAC performance under specific conditions, but their adaptation logic is usually embedded locally within the protocol and driven by predefined rules, thresholds, or application-specific assumptions. As a result, adaptation is often reactive and parameters are modified after performance degradation has already occurred.

In multi-hop LLWNs, nodes do not experience the same traffic pressure or forwarding responsibility. A node close to the sink may suffer from queue buildup and retransmissions because it forwards traffic for many descendants, while a leaf node may remain lightly loaded. Local adaptation alone may therefore miss broader network trends and emerging bottlenecks. This motivates a more proactive and network-aware adaptation model, in which runtime measurements are collected across nodes and used to anticipate future reliability, delay, or energy degradation before it becomes critical. The next subsection therefore reviews Machine Learning (ML)-based approaches for MAC performance prediction and MAC-layer adaptation in LLWNs.

3.3.2 Machine Learning for MAC-layer in LLWNs

Machine learning has increasingly been used to predict communication performance and support adaptive decision-making in LLWN networks. This direction is motivated by the difficulty of deriving accurate analytical models for dynamic wireless environments. Link quality, queue occupancy, contention, traffic burstiness, and topology-dependent forwarding load interact in complex ways. Machine-learning models can learn these relationships from data and provide predictions that support proactive adaptation.

A first group of works focuses on predicting communication performance metrics such as reliability, energy consumption [112], delay [113], throughput [114], Signal-to-Noise Ratio (SNR) [115], and Packet Delivery Ratio (PDR) [116].

In this direction, Ateeq et al. [112] show that communication performance can be inferred from a combination of stack-level parameters collected across the application, MAC, and physical layers. Their model uses a public dataset collected over several months.

Their deep neural models use features such as traffic characteristics, queue behavior, retransmission-related parameters, transmission power, and distance to predict PDR and energy consumption. The results show that deep learning can capture the nonlinear relationship between protocol parameters and network performance more accurately than linear regression, gradient boosting, random forest, and simpler neural-network baselines. The same research direction is extended in [113] to packet-delay prediction for delay-sensitive IoT applications. In this case, the model relies mainly on MAC- and physical-layer parameters and uses a deep neural network to estimate packet delay, outperforming linear regression, boosted-tree regression, baseline neural networks, and decision-tree regression. These works show that deep neural models can improve prediction accuracy, but this comes at the cost of higher model complexity, since deep neural networks generally require more parameters, larger training datasets, and greater computational resources than many classical machine-learning models.

Hameed et al. [114] focus on throughput prediction on a real IEEE 802.15.4 testbed using ML. The raw data are collected from transmitted and received packets. From these traces, the authors engineer additional features such as total transmitted messages, total received messages, transmission success, average latency, PDR, and throughput. They evaluated several supervised ML regression models, including multiple linear regression, decision-tree regression, gradient-boosted regression trees, support-vector regression (SVR), and k-nearest-neighbor regression (KNN) that are trained separately. The results show that multiple linear regression provides the best overall trade-off between prediction accuracy and computational cost. This work is important because it uses real testbed traffic and lightweight regression models compared to neural networks.

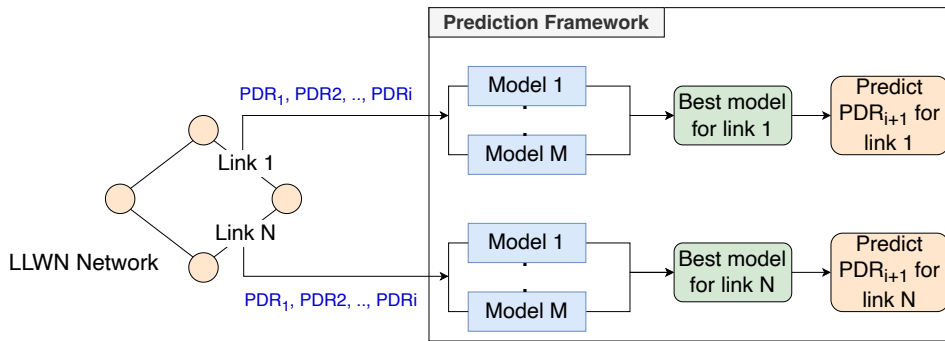


Figure 3.11: Per-link prediction framework [116]

More recently, Si-Mohammed and Theoleyre [116] propose a data-driven digital-twin approach for wireless link-quality prediction in real deployments. Instead of using a single global predictor, they train individualized models per link to capture link heterogeneity, as illustrated in Figure 3.11. They evaluate SVR, gradient boosting, decision trees, AdaBoost, and Extra Trees, and compare a fixed-model strategy with an adaptive one that selects the best-performing model based on recent prediction errors. The adaptive strategy consistently improves performance for short-term prediction although errors increase over

longer horizons. This work addresses per-link heterogeneity and online model selection, but maintaining separate models per link introduces additional computational and operational overhead, especially in dense networks.

Overall, these works show that ML can accurately predict several network performance indicators. They demonstrate that such predictions can be effectively derived from stack parameters spanning different layers. In addition, several studies highlight that relatively lightweight ML models can achieve high prediction accuracy with lower computational cost, making them suitable for real-time prediction. However, most existing approaches remain prediction-oriented rather than control-oriented. They estimate future performance, but they do not directly determine which parameters should be reconfigured, when the reconfiguration should be applied, or how different nodes should be configured according to their topological roles.

A second group of works goes beyond prediction and uses ML to directly adapt MAC-layer behavior. A recent survey [117] shows that ML-based MAC research has grown significantly during the last decade.

Reinforcement-learning-based approaches are used widely in distributed MAC design because they enable nodes to learn suitable configurations through trial-and-error interaction with the environment. Gomes et al. [118] propose a self-organizing MAC framework that switches between contention-free and contention-based MAC protocols, namely TDMA and CSMA. The switching decision is driven by network conditions such as traffic load, PDR, delay, and channel conditions. The results show that the dynamic switching between MAC techniques achieves a higher packet reception ratio, lower control overhead, and lower energy consumption. However, the reconfiguration granularity remains coarse because the action consists of selecting an entire MAC protocol rather than fine-grained tuning of protocol parameters or per-node scheduling decisions.

Musaddiq et al. [119] propose a cross-layer optimization mechanism. The model uses channel collision probability obtained from the MAC layer and combines it with network-layer ranking states to improve forwarding decisions. The results show improvements in packet reception and energy consumption. Wu et al. [120] propose an approach that directly reconfigures the contention-period length according to traffic load. A cluster-head node periodically observes the number of successful, collided, and idle slots and reconfigures the next contention-period length accordingly. The results show that dynamic MAC reconfiguration achieves higher throughput, lower end-to-end delay, and lower failed-contention energy consumption than static configuration.

In addition, different ML approaches are used to optimize MAC-layer performance. Albalawi et al. [121] propose a priority-aware hybrid MAC protocol. The protocol combines TDMA with bitmap-assisted (BMA) contention-based MAC, a contention-reduction scheme in which nodes use a bitmap to indicate transmission intentions or reserve slots to avoid collisions. Continuous monitoring nodes use allocated TDMA slots, while periodic-monitoring nodes use the BMA protocol. For periodic-monitoring nodes, the protocol adopts an

ML-based mechanism that estimates how many transmissions are needed and when these transmissions should occur, based on their data-generation patterns, in order to configure bitmap-assisted access. The evaluation shows reduced energy consumption and lower transmission latency compared with static benchmarks.

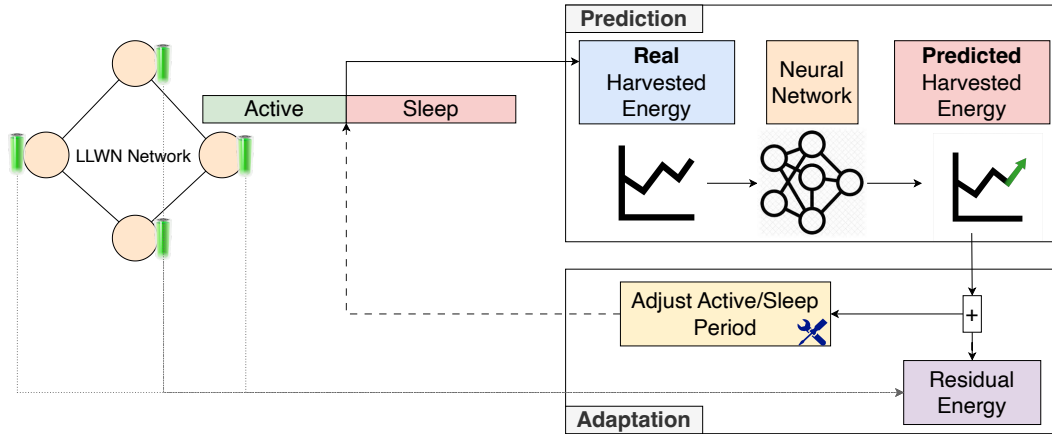


Figure 3.12: Prediction-based adaptive duty-cycle for solar energy-harvesting networks [122]

Recent work connects prediction and MAC adaptation to provide proactive MAC-layer reconfiguration. Sarang et al. [122] propose a prediction-based adaptive duty-cycle MAC protocol for solar energy-harvesting networks. As illustrated in Figure 3.12, the key idea is to use a neural network to predict future harvested solar energy and combine this prediction with the current residual energy to adjust the receiver duty cycle. The protocol also includes a self-adaptation mechanism in which the receiver advertises its duty cycle in the wake-up beacon, allowing sender nodes to align their sleep duration with the receiver schedule and reduce idle listening. The reported results show reductions in packet delay and energy consumption compared with existing static energy-harvesting MAC protocols. However, the adaptation mainly targets energy-harvesting networks and focuses on a single-hop clustered topology rather than general multi-hop LLWNs.

Overall, existing ML-based works show that learning techniques can support LLWN adaptation in two main ways. First, prediction-oriented studies demonstrate that ML frameworks can infer important network metrics from runtime or historical measurements. Second, adaptation-oriented studies show that ML can be used to adjust network protocol configurations and enhance performance. However, most existing studies address only one part of the adaptation loop. Prediction-oriented works usually stop at performance estimation and do not translate forecasts into concrete protocol changes. Conversely, MAC adaptation works often embed the learning logic inside a specific protocol and react to observed conditions, but they do not provide a complete architecture that continuously collects network metrics, infers near-future performance, decides which protocol changes are required, and pushes these changes to the nodes before degradation becomes critical.

3.3.3 Conclusion

Adaptive MAC protocols show that static configurations are insufficient when traffic load, node density, packet priority, or application requirements change. They also demonstrate the value of hybrid MAC protocols that combine contention-based access with scheduled transmission opportunities. However, their adaptation logic is usually local and driven by predefined configurations, thereby missing broader and more complex network conditions.

Machine learning has emerged as a strong candidate for providing intelligent and efficient management of wireless networks by capturing the complex relationships between network conditions, network configuration, and performance variations. Machine-learning-based studies show that runtime measurements can be used to predict different network performance metrics. These prediction models provide a concrete basis for anticipating future performance degradations, thereby allowing the network to adapt proactively and avoid prolonged degradation phases. Other works go further to adapt network configuration, particularly MAC behavior. Collectively, these studies demonstrate the potential of machine learning for MAC-layer control.

3.4 Conclusion

This chapter reviewed the state of the art along three dimensions that are central to the evolution of LLWNs. The first part of the chapter showed that conventional firmware-image updates remain the dominant maintenance mechanism in LLWNs, but they are poorly suited to frequent and selective evolution of the network stack. Existing alternatives reduce some update overheads, but they remain largely application-oriented and relatively coarse-grained. The second part of the chapter examined control plane deployment models in LLWNs, with particular attention to hybrid deployments. Hybrid deployments offer a more balanced direction by combining local autonomy with centralized coordination. Nevertheless, existing hybrid solutions in LLWNs still provide only limited interaction between local and global control logic, and they do not fully address the coordinated and state-aware runtime evolution of the communication stack. The third part of the chapter focused on MAC-layer adaptation. Adaptive MAC protocols improve performance under specific conditions, but their adaptation mechanisms are typically reactive, local, and rule-based. Machine-learning-based studies further show that network measurements can be used to predict important performance metrics and directly adapt MAC behavior. However, most approaches treat prediction and adaptation as two separate tasks, without providing an integrated framework for proactive reconfiguration.

This thesis addresses these gaps through three contributions in the following chapters. In Chapter 4, it first proposes a modular network-stack design based on lightweight virtual functions, providing fine-grained decomposition and enabling selective and fast runtime updates. This fine-grained design is then supported by semantic and state-aware

management of network-stack updates, enabled by hybrid control plane deployments as illustrated in Chapter 5. Finally, in Chapter 6, the thesis proposes proactive traffic-aware MAC-layer control based on ML. The framework integrates real-time measurements with short-term predictions to anticipate future performance degradation. It also uses recommendation models to propose proactive MAC-layer reconfigurations based on these predictions in order to maintain performance and minimize power consumption.

A Modular Network Stack Design for LLWNs

Chapter contents

4.1	Introduction	54
4.2	The FINE-STACK Design	55
4.2.1	Design Concepts	55
4.2.2	Network Stack	55
4.2.3	Complete Architecture	57
4.2.4	Update Process	58
4.3	Evaluation	59
4.3.1	Packet Delivery Ratio (PDR)	61
4.3.2	Update Size and Time	62
4.3.3	Post-update Convergence	64
4.3.4	Update Overhead	64
4.3.5	Security and Recovery Cost	66
4.3.6	Memory Footprint	67
4.3.7	Power Consumption	68
4.3.8	Execution Time	69
4.4	Conclusion	70

4.1 Introduction

The inherent dynamic nature of wireless networks creates a clear need for network programmability to enable runtime reconfiguration of the protocol stack to respond to changing conditions and requirements, improve performance, while reducing overhead and respecting LLWN constraints. In many LLWN operating systems such as RIOT [68] and Contiki-NG [69], the application logic, network stack, and functional modules are compiled into a single firmware image. As a result, network stack updates require rebuilding and redistributing the entire image [13].

This thesis started the study by analyzing the firmware size of different operating systems in LLWNs to show that this kind of update is not suitable for frequent and dynamic network stack updates. The delivery of large binaries is particularly challenging, resulting in long update durations, increased congestion with application traffic, higher power consumption, and a greater probability of failure. If the update fails during or after a long download phase, recovery may be costly because a significant portion of the image may need to be retransmitted. As a result, the larger the update and the longer the update process, the more vulnerable it becomes to the network conditions and the constraints of LLWNs.

The bulky and rigid nature of image-based updates creates a significant gap between network requirements, network constraints, and the currently available solutions. These limitations motivate a different approach to network-stack evolution, based on modularity and selective updates. Instead of replacing the whole firmware image, the update mechanism should target only the affected network function, preserve the unaffected protocol state whenever possible, and keep communication, memory, energy, and execution overhead low.

This chapter proposes the *Flexible architecture for low-power lossy IoT Networks with programmable network STACK and semantic runtime updates (FINE-STACK)*, a unified and adaptive network architecture for LLWNs. The core component of FINE-STACK is a modular network stack in which network functions are implemented as lightweight Virtual Functions (VFs). These VFs provide a fine-grained structure that can be dynamically composed to support different network protocols by chaining sets of VFs. The modular stack is managed through a central orchestrator that enables selective OTA updates. FINE-STACK thus provides a wide range of programmability, from fine-grained parameter tuning, such as adjusting the retransmission limit at the MAC layer, to full protocol or function replacement, such as augmenting UDP with reliable features.

This chapter validates the core mechanism through a UDP-based proof-of-concept. The validation aims to assess the runtime performance enhancements enabled by the modular design of FINE-STACK, the resulting update optimization, and its compliance with LLWN constraints. More complex protocol implementations and state-aware control plane reconfiguration are addressed in the next chapter.

4.2 The FINE-STACK Design

FINE-STACK introduces a modular network stack coupled with centralized orchestration. In this architecture, core networking functions are decomposed into Virtual Functions (VFs) that can be composed to form complete network protocols. This modular design gives the stack a highly fine-grained structure, enabling precise and efficient management of individual protocol components. Such fine granularity is complemented by remote OTA updates, which make it possible to apply fast, selective, and lightweight modifications to the stack without requiring full firmware replacement.

The central orchestrator is responsible for managing the network stack across the nodes by updating or replacing active VFs as needed, thereby ensuring consistent and safe runtime updates throughout the network.

4.2.1 Design Concepts

The FINE-STACK architecture is founded on four key design principles:

1. **Fine-Grained Modularity:** Instead of a monolithic stack, protocol functionality is decomposed into independently updatable services (Virtual Functions). This enables selective changes with much smaller update payloads and shorter update times, thereby reducing bandwidth usage, energy consumption, and update failure probability.
2. **Logical and Execution Isolation:** Network functions are isolated from each other and from the operating system, preventing fault propagation and simplifying lifecycle management. Isolation improves reliability, supports safe rollouts/rollbacks, and eliminates interdependencies that hinder evolution.
3. **Over-the-Air Update Capability:** The advantages of modularity are fully realized when combined with a secure and reliable remote update mechanism. Integrating OTA capabilities with a modular architecture enables frequent and efficient updates, ensuring continuous optimization of network performance over time.
4. **Central Network Orchestrator:** A logically central orchestrator supervises the network stack across devices to coordinate configuration, scheduling, and adaptation. In addition to orchestration, it can host control logic in some deployments when global optimization is required, improving performance and policy consistency.

4.2.2 Network Stack

The network stack is built from **Virtual Functions (VFs)**, which are minimal building blocks that each implement a specific network function through well-defined interfaces. A *submodule* denotes a logical unit formed by composing two or more VFs in order to realize

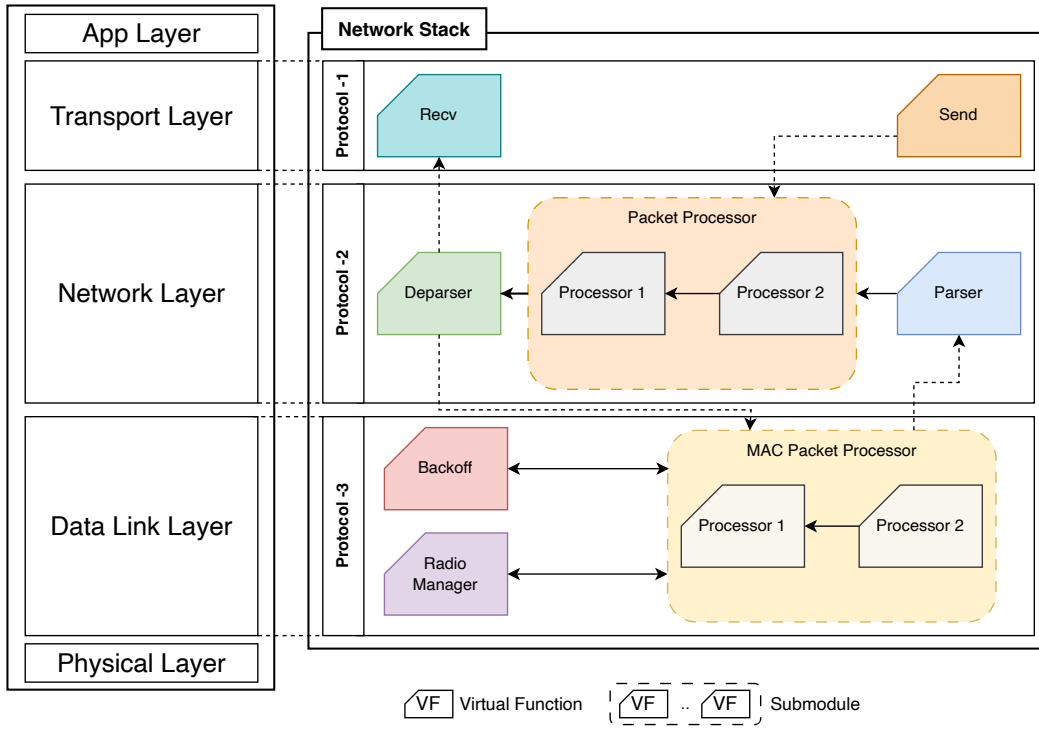


Figure 4.1: FINE-STACK VF-based network stack in an LLWN device

a concrete capability or feature within the network stack. A complete network protocol can therefore be constructed by combining VFs and submodules, enabling the implementation of communication protocols across different layers of the stack, as illustrated in Figure 4.1.

For example, at the network layer, a *Parser* VF may process a received packet and extract its header fields before forwarding it to a *Packet Processing* submodule composed of multiple VFs. Subsequently, another VF may act as a *Deparser* to prepare the packet for transmission. VFs, whether used individually or grouped into submodules, can implement a wide range of mechanisms, including backoff algorithms, packet scheduling, link-quality estimation, routing decisions, and neighbor discovery. The primary goal of this stack design is protocol agnosticism, allowing any protocol, whether belonging to the data plane or the control plane, to be implemented entirely through VFs.

The adoption of lightweight VFs is driven by the need to minimize overhead in resource-constrained LLWNs, ensuring strong isolation and independence both among VFs themselves and between VFs and the underlying operating system. This isolation guarantees *safe* and *secure* updates of network functions without compromising system stability. Constructing the network stack at the VF level introduces a high degree of modularity, enabling reuse of the same VF across different layers and protocols, thereby improving *reusability* and reducing redundancy. This fine-grained modular design enables *fast*, *selective*, and *consistent* runtime updates of the stack, offering an efficient and low-overhead update mechanism, especially when integrated with remote OTA management described in the next section.

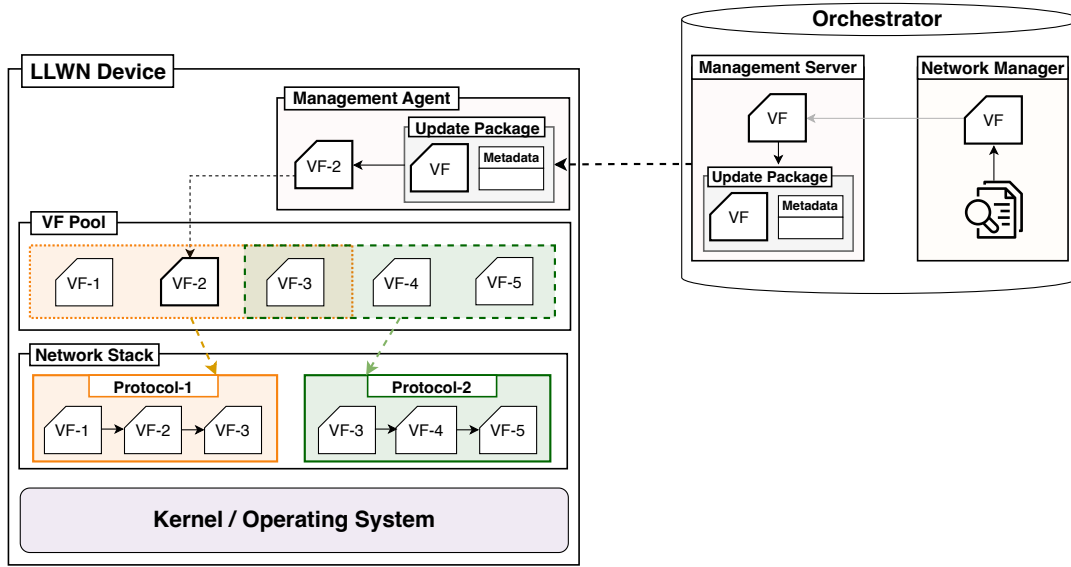


Figure 4.2: FINE-STACK architecture. The orchestrator manages the VF-based network stack through the Network Manager and Management Server, while each device maintains a VF Pool and Management Agent to validate, install, and coordinate updates across constrained nodes.

4.2.3 Complete Architecture

Figure 4.2 details the FINE-STACK architecture. To implement the modular network stack in LLWN devices, each device maintains a **VF Pool** that stores all locally available functions, which can be dynamically triggered by the network stack to execute specific protocol- or layer-level operations. This architecture promotes strong modularity and reusability, allowing the same VF to be invoked by multiple layers or protocols, thereby improving memory efficiency and reducing redundancy.

To manage network stacks across LLWN devices, FINE-STACK integrates a central orchestrator responsible for monitoring and configuring devices over the air. The orchestrator includes two main components: (i) the **Network Manager** and (ii) the **Management Server**.

The **Network Manager** is responsible for collecting network metrics, evaluating network conditions, and making adaptive decisions accordingly. These decisions may involve modifying an active VF or replacing it with a new one. Once an update is required, the Network Manager generates the new VF(s), compiles and validates them, and transfers the resulting binaries to the Management Server for distribution.

The **Management Server** serves as the orchestrator’s interface to LLWN devices and ensures secure and reliable update delivery. To achieve this, each VF is encapsulated with metadata to form an *Update Package*. The metadata ensures secure delivery through authentication (using cryptographic keys) and integrity verification (using hash functions), as well as version control. It also contains installation-related information such as the

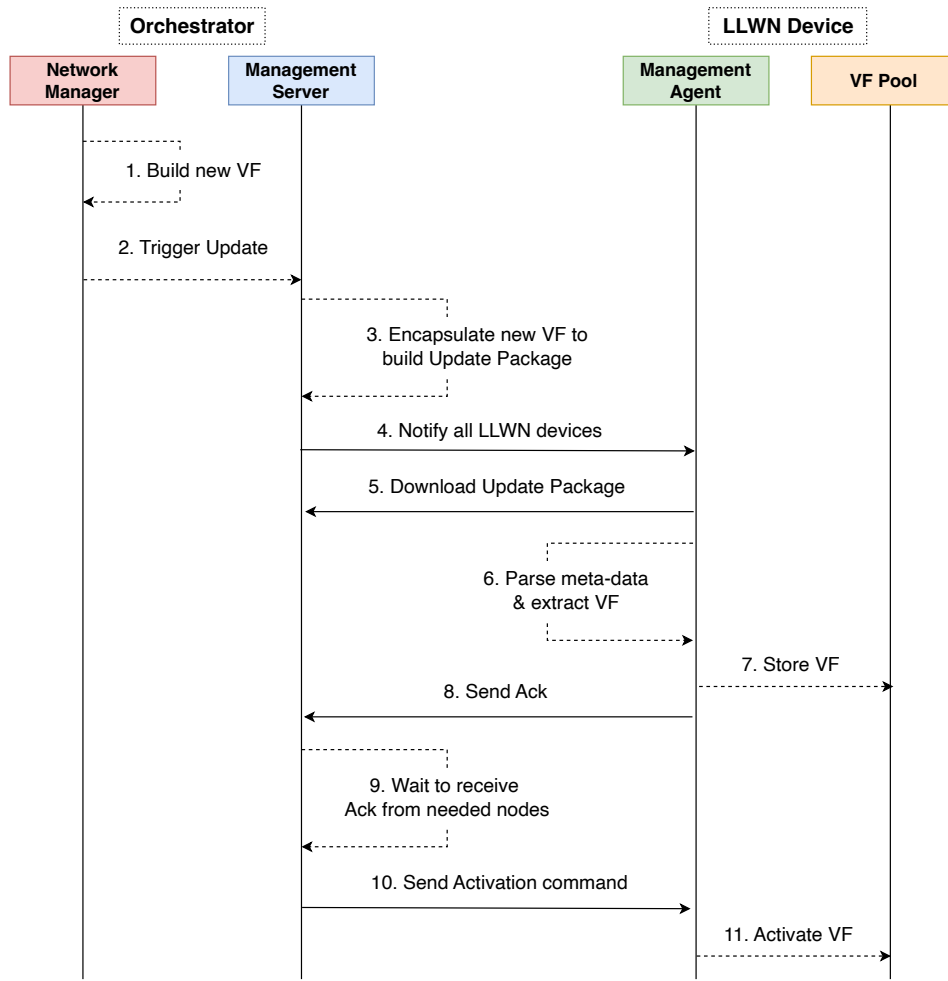


Figure 4.3: Update process. Two-phase update procedure (install then activate) ensuring network-wide consistency: nodes first authenticate and store the VF in the pool, then activate it synchronously after enough nodes acknowledge installation.

target device type, update size, and the VF's designated position within the VF Pool.

On the device side, each node hosts a **Management Agent**, which forms the endpoint of the management loop with the Management Server. Upon receiving an Update Package, the Management Agent authenticates, verifies, and decapsulates the VF and metadata. The validated VF is then installed into the VF Pool, guided by the provided installation parameters.

4.2.4 Update Process

The update process, illustrated in Figure 4.3, follows a coordinated two-phase procedure—install, then activate—to preserve network-wide consistency. It begins when the orchestrator decides to initiate an update and generates the corresponding VF binaries (1). The new VF is then transferred to the Management Server (2), which is responsible for

delivering the update to the LLWN devices.

The Management Server encapsulates the VF with the required security and installation metadata to create an Update Package (3) and subsequently notifies all Management Agents in the LLWN devices about the available update (4). Each Management Agent then authenticates the orchestrator (signature verification) before initiating the download of the new VF (5). Upon successful reception, the Management Agent parses the Update Package, extracts the hash to verify integrity, retrieves the installation metadata (6), and stores the new VF in the VF Pool, awaiting an activation command (7).

Once installation is complete, each Management Agent sends an acknowledgment (Ack) message to the Management Server (8). The Management Server waits until a sufficient number of LLWN devices confirm successful installation (9). After that, the Management Server issues an activation command to all Management Agents (10), enabling the new VF simultaneously and prompting them to activate the new VF in the pool (11). This two-phase scheme ensures that all devices run the same VF version, maintaining semantic consistency across the network.

For reliability, if an acknowledgment is not received within a specified *timeout*, the Management Server retransmits the Update Package up to a predefined *Maximum Number of Retries*. Nodes that remain unresponsive are marked as inaccessible. The activation process proceeds once the *Minimum Number of Updated Devices* threshold is reached.

4.3 Evaluation

This section presents a proof-of-concept implementation on real hardware platforms to experimentally validate the proposed FINE-STACK architecture. Specifically, the User Datagram Protocol (UDP) is decomposed into two VFs: UDP-Send and UDP-Recv. This protocol is chosen for its simplicity, making it an ideal candidate for an initial validation of the framework. To demonstrate FINE-STACK's flexibility, UDP is then extended to support reliability through the addition of sequence numbers, acknowledgment messages (ACKs), and a retransmission queue.

For benchmarking, the FINE-STACK UDP implementation is compared with the native UDP module of the RIOT operating system [68]. Figure 4.4 illustrates the network stack and update methodology of both implementations. In the native implementation, any update or modification of UDP requires retransmission of the entire firmware image. In contrast, FINE-STACK selectively updates only the corresponding VFs, namely UDP-Send and UDP-Recv. A direct quantitative comparison with prior work is not feasible due to differences in supported update granularity and underlying operating systems, which strongly influence timing and memory behavior.

Femto Containers (FCs) [88] are used to implement the VFs. Femto Containers are a middleware framework designed to deploy lightweight virtual functions on constrained

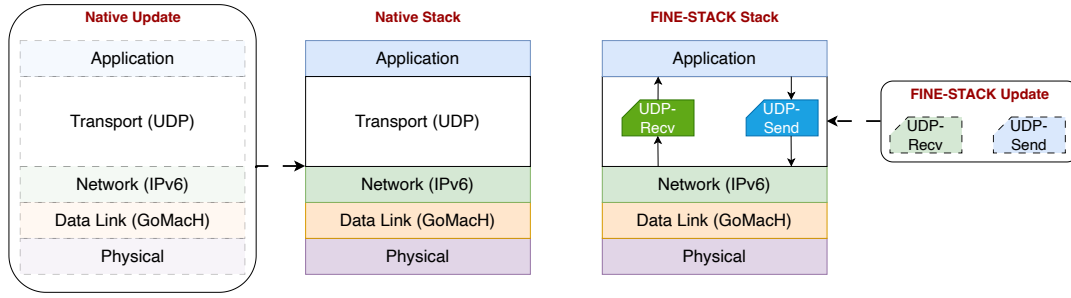


Figure 4.4: Native and FINE-STACK stacks and update methodology

devices. More details are provided in Section 3.1.2.3. Although the original design of FCs targets the application layer, we extend their scope to other layers of the network stack. It is important to note that our architecture is not tied to a specific virtualization technology. While we adopt Femto Containers in our implementation, any other lightweight virtualization technique could also be employed to realize the proposed VFs.

For this implementation, the Constrained Application Protocol (CoAP) [44] is employed to establish the management plane, where the Management Server and Management Agent operate as a CoAP server and client, respectively. Software Updates for the Internet of Things (SUIT) [123], a standardized and secure mechanism to describe, authenticate, and manage software updates for constrained IoT devices, is used to encapsulate VF update files and metadata in SUIT manifests. The resulting management plane follows a CoAP-SUIT/UDP/IPv6/6LoWPAN stack, ensuring compatibility with existing IoT standards and efficient operation in constrained environments.

All experiments are conducted on the SLICES-FR testbed at the Lille site, using IoT-LAB M3 nodes representative of typical LLWN devices. Each node features 256 kB of Flash memory, 64 kB of RAM, an ARM Cortex-M3 processor, and a 2.4 GHz transceiver. To ensure reproducibility, the complete implementation is publicly available on GitHub ¹.

To emulate realistic operating conditions, the GoMacH duty-cycling protocol [111], introduced in Section 3.3.1, is integrated to reduce power consumption. The network stack used in our experiments follows the protocol suite composed of UDP, IPv6-RPL, 6LoWPAN, and GoMacH.

As shown in Figure 4.5, the experimental topology consists of 25 nodes physically deployed across the SLICES-FR testbed. One node operates as the Border Router (BR), providing end-to-end connectivity to the Orchestrator, while the remaining nodes form a multi-hop topology using the RPL routing protocol [41]. Depending on their physical placement, the network depth extends up to three hops from the BR. Each node runs a temperature-sensing application that periodically transmits a 1-byte UDP packet containing the measured temperature to the BR every 40 seconds, with staggered transmission intervals to avoid collisions.

¹https://github.com/ahmahmod/fine_stack_0

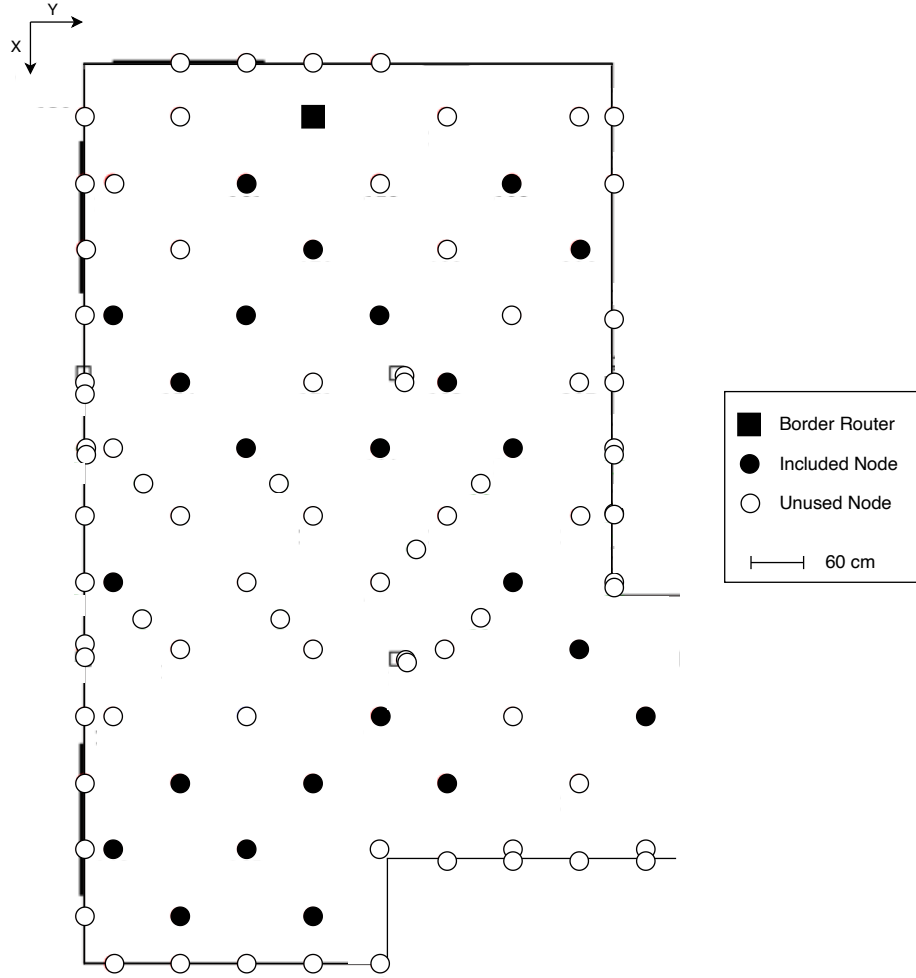


Figure 4.5: Topology on SLICES-FR testbed. It includes 25 M3 nodes, one node serves as the Border Router (BR), providing connectivity to the orchestrator.

The evaluation focuses on three key aspects: (i) the impact of updates on network performance, (ii) update performance in terms of size, duration, security, and recovery, and (iii) system-level metrics such as resource utilization and energy efficiency.

4.3.1 Packet Delivery Ratio (PDR)

To demonstrate the benefits of the proposed UDP update, this section designs a controlled scenario in which link quality degradation renders the connection unreliable and causes packet loss. To mitigate this issue, FINE-STACK dynamically updates the running UDP implementation to a reliable variant by integrating sequence numbers, acknowledgments, and retransmissions upon timeout.

In this scenario, both the native and FINE-STACK implementations of UDP are

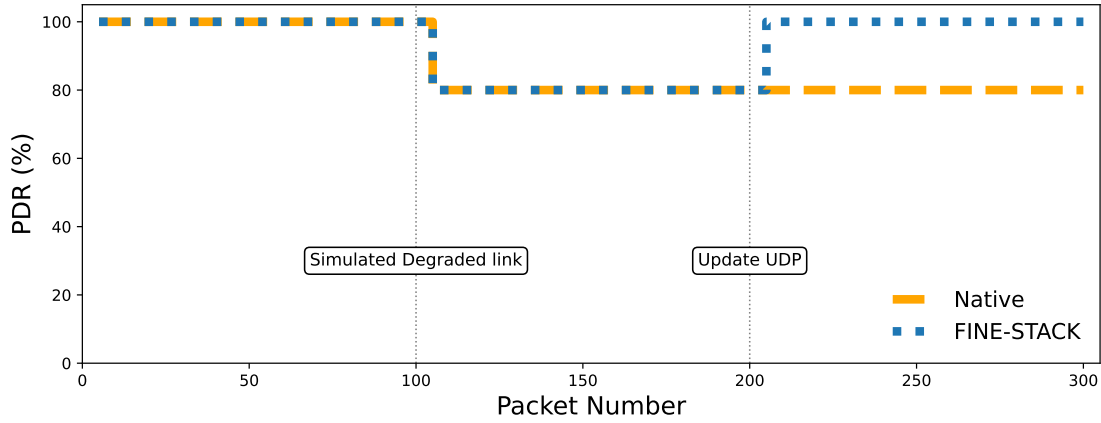


Figure 4.6: Packet Delivery Ratio (PDR) comparison between FINE-STACK and native implementation. FINE-STACK restores PDR by updating the network stack at runtime.

evaluated. Two neighboring nodes from the topology, denoted as node A and node B, are selected. Node A transmits 300 UDP packets to node B in three successive stages of 100 packets each. The evolution of the PDR across the three stages is depicted in Figure 4.6. In the first stage, both implementations use the baseline unreliable UDP without any induced loss, thereby achieving a 100% Packet Delivery Ratio (PDR). In the second stage, node B is configured to drop every fifth packet, simulating link degradation and resulting in a 20% packet loss in both implementations. In the third stage, UDP is updated in FINE-STACK to a reliable variant that retransmits lost packets at the next available transmission opportunity. As a result, FINE-STACK restores the PDR to 100% at runtime and without service interruption, whereas the native implementation remains at 80%.

This experiment highlights the importance of fast and selective network stack reconfiguration to maintain the QoS required by diverse LLWN applications. Although the native implementation could also achieve reliability by updating UDP, doing so would require a full-image OTA update involving the transmission of a large binary image, longer update duration, and a reboot that interrupts communication.

4.3.2 Update Size and Time

Installing reliable UDP extensions in the native implementation requires modifying the operating system’s UDP module and transmitting the full firmware image to all nodes. In contrast, FINE-STACK updates and transmits only the two VFs that implement the UDP protocol, namely UDP-Send and UDP-Recv, rather than the full firmware. This subsection presents a comparison of update size and duration between the two approaches.

Table 4.1 summarizes the update size transmitted to each node. The native full-image update produces a substantially larger payload (109.6 kB) compared with FINE-STACK (5.54 kB). This reduction directly translates into fewer packets transmitted from the orchestrator to the nodes. Assuming a 64-byte payload per packet, as typically used for

Table 4.1: Update size comparison between FINE-STACK and native implementation. FINE-STACK reduces the update size by 94.9%

Metric	Native	FINE-STACK	
		VF1 (UDP-Send)	VF2 (UDP-Recv)
Updated files	Full firmware		
Size (kB)	109.6	2.67	2.87
Total size (kB)	109.6	5.54	

CoAP update delivery in LLWNs, the native update requires approximately 1,713 packets, whereas the FINE-STACK-based update needs only 87 packets ($42 + 45$), representing a 94.9% reduction in update size.

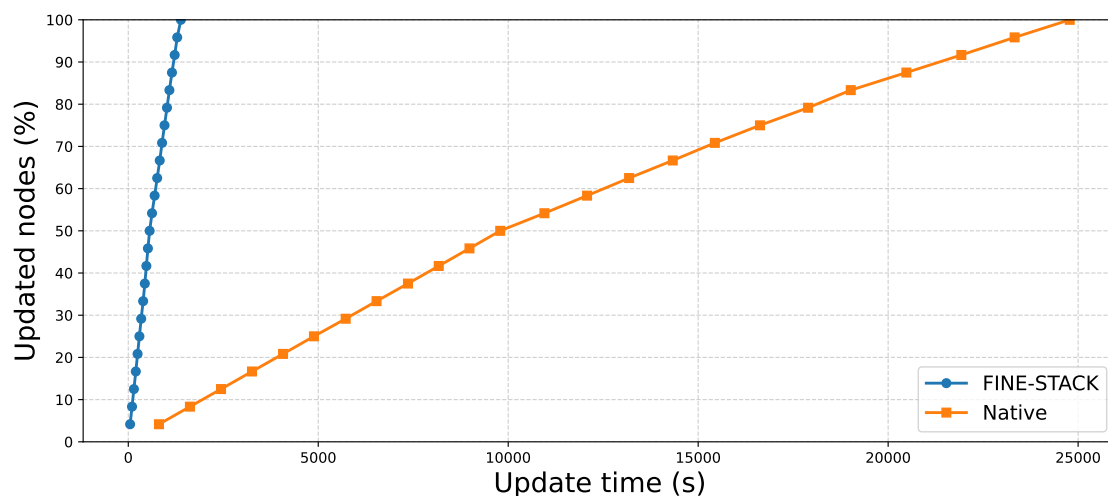


Figure 4.7: Update time comparison between FINE-STACK and native implementation. FINE-STACK reduces the update time significantly by 94.4%

The corresponding update duration scales critically with network size. As shown in Figure 4.7, FINE-STACK reduces the total update time by 94.4% compared to the full-image approach. The native update experiences substantial delays due to multi-hop communication, duty-cycling effects, and concurrent application traffic, all of which increase collision probability and packet loss. Under the same conditions, FINE-STACK completes the update in less than 1,385 s thanks to its significantly smaller update size, whereas the native update exceeds 24,779 s. The measured update time also includes retransmissions to recover from packet loss at both the MAC layer (GoMacH) and the application layer (CoAP).

Overall, FINE-STACK demonstrates superior scalability and efficiency, particularly in dense, multi-hop, and lossy networks where minimizing update size is crucial for rapid reconfiguration. Larger updates introduce higher packet overhead, increased loss probability, more retransmissions, and elevated network contention, all of which degrade network performance.

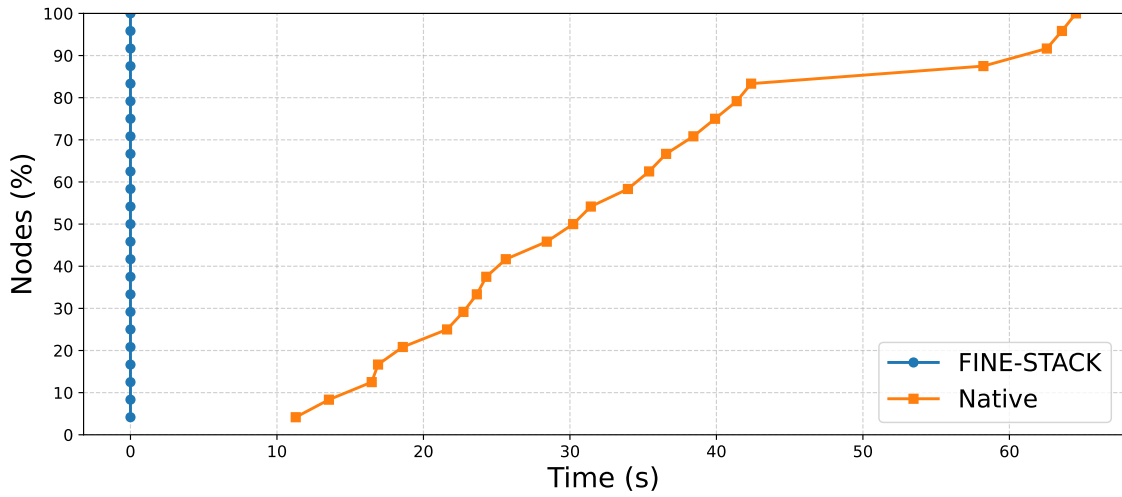


Figure 4.8: Post-update convergence time. While the native implementation requires more than 60 s to reconverge, FINE-STACK avoids any reconvergence delay

4.3.3 Post-update Convergence

One major limitation of full-image firmware updates is the loss of volatile network state caused by the mandatory reboot that follows installation. This reboot interrupts network connectivity, forcing nodes to re-establish communication and rebuild routing tables. In contrast, FINE-STACK allows selective updates of VFs without requiring a device reboot, preserving active connections and eliminating the need for time-consuming reconvergence.

The post-update reconvergence time of both FINE-STACK and the native implementation is illustrated in Figure 4.8. Here, post-update reconvergence time denotes the duration required for the Border Router (BR) to re-register a node in its forwarding table and restore end-to-end communication. In the native implementation, this process takes more than 60 s and increases with hop distance from the BR, as nodes must sequentially exchange routing messages to rebuild the topology. In contrast, FINE-STACK preserves the existing network state during the update by avoiding a reboot, allowing nodes to remain fully operational and effectively eliminating reconvergence delay.

This protects ongoing application sessions from service interruptions and scales gracefully with network depth, demonstrating that fine-grained VF updates can achieve seamless operation while enabling rapid and reliable network reconfiguration.

4.3.4 Update Overhead

In both FINE-STACK and the native implementation, the update process requires the exchange of a SUIT manifest between the orchestrator and the updated nodes. This manifest carries the metadata of the update package required for authentication, integrity verification, version control, and installation. Table 4.2 reports the size of the control metadata overhead and the associated processing time. Note that the overhead reported in

Table 4.2: Update overhead cost. The cost in FINE-STACK is multiplied by the number of updated VFs.

Metric	Native	FINE-STACK	
		VF1 (UDP-Send)	VF2 (UDP-Recv)
Update overhead load (bytes)	513	275	275
Total (bytes)	513	550	
Processing-time overhead (ms)	3689	3689	3689
Total average (ms)	3689	7378	

this section excludes the bytecode or binary payload corresponding to the new functionality being deployed, which is already separately reported in Table 4.1.

In the native full-image update, the manifest size is 513 bytes, whereas in FINE-STACK UDP-Send and UDP-Recv VFs require manifests of 275 bytes. This difference stems from the metadata associated with each update type: the native manifest describes a complete bootable firmware image, while the FINE-STACK manifest describes an individual VF. However, the native manifest is transmitted only once per full-image update, whereas FINE-STACK requires one manifest per updated VF. Consequently, the management overhead in FINE-STACK increases linearly with the number of updated VFs, i.e., $N \times 275$ bytes, where N denotes the number of VFs included in the update. Using a 64-byte CoAP payload in our evaluation, manifest transmission requires 9 control packets for the native full-image update to deliver the 513-byte manifest. In FINE-STACK, manifest transmission requires 5 control packets for each VF.

A similar trend is observed for the processing-time overhead. Processing includes the security and installation steps, such as signature authentication, version verification, sequence-number validation, orchestrator-ID verification, device-class verification, and digest verification after downloading the binary file. As reported in Table 4.2, each update requires approximately 3689 ms of average processing time among the nodes. For the native implementation, this processing cost is incurred only once, since the full firmware image is described by a single manifest. In FINE-STACK, however, each updated VF is handled as an independent update unit and, therefore, requires its own manifest processing. Consequently, the average processing overhead scales linearly with the number of updated VFs, i.e., $N \times 3689$ ms. Therefore, when two VFs are updated, the total security verification time in FINE-STACK roughly doubles that of the native update.

Overall, the increased management overhead observed in FINE-STACK is the direct cost of fine-grained modularity, where each VF is handled independently, with its own metadata, security checks, and installation process. Nevertheless, this overhead remains modest when a small number of VFs are transmitted compared to the substantial reduction in update payload size and update time achieved by avoiding full-image firmware replacement.

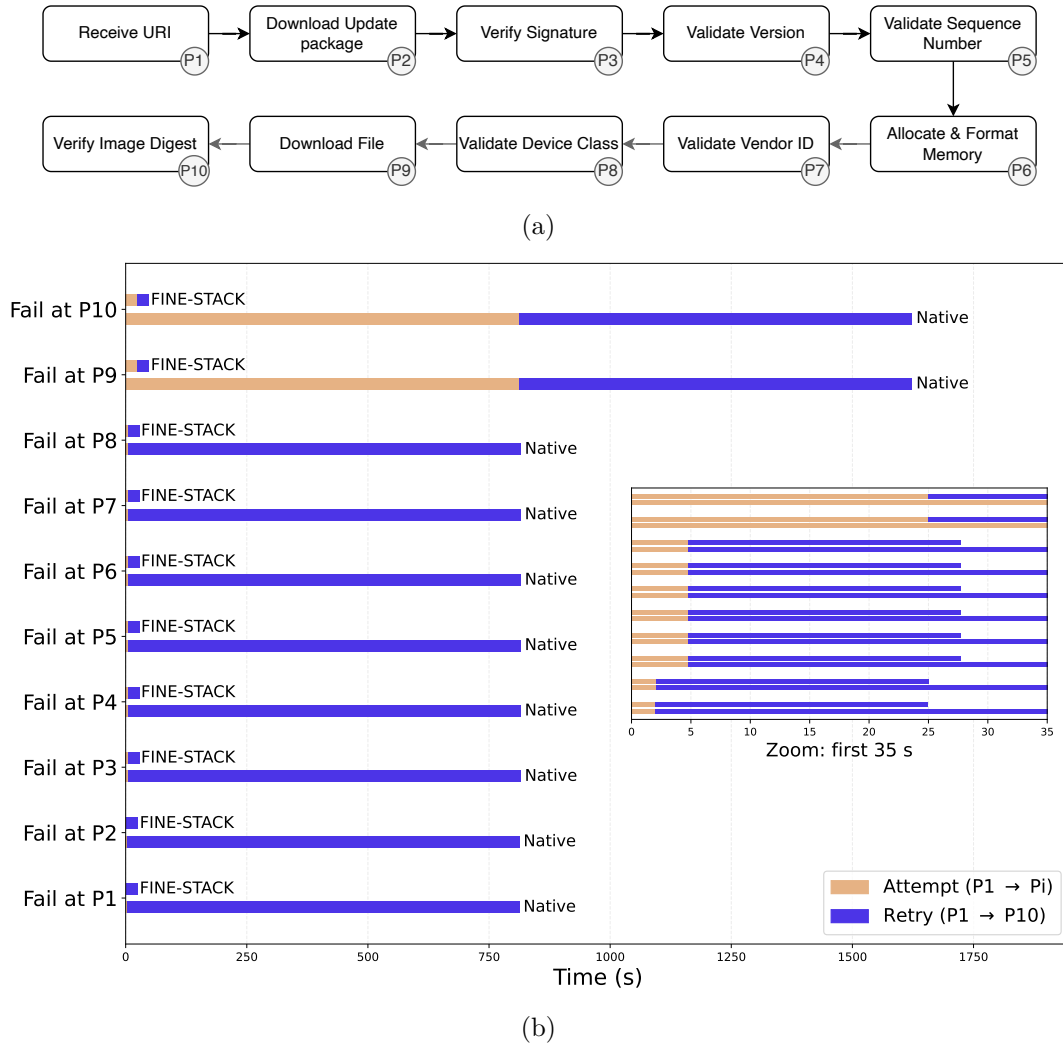


Figure 4.9: Update and recovery: (a) Update phases, (b) Recovery cost at each phase

4.3.5 Security and Recovery Cost

FINE-STACK incorporates multiple security mechanisms to ensure the authenticity, integrity, and reliability of the update process. Figure 4.9-(a) illustrates the sequence of update phases together with the corresponding security checks, including signature and digest verification, sequence-number and version validation, as well as vendor and device-class identification.

FINE-STACK also includes recovery mechanisms to preserve robustness in the presence of update failures. If any phase of the update process fails, the device notifies the orchestrator, which then triggers a recovery procedure to retransmit the corrected update. Figure 4.9-(b) compares the recovery time of FINE-STACK and native full-image firmware updates after a failed update of the UDP-Recv VF and the full firmware image, respectively, on the same node. The recovery time consists of two components: the time spent in the failed attempt and the time required to retry and retransmit the update.

For the retry component, the native implementation always incurs a longer delay because it must repeat the entire update process, including transmission of the full firmware image, whereas FINE-STACK retransmits only the affected VF.

For the failed-attempt component, when the same update sequence is applied, FINE-STACK and native full-image updates exhibit similar attempt times if the failure occurs before the download phase (P9), since the binary has not yet been transmitted. In contrast, when the failure occurs during or after P9, which includes downloading the updated file, the failed attempt takes longer and the total recovery cost increases, particularly for full-image updates because of the larger file size and the corresponding longer download time.

Overall, the recovery latency of FINE-STACK remains significantly lower than that of native full-image updates, especially for failures occurring during the long download phase, owing to the substantial reduction in update size. This result confirms the resilience and efficiency of FINE-STACK under adverse update conditions.

4.3.6 Memory Footprint

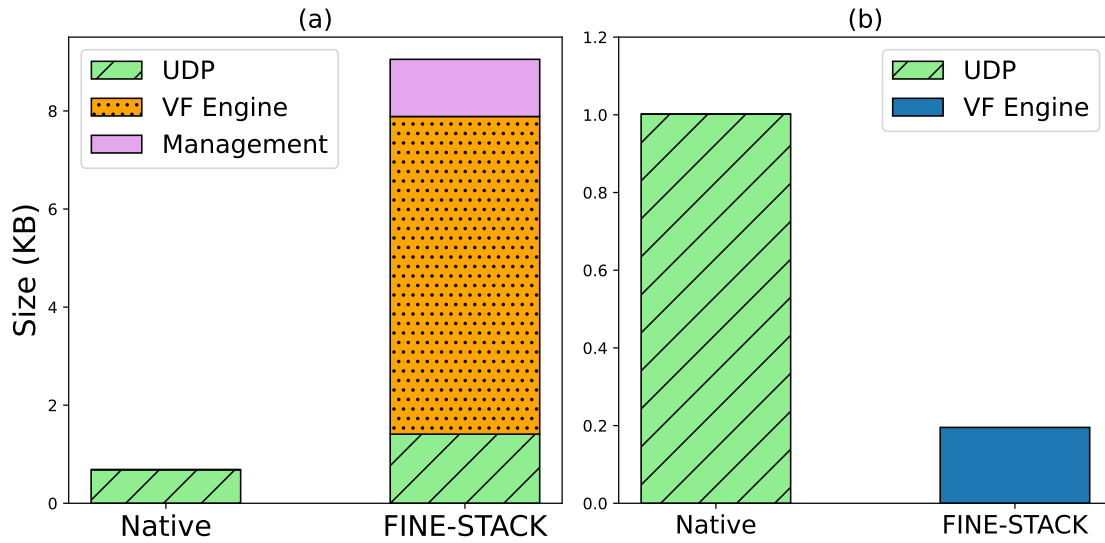


Figure 4.10: Memory footprint: (a) Flash memory, (b) RAM memory

Enabling UDP using FINE-STACK in the RIOT operating system requires adding specific modules. Figure 4.10 compares the flash and RAM memory requirements of FINE-STACK with those of the native RIOT implementation for the UDP protocol.

Regarding flash memory, FINE-STACK introduces an additional footprint composed of two parts: (i) Architecture-related: includes the VF engine that enables virtual functions in the operating system, as well as the management plane components that interface between the orchestrator and the LLWN devices. These elements are required only once, regardless of the number of implemented protocols. (ii) Protocol-related: contains the UDP functionalities. In FINE-STACK, the UDP code size is nearly twice that of

the native implementation due to the decomposition of UDP functions into two VFs, which introduces some code duplication, additional dependencies, and reduced compiler optimization. However, the overall increase in flash memory remains small and acceptable, totaling 8.57 kB, which corresponds to 7.82% of the native RIOT firmware’s total size.

Regarding RAM memory, the native RIOT implementation allocates a dedicated thread for each protocol, meaning that the UDP protocol alone requires a 1 kB RAM stack. In contrast, FINE-STACK invokes the UDP-Send or UDP-Recv VFs directly from the triggering thread, eliminating the need for an additional dedicated thread. Although it introduces an additional 200 bytes of RAM for the VF engine, FINE-STACK reduces the overall RAM usage by 800 bytes.

Download Slot

The download slot refers to the memory space required to temporarily buffer the incoming data chunks during an update. This requirement represents a critical limitation for LLWNs when using traditional full-image updates, since they demand available flash memory almost equal to the size of the running firmware. In practice, such free memory is rarely available on constrained devices, often making updates infeasible. FINE-STACK overcomes this limitation by requiring only a RAM slot equal to the size of the largest VF. In our experiment, updating UDP requires 109.6 kB of flash memory in the native implementation, whereas FINE-STACK needs only 3 kB of RAM, overcoming one of the major limitations of constrained devices. This drastic reduction not only eliminates a key barrier for updating constrained devices but also improves the practicality and reliability of in-field updates.

Overall, the memory extensions introduced by FINE-STACK remain negligible, while the elimination of excessive slot size requirements clearly demonstrates the suitability of our architecture for highly resource-constrained LLWNs.

4.3.7 Power Consumption

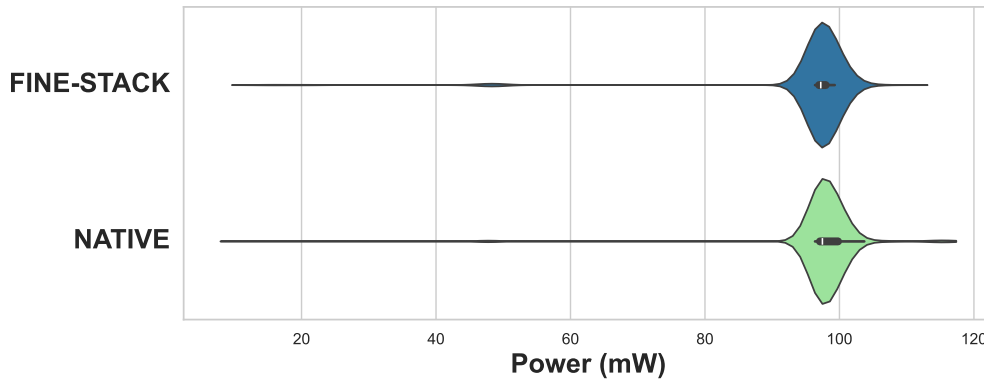


Figure 4.11: Power Consumption

Power is one of the most constrained resources in LLWNs, making it crucial to ensure that FINE-STACK does not significantly increase power consumption. To compare the power consumption of FINE-STACK-based UDP with the native UDP implementation in RIOT, 1,000 packets were transmitted locally using the loopback interface on one node. This setup allows us to measure the combined transmission and reception power consumption on the same node while excluding the contribution of the radio transceiver.

Measurements were performed using the INA226 hardware monitor available on the SLICES-FR testbed. Power samples were collected every $588 \mu\text{s}$, with an averaging count of 512. Figure 4.11 presents the distribution of the collected power consumption samples. Results show that the power consumption of FINE-STACK is comparable to that of the native implementation, confirming that it introduces no significant overhead.

In general, FINE-STACK exhibits power consumption comparable to that of the native implementation, reflecting the lightweight execution model of VFs. These results indicate that FINE-STACK preserves the low-energy operation required in LLWNs while enabling modularity and programmability.

4.3.8 Execution Time

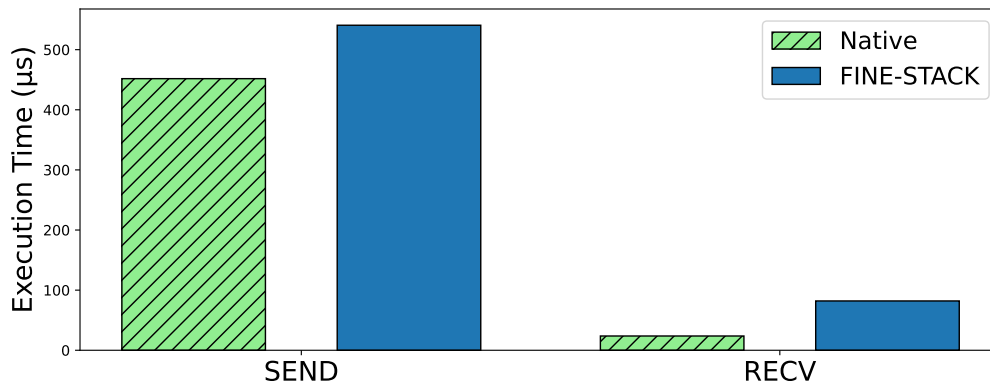


Figure 4.12: Execution Time

This section evaluates the processing time, a key factor in LLWNs when stringent timing constraints may apply, such as in TDMA-based MAC protocols. To this end, this section quantifies the in-device processing time required to send and receive a packet at the UDP layer—referred to as the *execution time*—and compares it with that of the native RIOT implementation. A total of 1,000 messages were sent between two nodes, from node A to node B, measuring the send time on node A and the receive time on node B.

Figure 4.12 shows the average send and receive times over the 1,000 transmissions. First, the send time is longer than the receive time in both implementations due to implementation details, as the send process includes a *while* loop. Second, FINE-STACK introduces a small execution time overhead in both sending and receiving due to virtualization with VFs, amounting to $88.86 \mu\text{s}$ for sending and $58.29 \mu\text{s}$ for receiving. This additional delay

mainly results from the interactions between the UDP VFs (UDP-Send and UDP-Recv) and the operating system through APIs.

Nevertheless, the observed overhead—on the order of tens of microseconds—is negligible compared with the typical 10 ms time-slot duration of TDMA-based MAC protocols, ensuring that it does not compromise timing requirements.

4.4 Conclusion

This chapter introduced FINE-STACK, a network architecture that marks a shift from hardware-centric to software-driven networking and paves the way toward fully programmable wireless networks. FINE-STACK relies on a modular network-stack design based on lightweight virtual functions that can be composed into protocols, updated over the air, and managed securely by a central orchestrator.

FINE-STACK was validated through a proof-of-concept implementation based on the standard unreliable UDP protocol, which was then dynamically extended with reliability support. UDP was selected because of its simplicity, as the protocol can be represented using only two virtual functions, namely UDP-Send and UDP-Recv, making it well suited for an initial demonstration of FINE-STACK’s capabilities. To introduce reliability into UDP, FINE-STACK significantly reduced update size and update duration compared with full firmware replacement. It also preserved seamless, reboot-free operation, thereby avoiding post-update reconvergence and service interruption. In addition, the update overhead of FINE-STACK remained acceptable while still providing secure updates and resilient recovery. Overall, resource usage in FINE-STACK remains practical: the memory overhead is acceptable, power consumption is comparable to the baseline, and processing overhead is negligible, confirming the suitability of FINE-STACK for constrained LLWNs.

The next chapter extends the validation to a more complex protocol, namely the RPL routing protocol. It also extends the flexibility of the architecture to support dynamic control plane deployment and, in particular, hybrid control plane operation that combines local autonomy with global management to enable state-aware updates of the network stack. This operation thereby avoids inconsistencies resulting from state loss that may otherwise arise during modular runtime updates.

Hybrid Control Plane Deployments and State-Aware Runtime Updates for LLWNs

Chapter contents

5.1	Introduction	72
5.2	FINE-STACK with Hybrid Control Plane Deployment	73
5.2.1	Network Stack for Different Control Plane Deployments	73
5.2.2	Architectural Support for Hybrid Deployment	75
5.3	Evaluation	76
5.3.1	Update Size and Time	77
5.3.2	Control Plane Deployments Performance Comparison	80
5.4	Conclusion	82

5.1 Introduction

The previous chapter introduced FINE-STACK as a modular network-stack architecture and showed that selective runtime updates can reduce update size and update time compared with full-image replacement. In addition, it eliminated the need to reboot after updates, thereby avoiding the loss of previously established state and the subsequent reconvergence wave. However, eliminating the reboot does not, by itself, resolve all runtime-update challenges. Some network-stack updates, particularly control plane updates, also modify protocol state and may therefore trigger reconvergence even when no reboot occurs.

This bootstrapping overhead increases with network size, leading to service interruptions, higher energy consumption, and reduced network availability. Reconvergence delays can

be significantly mitigated by explicitly understanding *what* is being updated and *how* to update it safely.

Centralized control plane deployments can leverage their global network view to store state and push it back to nodes after an update, thereby avoiding the reconvergence process. However, fully centralized deployments, such as those based on SDN, create a strong dependency on backhaul reliability. A backhaul link, which connects the nodes to the central control plane, must reliably carry telemetry and control information across the network. Congestion or disconnections on this link can degrade telemetry collection and delay policy dissemination, leaving nodes with outdated configurations or inconsistent states. Establishing a reliable backhaul also requires dedicating resources, which reduces the overall network capacity [18, 90].

The problem addressed in this chapter is therefore to design a control plane deployment and update mechanism that preserves the benefits of distributed control, exploits the global knowledge of centralized control when beneficial, and applies runtime updates capable of restoring or transforming protocol state after control plane modifications. Building on the foundation established in the previous chapter, the present chapter extends the FINE-STACK architecture by supporting enhanced versions of centralized and distributed deployments and introducing state-aware updates using hybrid control plane deployment. The validation compares the performance of the proposed deployments and evaluates the hybrid model against the distributed and centralized alternatives in terms of global optimization and local autonomy.

5.2 FINE-STACK with Hybrid Control Plane Deployment

This section discusses how FINE-STACK supports different control plane deployments, with a focus on routing, using RPL and SDN as representative distributed and centralized control plane deployments, respectively. It then introduces a state-preserving RPL update approach as a hybrid control plane deployment. Although this chapter focuses on existing routing-oriented control plane deployments, the proposed architecture is not limited to these cases and can be extended to other protocols, layers, and future hybrid models.

5.2.1 Network Stack for Different Control Plane Deployments

A given networking task in FINE-STACK can be realized using a single VF or a combination of multiple VFs grouped as a submodule. These VFs can be used to implement control plane logic/code locally on devices in distributed control plane deployments or to carry rules that define forwarding actions pushed remotely in centralized control/management plane deployments, as illustrated in Figure 5.1.

In distributed control plane deployments, the RPL routing protocol can be implemented using VFs at the network layer, as illustrated in Figure 5.1-(a). A VF may act as a parser

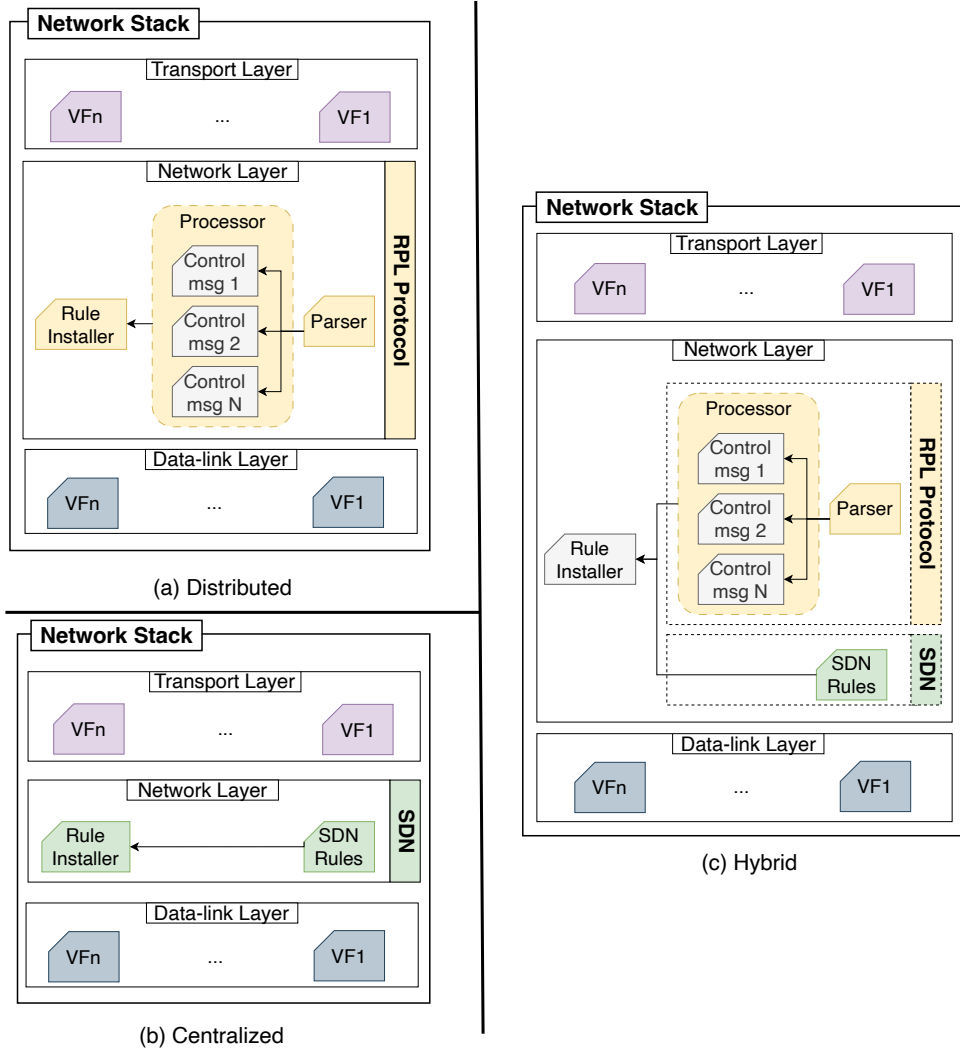


Figure 5.1: VF composition in the FINE-STACK network stack to realize different control plane deployments: (a) Distributed, (b) Centralized, and (c) Hybrid.

that processes incoming RPL packets and forwards them to a processing submodule. This submodule includes multiple VFs, each responsible for handling a specific type of routing control message in order to construct the routing state. The resulting output is then passed to a VF acting as a rule installer, which interacts with the forwarding table and installs the corresponding rules. The same design principle can be applied across other layers of the network stack.

In this deployment model, unlike traditional solutions that require full-image firmware replacement for reconfiguration, FINE-STACK enables targeted updates that modify only the relevant VFs. This results in significantly faster, more efficient, and more reliable updates.

In contrast, centralized control plane deployments use VFs to transport and install

rules pushed by the central controller into the data plane of the devices. For example, an SDN-like deployment can be implemented by executing VFs received from the controller that carry new rules, which are then installed through a rule installer interacting with the forwarding table, as shown in Figure 5.1-(b).

Moreover, FINE-STACK can disseminate rules spanning multiple layers or functional domains, such as the routing and MAC layers, thereby providing a unified and multipurpose centralized control plane solution. In addition, new rules can be defined and deployed through VFs without requiring a full software update on the devices, which significantly improves flexibility and reduces downtime.

These two approaches can also be combined, as illustrated in Figure 5.1-(c), to form a hybrid deployment in which a local control plane is implemented using VFs, for example to realize the RPL protocol, while additional VFs carry rules pushed by a global centralized control plane. A rule installer is then used to apply rules originating from both the local and global control planes.

5.2.2 Architectural Support for Hybrid Deployment

FINE-STACK provides the architectural support required to enable the proposed hybrid deployment by also hosting the global control plane at the orchestrator. In this model, the orchestrator continues to manage and update the VFs that implement the logic of the local control plane, as in the distributed model, while also injecting global decisions, as in the centralized model.

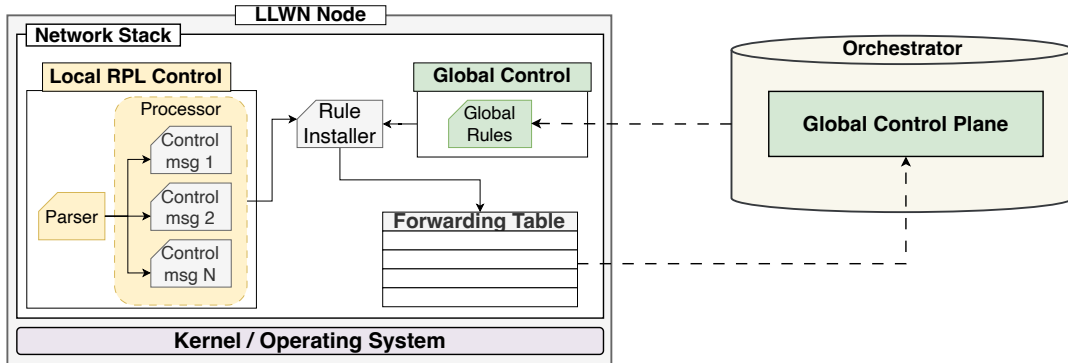


Figure 5.2: Proposed hybrid deployment in FINE-STACK.

For example, as illustrated in Figure 5.2, LLWN devices can execute RPL as a local distributed control plane, exchanging control messages to build and maintain the topology. At the same time, the orchestrator maintains a global view of the network and stores the state of each node, such as neighbor relationships and forwarding-table contents. During updates, for instance when switching RPL modes, the new mode would normally require rebuilding its state through the exchange of control messages, which results in a reconvergence phase. Instead, the orchestrator can push the stored state back to the

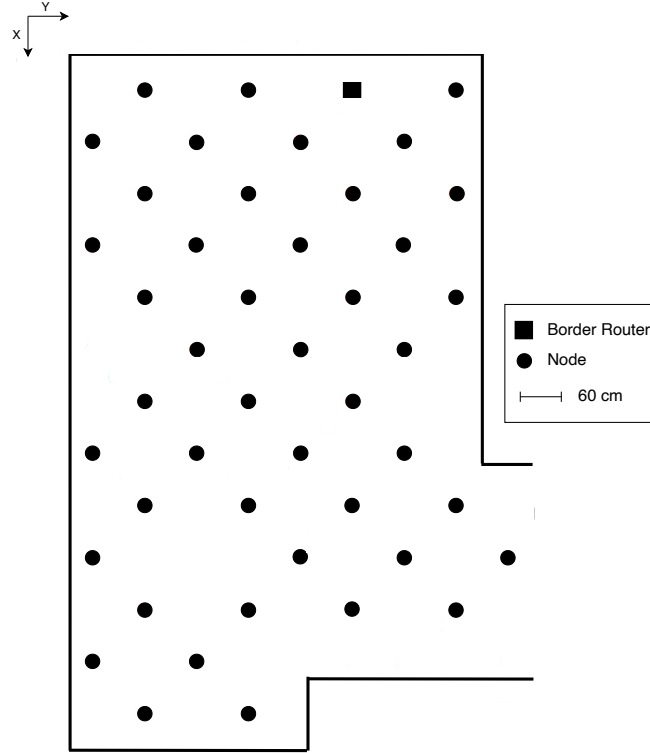


Figure 5.3: Topology on SLICES-FR. The network includes 46 M3 nodes; one node serves as the Border Router (BR), providing connectivity to the orchestrator and acting as the RPL root in the distributed deployment.

nodes in order to avoid the lengthy reconvergence process that would otherwise follow such an update. This approach reduces overhead, limits congestion, and prevents service interruption during topology reformation. To the best of our knowledge, this realization of a hybrid control plane has not yet been demonstrated in LLWN environments, and the VF-based design of FINE-STACK provides the modularity required to make it practical.

Overall, the use of VFs provides substantial flexibility, as they can encapsulate executable logic as well as carry rules or state. This versatility supports distributed, centralized, and hybrid control plane deployments and forms the foundation of a highly programmable network stack.

5.3 Evaluation

In this section, we implement FINE-STACK on the RIOT operating system [68] and evaluate its performance on a real-world testbed under three control plane deployment models.

In the distributed deployment, RPL is decomposed into 11 VFs to handle distinct

control messages and initiate the network. Runtime switching between storing and non-storing modes is done to adapt routing behavior. Because non-storing mode is not natively available in RIOT, we implemented and contributed this feature ourselves¹. In centralized deployment, the orchestrator acts as a central controller, installing topology rules on nodes in an SDN-like manner via VFs. The hybrid deployment represents a specific realization in which local RPL control is complemented by centrally injected state after updates. This hybrid case demonstrates how FINE-STACK can combine local protocol autonomy with global state restoration. However, general coordination between local and global control decisions remains outside the scope of this work.

To implement the VFs, we use FCs [88]. In addition, CoAP [44] and SUIIT [123] are used to realize the management plane, following the same evaluation setting adopted in the previous chapter. All three deployments are evaluated using a topology consisting of 46 M3 nodes as shown in Figure 5.3. Experiments run on the SLICES-FR real testbed, Lille site and adopt the stack of IPv6, 6LoWPAN, and GoMacH.

A direct quantitative comparison with prior work is not feasible due to differences in supported update granularity and underlying operating systems, which strongly influence timing and memory behavior. Therefore, the comparison baseline is the native RIOT stack, which mandates a full firmware reinstallation for any modification in the stack.

The evaluation focuses on two key aspects: (i) the update size and time of different deployment models compared with the native implementation, and (ii) network performance across deployments. To promote reproducibility, the implementation, scripts, and configurations are released as open source².

5.3.1 Update Size and Time

This section compares how VFs are used across the different deployments, each of which features a distinct form of VF update, resulting in different update sizes and corresponding delivery times.

To measure the update time and size for each control plane deployment, RPL switches its operating mode between storing and non-storing for the distributed deployment. For the centralized scenario, new forwarding rules are instead pushed to construct a topology that optimizes link quality over hop distance. The resulting topology consists of 22 nodes at one hop from the BR, 15 nodes at two hops, and 8 nodes at three hops, as determined by the pushed rules. Finally, in the hybrid deployment, RPL switches between storing and non-storing modes while preserving state information to avoid costly reconvergence.

Table 5.1: Update size among control plane deployments.

Architecture	Deployment	Update	Size (bytes)
FINE-STACK	Centralized	–	$N \times i, i \in [104, 208]$
	Distributed	To storing	3,758
		To non-storing	2,966
	Hybrid	To storing	$3,758 + N \times i, i \in [104, 208]$
		To non-storing	$2,966 + N \times i, i \in [104, 208]$
Native	Distributed	To storing	123,000
		To non-storing	125,728

5.3.1.1 Update Size

Table 5.1 compares the update sizes across the three deployments. In the centralized control plane deployment, the orchestrator distributes rules to network nodes to construct the topology. Each node receives a number of rules proportional to its position in the topology and the number of descendants it serves. In FINE-STACK, each rule occupies between 104 and 208 bytes, depending on whether optimizations are applied when grouping multiple rules into a single VF. The total update size for a node is therefore $N \times [104, 208]$ bytes, where N denotes the number of rules assigned to that node. In our evaluation, nodes were organized into three levels, resulting in different rule counts—and thus different update sizes—per node. N ranges from 1 for leaf nodes, which require only the default route to the parent, to 46 for the root, which defines a route to each node in addition to a default route.

In the distributed control-plane deployment, updates correspond to switching between the two operational modes of RPL, i.e., storing and non-storing modes. With FINE-STACK, only three of the 11 VFs implementing RPL require updating, namely those related to DAO message handling and root formation, whereas the native RIOT stack mandates a full firmware image replacement. The update size is identical for all nodes, as each node must update its protocol logic.

The hybrid deployment extends both approaches by retaining RPL as the local control plane while leveraging the orchestrator as a global control plane that pushes state information to nodes. This mechanism eliminates the long reconvergence delays typically observed after RPL updates. Consequently, in addition to the 3 VFs required in the distributed deployment, the hybrid mode includes a fourth VF (the State VF), which encodes per-node state (rules). The resulting update size therefore comprises a fixed component for the distributed protocol logic and a variable component determined by the number of rules in the State VF.

The native RIOT stack, by comparison, requires a full-image update even for minor modifications. This results in very large update sizes that are impractical for throughput-

¹https://github.com/ahmahmod/RIOT/tree/gnrc_rpl_non_storing

²https://github.com/ahmahmod/fine_stack

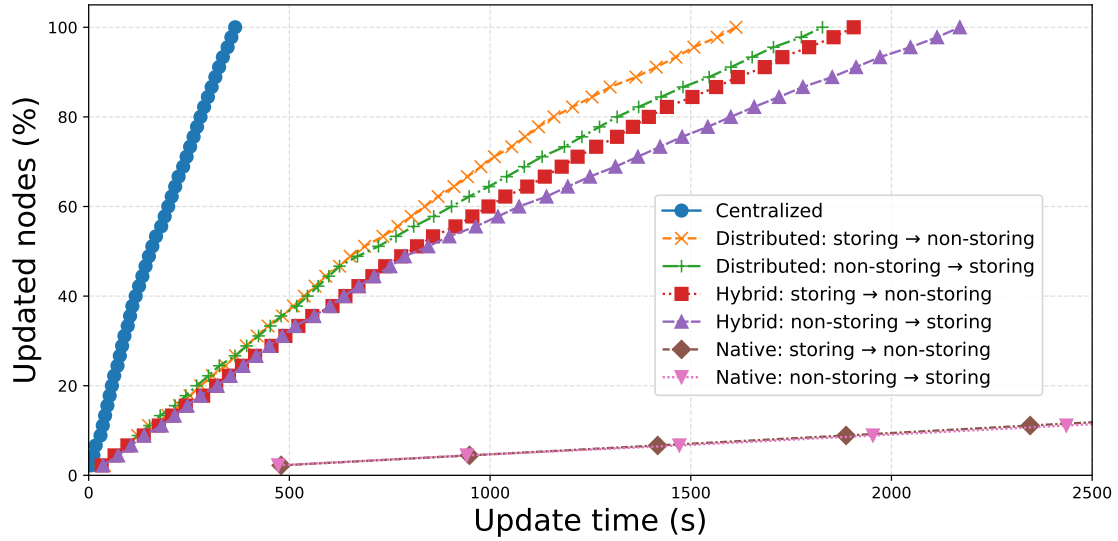


Figure 5.4: CDF of node update times across control plane deployments. Centralized mode is fastest with compact rule-only VFs, distributed mode is slower due to larger logic-bearing VFs, and hybrid mode is slowest because it installs logic-bearing VFs and a State VF.

limited networks. FINE-STACK achieves a substantial reduction in update size compared with full-image updates when switching RPL modes: 96.94% when switching from non-storing to storing mode, and 97.64% in the opposite direction. These slight differences arise from structural variations in the mode-specific VF implementations.

5.3.1.2 Update Time

The differing nature of updates across deployments leads to varying update sizes and consequently different update times. Figure 5.4 presents the CDF of node update times, showing the fraction of nodes that have completed the update as a function of time. The centralized deployment achieves the shortest update time, since its VFs contain only the rules pushed from the orchestrator to the nodes. However, VFs carrying rules must be transmitted frequently to reflect changes in network conditions. In contrast, the distributed deployment requires larger updates when switching between storing and non-storing modes, as the VFs contain updated control plane logic/code. This results in longer update times, although such updates occur far less frequently than in the centralized case. The hybrid deployment produces the longest update time because it includes the three VFs from the distributed deployment plus an additional State VF, increasing the overall update size. This overhead, however, is compensated by the elimination of reconvergence delays after updates, as discussed in the next section.

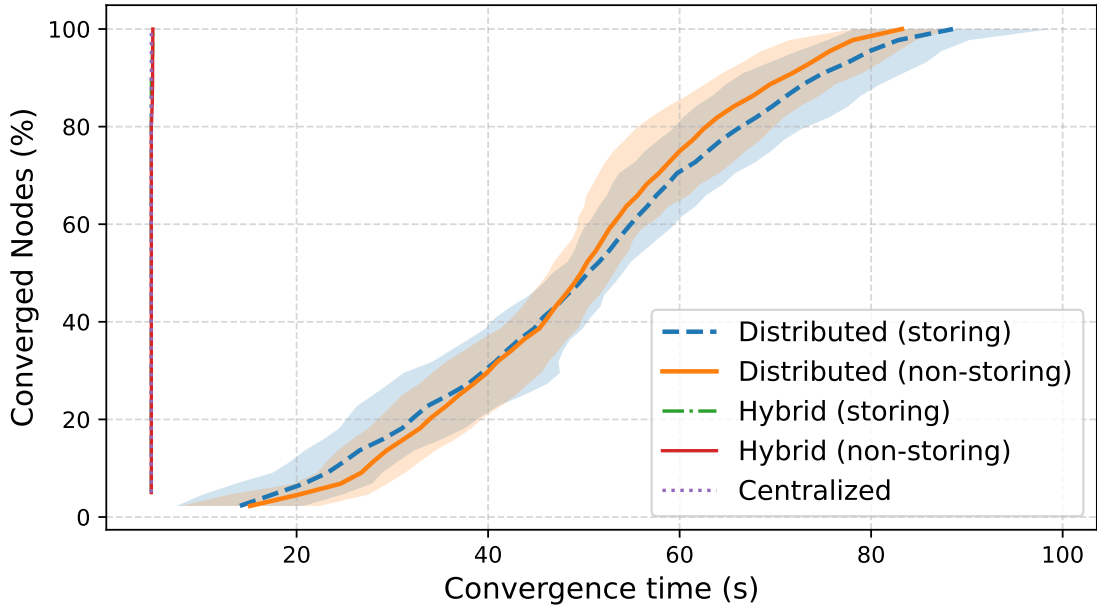


Figure 5.5: CDF of node convergence time after updates. Distributed mode shows long tails due to control plane re-exchange, while centralized and hybrid modes converge rapidly through rule installation or state restoration.

5.3.2 Control Plane Deployments Performance Comparison

This section compares the advantages and limitations of each control plane deployment model. Distributed deployments suffer from long convergence times, but they can recover autonomously under unreliable link conditions. Centralized deployments, by contrast, can achieve shorter convergence, but they may suffer from difficulties in delivering updates when links are unreliable. The section also illustrates how hybrid deployments can overcome the shortcomings of both fully centralized and fully distributed approaches.

5.3.2.1 Convergence Time

To evaluate convergence behavior after an update, the convergence time of each node is measured, defined as the interval until it becomes connected and reachable from the BR. Figure 5.5 shows the CDF of node convergence times after the update for all three deployments, based on 20 experiments. In the distributed deployment, after switching the RPL mode of operation, nodes operate autonomously and must re-exchange control messages to rebuild their forwarding tables according to the new mode. This process results in long convergence delays, as the extensive control-message exchange both slows convergence and burdens the throughput-limited medium. In the centralized deployment, the network is reconfigured to use a different parent-selection metric (link quality instead of hop count) by pushing new rules to the forwarding tables of all nodes, effectively rebuilding the topology from scratch, as occurs after RPL updates. Convergence is nearly

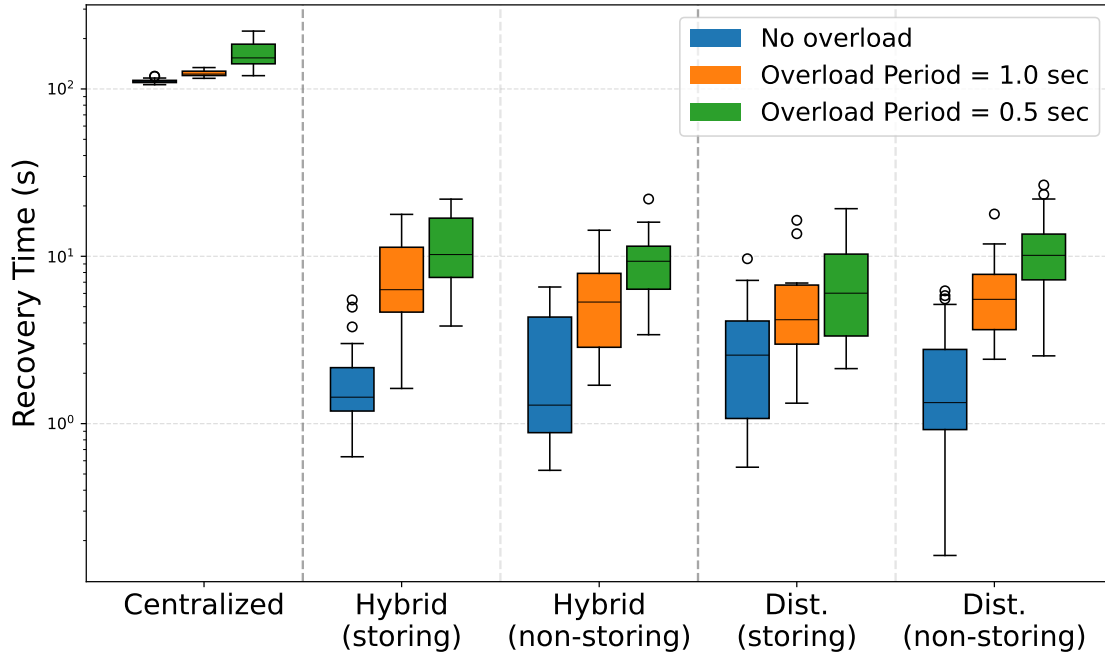


Figure 5.6: Maintenance recovery time across control plane deployments. Recovery increases with load, with centralized control degrading most due to congested multi-hop rule dissemination. Distributed and hybrid modes recover faster through local fallback.

instantaneous, since nodes simply execute the VFs containing the rules pushed by the orchestrator, without requiring any interaction with neighboring nodes. In the hybrid deployment, the distributed approach is extended by switching between storing and non-storing modes while additionally pushing state information to restore the forwarding-table contents. The pushed state attaches each node to the parent it used in the previous mode before switching. In storing mode, it also defines the routes to the node’s children. This design brings the near-instant convergence of centralized deployments to the distributed setting, effectively eliminating the lengthy reconvergence phase that typically follows an update.

5.3.2.2 Recovery Time

While distributed deployments perform poorly in convergence after updates, centralized deployments struggle to collect metrics to maintain the network view and deliver updates reliably under lossy conditions in a multi-hop network. To evaluate these contrasting behaviors, we measured the recovery time when some nodes leave the network. In our experiments, six nodes were removed from the topology, and we measured the time required for their children to reconverge. In the distributed deployment, this corresponds to the time needed for nodes to discover alternative parents; in centralized deployment, it reflects the time required for the orchestrator to recompute and disseminate new rules to select alternative parents. The six removed nodes were selected under identical conditions in all

experiments: each was a direct child of the BR, had at least two children, and each of those children had at least one descendant. To highlight the vulnerability of centralized deployments in lossy environments, we introduced network overload scenarios in which nodes periodically transmit 1-byte UDP packets (every 1 s and 0.5 s) to the BR, emulating common LLWN applications, such as healthcare applications [124]. Figure 5.6 compares the recovery time across the three deployments under varying overload conditions, using 20 experiments per scenario.

Recovery time increases with load in all deployments. However, the centralized deployment experiences the longest recovery delays, as rules must be recomputed first at the orchestrator and then delivered via VFs through lossy multi-hop paths to the affected nodes. Under high load, this leads to prolonged connectivity disruption, as nodes remain disconnected while waiting for delayed reconfiguration from the central control plane. In contrast, the distributed deployment recovers more rapidly, as affected nodes can locally exchange control messages with stable neighbors and select alternative parents without relying on remote rule delivery (no VF delivery). In our experiments, nodes were configured not to cache alternative parents; enabling parent caching would allow the distributed deployment to achieve near-instant recovery.

The hybrid deployment overcomes the limitations of centralized control in lossy environments by allowing nodes to fall back on their local control plane to exchange control messages and reattach to the topology, similarly to the distributed deployment. This mechanism ensures rapid reconnection and minimizes service interruption even with suboptimal parent selection, while still allowing the global control plane to subsequently optimize parent selection based on its global view when possible.

Overall, the results show that centralized and distributed deployments offer distinct advantages depending on network conditions. Centralized deployments converge immediately once updates are delivered, but they struggle with reliable update dissemination under lossy and multi-hop conditions. In contrast, distributed deployments require a reconvergence phase after updates; yet, they do not depend on rule delivery from a remote controller. The hybrid deployment consistently outperforms both approaches by combining the global optimization capabilities of centralized control with the resilience and autonomy of distributed control. A remaining challenge lies in managing potential conflicts between local and global decisions, which warrants further investigation.

5.4 Conclusion

In this chapter, FINE-STACK was extended to support programmable distributed and centralized control plane deployments and to introduce a hybrid deployment model for LLWNs. The proposed hybrid control plane model combines the benefits of centralized approaches, which provide globally optimized network decisions, with those of distributed approaches, which enable fast reactions to local changes.

Through extensive evaluation, FINE-STACK was shown to significantly reduce update overhead across the three deployment models, thereby shortening update time and enabling more frequent network reconfiguration without disrupting running applications. By preserving network state during updates, the hybrid mode avoids costly reconvergence, achieves near-instant convergence, and improves network availability. This also allows nodes to maintain stable operation even when the central controller becomes temporarily unreachable.

The centralized deployment evaluated in this chapter assumes that the data plane and the control/management plane share the same communication channel. However, some architectures provide a separate channel or dedicated resources for central control in order to improve its availability and reduce interference with data traffic. Exploring this direction would be a valuable extension of the proposed hybrid deployment.

Although FINE-STACK significantly reduces update time, it currently relies on unicast dissemination, which leads to redundant transmissions in scenarios where multiple nodes receive the same update, such as in the distributed deployments. A promising direction for future work is therefore the integration of multicast dissemination in order to further accelerate and optimize update delivery.

Finally, the updates considered in this chapter remain reactive. The next chapter extends this work by introducing intelligent orchestration mechanisms at the orchestrator's global control plane to automate network updates and enable proactive, self-adaptive reconfiguration of the network stack.

AI-Enabled Traffic-Aware Closed-Loop Orchestration for LLWNs

Chapter contents

6.1	Introduction	85
6.2	Methodology and Framework	87
6.2.1	Dataset Collection	88
6.2.2	Dataset Construction	90
6.2.3	Offline Training and Model Design	93
6.2.4	Model Families and Performance Comparison	95
6.2.5	Online Closed-Loop Management	97
6.3	Evaluation	98
6.3.1	Prediction Model Performance	99
6.3.2	Forecasting-based Recommendation Performance	101
6.3.3	Adaptability to Data Profiles	103
6.3.4	Scalability over a Large Heterogeneous Topology	105
6.4	Conclusion	106

6.1 Introduction

The capacity-energy trade-off of LLWNs is difficult to manage because LLWN conditions are not stationary. In addition, application traffic may alternate between sporadic and bursty event-driven phases [125]. A configuration that is energy-efficient during sporadic and low-rate periodic traffic may therefore become unreliable during bursts, while a configuration

designed for bursty traffic may waste energy during normal operation. Consequently, the most suitable MAC configuration depends not only on the node's position in the routing topology and the forwarding burden imposed by multi-hop communication, but also on the traffic regime [126].

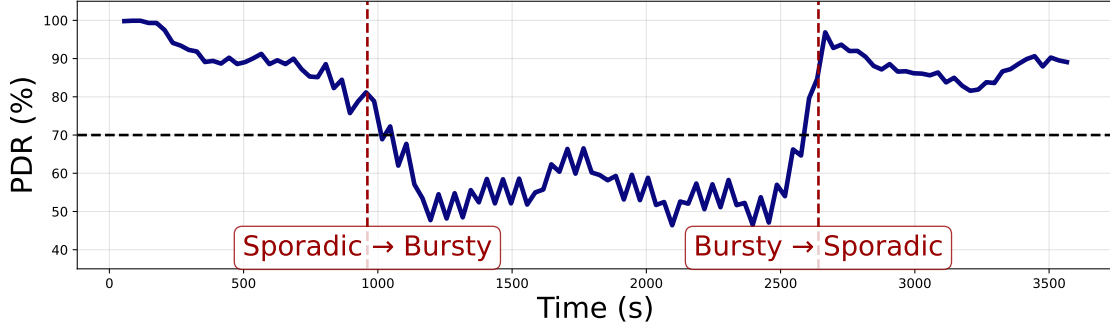


Figure 6.1: PDR under sporadic and bursty traffic regimes

Figure 6.1 illustrates this issue by comparing the packet delivery ratio (PDR) under alternating sporadic and bursty traffic regimes using contention-based duty cycling only, against a target PDR of 70%. The results, obtained from an experiment conducted on the SLICES-FR testbed, show that the contention-based configuration satisfies the target during sporadic traffic. However, when the bursty phase begins, the node fails to maintain the target PDR, highlighting the need for more adaptive MAC behavior. These observations motivate runtime adaptation to balance energy consumption and QoS requirements according to current network conditions.

Chapters 4 and 5 introduced FINE-STACK, our modular network-stack architecture that enables fast and fine-grained runtime reconfiguration of communication protocols. However, the availability of a reconfiguration mechanism does not by itself determine when reconfiguration should be triggered, which nodes should be affected, or which configuration should be selected.

This chapter addresses this missing decision-making layer in the orchestrator. It focuses on traffic-aware closed-loop MAC-layer control, in which runtime monitoring, short-horizon prediction, recommendation, and online reconfiguration are combined to adapt MAC behavior according to current and near-future network conditions. Such automation is particularly important in LLWNs because the impact of a MAC configuration depends on nonlinear interactions among traffic load, topology position, forwarding burden, contention, retransmissions, and duty-cycle behavior. Machine-learning-based models can help capture these interactions and support proactive decisions that would be difficult to derive using static rules alone. The role of the orchestrator is therefore to use ML-based models to maintain a target PDR while reducing the duty cycle whenever possible, in order to manage the capacity-energy trade-off.

6.2 Methodology and Framework

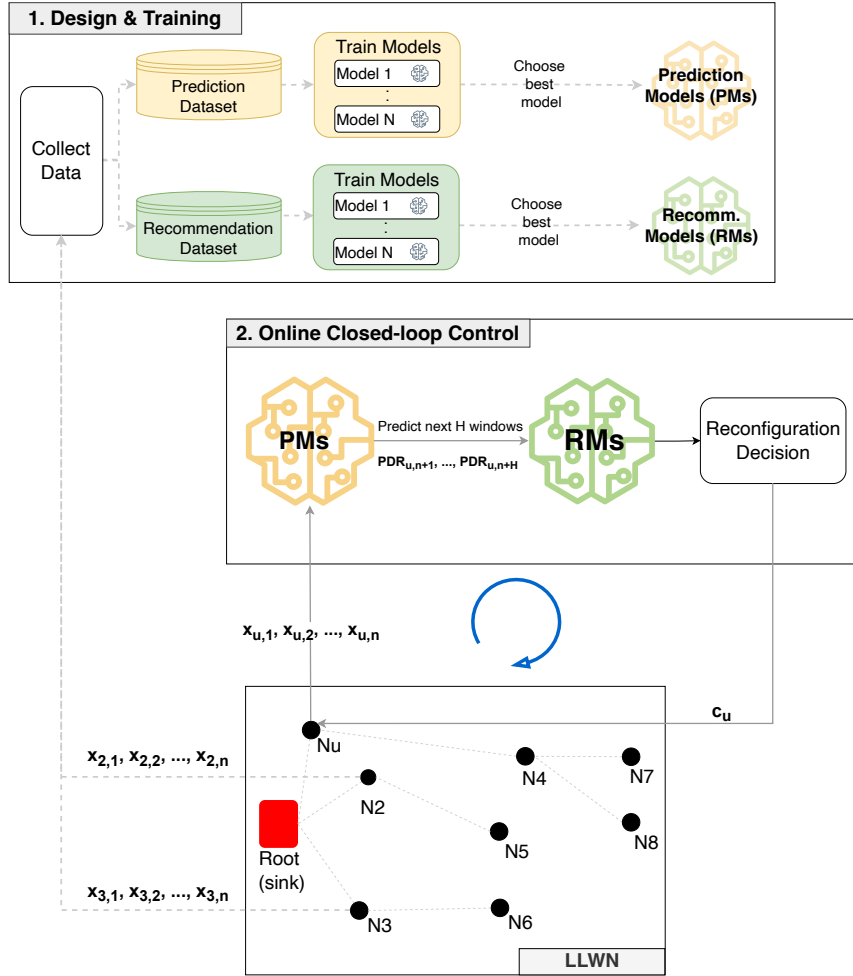


Figure 6.2: Closed-loop control framework. It consists of two phases: Offline training and Online closed-loop control.

The proposed framework exploits per-node telemetry to capture communication quality, topology-dependent forwarding conditions, and traffic burstiness. It then uses learning-based models to anticipate near-future PDR and duty-cycle evolution and to trigger suitable reconfiguration actions. These actions include node-specific duty-cycle adaptation under low-rate traffic and TDMA enabling under bursty traffic. The framework is built on GoMacH [111], introduced in Section 3.3.1. Unlike approaches that treat predictive modeling, adaptive MAC protocols, and closed-loop control as separate problems, the proposed framework combines them into a single architecture deployed and evaluated through realistic testbed experiments.

Our framework is organized into two main stages: offline design and training, and online runtime management, as illustrated in Figure 6.2. The first stage aims to construct the knowledge required for future performance forecasting and runtime protocol reconfiguration.

To this end, the orchestrator collects realistic and representative measurements of network state, context, and performance across a wide range of scenarios involving different MAC configurations and traffic profiles. These measurements are then processed to build structured datasets for both forecasting and recommendation. Based on these datasets, several machine-learning models are trained and compared to select the most suitable models for forecasting and runtime recommendation.

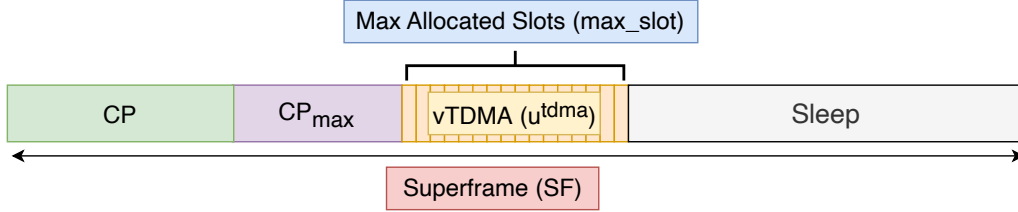


Figure 6.3: GoMacH configuration parameters.

In the second stage, the selected models are deployed online to continuously monitor real-time network measurements and track the current network state. This runtime state is then provided to the forecasting model, which predicts potential near-future performance degradations. Based on these forecasts, together with the current node conditions, the recommendation model determines whether GoMacH requires contention-parameter reconfiguration or TDMA-oriented protection. The GoMacH configuration parameters are illustrated in Figure 6.3. In the contention-based reconfiguration case, the framework adjusts the initial contention period CP , the maximum contention-period extension CP_{max} , and the superframe duration SF . In the TDMA-oriented reconfiguration case, it also determines whether TDMA should be enabled u^{tdma} and selects the corresponding slot-allocation setting max_slot in order to preserve the target performance while limiting duty-cycle cost. The individual components of the framework are described in greater detail in the following sections.

6.2.1 Dataset Collection

We consider an LLWN organized in a tree topology rooted at a sink node. The network is composed of a set of nodes

$$\mathcal{N} = \{1, 2, \dots, N\},$$

organized around the root/sink. Each node periodically reports MAC and routing indicators. At an instant t , the per-node GoMacH timing configuration of node $i \in \mathcal{N}$ is represented by

$$\mathbf{c}_{i,t} = (CP_{i,t}, CP_{i,t}^{\max}, SF_{i,t}), \quad (6.1)$$

where $CP_{i,t}$, $CP_{i,t}^{\max}$, and $SF_{i,t}$ represent the nominal contention period, the extended contention period, and the superframe duration illustrated in Figure 6.3, together with the network-wide TDMA state $u_t^{\text{tdma}} \in \{0, 1\}$ and the common slot budget max_slot , which

represents the maximum number of slots that a receiver node can schedule in TDMA.

Table 6.1: Dataset collection parameters.

Parameter	Values
Testbed (site)	SLICES-FR (Lille)
Number of nodes	25
Measurement window (Δ)	15 s
Operating system	RIOT
Network stack	UDP/RPL/6LoWPAN/GoMacH
Contention period (CP)	[10, 15, 20] ms
Max. contention period ($CP_{\max}$)	[3, 7, 11] $\times CP$
Superframe (SF)	[10, 15, 20, 25, 30] $\times CP$
Periodic data rate	2 packets/s
Max. allocated slots (max_slot)	[10, 30, 50, 70, 90, 110, 130, 150]
Burst packets ($burst_pkts$)	[5, 10, 20, 30, 40]

Table 6.1 summarizes the dataset collection parameters. In total, the dataset consists of 85 experiments conducted on the SLICES-FR testbed and obtained through two complementary parameter sweeps.

The first sweep explores the general GoMacH timing space through a full-factorial design. The sweep considers 3 values for the contention period, 3 values for the maximum contention period, and 5 values for the superframe duration, yielding $3 \times 3 \times 5 = 45$ timing configurations, each run for a 1-hour experiment. The second sweep focuses on burst-sensitive operation and TDMA-oriented behavior. In this case, the baseline timing configuration is fixed, while the explored parameters are the TDMA allocation capacity and the burst size, with 8 values for the former and 5 values for the latter. This produces $8 \times 5 = 40$ additional configurations, each run for a 30-minute experiment. This separation is scientifically important: the first dataset captures the impact of general GoMacH timing adaptation, whereas the second specifically isolates the interaction between bursty traffic and TDMA-based slot allocation.

Each experiment involves 25 M3 nodes deployed on the Lille site. Each node runs the RIOT operating system [68], on top of which we implemented software modules to periodically collect measurements and export them to CSV files. Each node runs a protocol stack composed of UDP, IPv6-RPL, 6LoWPAN, and GoMacH.

In every experiment, one node acts as the root, that is, the data sink. The remaining nodes establish routes toward the root through RPL control exchanges. As a result, the routing topology may vary across experiments due to environmental conditions and testbed dynamics, which increases the representativeness of the dataset with respect to real deployments. After an initial 3-minute warm-up period allowing routing convergence, each node generates a 1-byte data packet in bursts or periodically, depending on the case. During each run, every node exports MAC- and routing-level counters once per measurement window t of duration Δ seconds. These raw measurements are subsequently collected by the

orchestrator, which transforms them into supervised datasets for training and evaluating the prediction and recommendation models, as detailed in the next subsection.

6.2.2 Dataset Construction

For each node and each experiment, a per-node trace is generated, from which the pipeline derives window-level features consistent with the operation of the online orchestrator. Let ΔS_t , ΔF_t , ΔB_t , and ΔQ_t denote, respectively, the per-window increments of successful transmissions, failed transmissions, busy-channel events, and queue drops. Let ΔW_t denote the wake duration and ΔT_t the elapsed time over the same observation window. The delivery denominator D_t is defined as

$$D_t = \Delta S_t + \Delta F_t + \Delta B_t + \Delta Q_t. \quad (6.2)$$

From these quantities, the main MAC-level performance indicators are computed as

$$\begin{aligned} \text{PDR}_t &= \frac{\Delta S_t}{D_t + \varepsilon}, \\ \text{Duty}_t &= \frac{\Delta W_t}{\Delta T_t + \varepsilon}, \\ \text{BusyRatio}_t &= \frac{\Delta B_t}{D_t + \varepsilon}, \\ \text{FailRatio}_t &= \frac{\Delta F_t}{D_t + \varepsilon}, \\ \text{QueueLoss}_t &= \frac{\Delta Q_t}{D_t + \varepsilon}. \end{aligned} \quad (6.3)$$

where $\varepsilon > 0$ is a small constant introduced to prevent division-by-zero in degenerate windows.

To enrich these instantaneous MAC observations, the feature construction also incorporates topology-related information derived from the RPL routing protocol [41], together with traffic-shape descriptors. For node i at window t , the resulting input vector is defined as

$$\mathbf{z}_{i,t} = \left(\text{PDR}_{i,t}, \text{Duty}_{i,t}, \text{BusyRatio}_{i,t}, \text{FailRatio}_{i,t}, \text{QueueLoss}_{i,t}, h_{i,t}, d_{i,t}, rssi_{i,t}, b_{i,t} \right), \quad (6.4)$$

where $h_{i,t}$ is the hop distance from node i to the root, $d_{i,t}$ is the number of descendants of node i in the RPL tree, $rssi_{i,t}$ is the received signal strength indicator, and $b_{i,t}$ is the burstiness indicator associated with the traffic generated by that node.

The inclusion of $h_{i,t}$ and $d_{i,t}$ is important because node performance in a multi-hop LLWN depends not only on local contention, but also on topological position. Nodes located close to the root or serving many descendants generally relay more traffic and therefore experience different reliability and duty cycle conditions than leaf nodes. In addition,

wireless channel conditions are partially reflected by the signal-quality indicator $rss_{i,t}$. Likewise, the burstiness term captures the temporal structure of the offered traffic, which is especially relevant for identifying operating conditions under which a TDMA-oriented mode becomes beneficial.

The burstiness indicator is computed within each observation window from the inter-generation intervals of consecutive packets. Let iat_k denote the k -th inter-generation interval and let n be the number of interval samples in the considered window. The burstiness is defined as

$$b_{i,t} = \frac{n \sum_{k=1}^n iat_k^2 - (\sum_{k=1}^n iat_k)^2}{n \sum_{k=1}^n iat_k^2 + \varepsilon}, \quad (6.5)$$

which yields a normalized value in the range $[0, 1]$. Values close to 0 correspond to nearly periodic traffic, whereas values close to 1 indicate highly clustered transmissions separated by long silent periods.

For example, if we have two traffic profiles, one generates continuous data at the rate of one packet every 500 ms, while the other one generates bursty data at the rate of 20 packets every 10 s with an interval of 3 ms. We can illustrate the metric over a 20-second observation window.

For continuous traffic with one packet generated every 500 ms, all inter-generation intervals are identical. Over 20 s, the number of intervals is

$$n = \frac{20000}{500} = 40.$$

Hence,

$$\sum_{k=1}^n iat_k = 40 \times 500 = 20000,$$

and

$$\sum_{k=1}^n iat_k^2 = 40 \times 500^2 = 40 \times 250000 = 10000000.$$

Therefore,

$$b_{i,t} = \frac{40 \times 10000000 - 20000^2}{40 \times 10000000} = \frac{400000000 - 400000000}{400000000} = 0.$$

For bursty traffic with 20 packets every 10 s and an intra-burst spacing of 3 ms, each burst contains 19 short intervals of 3 ms, yielding a burst duration of

$$19 \times 3 = 57 \text{ ms.}$$

Since bursts repeat every 10000 ms, the silence between two consecutive bursts is

$$10000 - 57 = 9943 \text{ ms.}$$

Over a 20-second window, the interval sequence therefore contains 38 short intervals of 3 ms and 2 long intervals of 9943 ms, which gives

$$n = 40.$$

Thus,

$$\sum_{k=1}^n iat_k = 38 \times 3 + 2 \times 9943 = 114 + 19886 = 20000,$$

and

$$\sum_{k=1}^n iat_k^2 = 38 \times 3^2 + 2 \times 9943^2 = 38 \times 9 + 2 \times 98863249 = 342 + 197726498 = 197726840.$$

The resulting burstiness is

$$b_{i,t} = \frac{40 \times 197726840 - 20000^2}{40 \times 197726840} = \frac{7909073600 - 400000000}{7909073600} = \frac{7509073600}{7909073600} \approx 0.949.$$

To complement raw window metrics, the dataset also includes temporally smoothed indicators computed through an exponentially weighted moving average (EWMA). For a generic metric x_t , the EWMA is defined as

$$\tilde{x}_t = \alpha x_t + (1 - \alpha)\tilde{x}_{t-1}, \quad (6.6)$$

where $\alpha \in (0, 1]$ controls the weight assigned to the most recent observation. This operation is applied to several core indicators, including PDR, duty cycle, transmission attempts, busy ratio, and queue-loss ratio, thereby complementing raw per-window measurements with a compact representation of recent history.

EWMA features provide several advantages for both learning and control. First, they attenuate short-lived fluctuations caused by transient contention, brief interference episodes, or isolated queue events, thereby reducing sensitivity to noise. Second, unlike a simple moving average, EWMA preserves recency by assigning larger weights to the most recent windows, which is desirable in non-stationary wireless environments where the operating point may evolve over time. Third, EWMA is lightweight to compute online, since only the previous smoothed value must be retained, making it well suited to constrained monitoring pipelines. As a result, these features provide a more stable yet still responsive representation of the recent network state.

Overall, the constructed windowed dataset captures not only instantaneous MAC behavior, but also the structural role of each node in the routing tree, the temporal variability of its traffic generation process, and the recent evolution of communication quality. These complementary dimensions are essential for accurate prediction and recommendation.

6.2.3 Offline Training and Model Design

6.2.3.1 Prediction Models

The prediction stage estimates short-term network performance from the current observed state. For each node i , the model learns a mapping from the current input vector $\mathbf{z}_{i,t}$ to the next-window communication outcome.

This formulation is well suited for an orchestrator operating over monitoring windows. Rather than directly estimating a long-term aggregate, the predictor captures the immediate effect of the current operating condition on the next observation window. The models are trained offline using the timing-sweep dataset, and two separate regressors are learned:

$$\begin{aligned}\widehat{\text{PDR}}_{i,t+1} &= f_{\text{pdr}}(\mathbf{z}_{i,t}), \\ \widehat{\text{Duty}}_{i,t+1} &= f_{\text{duty}}(\mathbf{z}_{i,t}).\end{aligned}\tag{6.7}$$

Two separate prediction bundles are trained offline: a non-bursty (sporadic and periodic traffic) bundle and a burst bundle. The orchestrator evaluates a runtime burstiness gate and selects which bundle to use for the current node state. This allows the prediction mechanism to adapt to different traffic regimes without forcing a single model to explain both non-bursty and bursty dynamics equally well.

At runtime, the online orchestrator can invoke the predictor in a proactive mode. For node i , the predictor is rolled over the next H windows to estimate the average short-term PDR:

$$\widehat{\text{PDR}}_{i,t}^{(H)} = \frac{1}{H} \sum_{h=1}^H \widehat{\text{PDR}}_{i,t+h}.\tag{6.8}$$

The predictor is thus not used as an end in itself, but as an anticipatory mechanism that alerts the orchestrator before performance degradation becomes clearly visible in the measured data.

6.2.3.2 Recommendation Models

Whereas the predictor estimates what is likely to happen next, the recommender estimates which configuration should be applied. The recommendation stage uses recent summarized observations together with candidate GoMacH settings to identify a control action that preserves reliability while limiting duty cycle cost. The framework includes two recommendation branches: a *Contention Period (CP)-oriented recommender* for duty-cycle timing adaptation and a *TDMA-oriented recommender* for burst-driven slot allocation.

CP-oriented recommendation. The CP-oriented recommender is trained from summaries computed over the last N windows. The summary vector includes statistics such as the mean, minimum, maximum, standard deviation, and last observed value over the

considered horizon.

Let $\mathbf{s}_{i,t}^{(N)}$ denote this last- N -window summary vector. The recommender then learns the mappings

$$\begin{aligned}\widehat{\text{PDR}}_i &= g_{\text{pdr}}(\mathbf{s}_{i,t}^{(N)}, c_i), \\ \widehat{\text{Duty}}_i &= g_{\text{duty}}(\mathbf{s}_{i,t}^{(N)}, c_i),\end{aligned}\tag{6.9}$$

where c_i denotes a candidate GoMacH timing configuration for node i .

At runtime, the orchestrator first identifies the set of affected nodes, defined as those whose rolled predicted PDR over the next H windows falls below the target:

$$\mathcal{A}_t = \left\{ i \in \mathcal{N} \mid \widehat{\text{PDR}}_{i,t}^{(H)} < \text{PDR}_{\text{target}} \right\}.$$

A candidate configuration c_i is considered feasible for node $i \in \mathcal{A}_t$ if

$$\widehat{\text{PDR}}_i(c_i) \geq \text{PDR}_{\text{target}}.\tag{6.10}$$

Among the feasible candidates, the selected configuration for node i is the one minimizing the predicted duty-cycle cost:

$$c_i^* = \arg \min_{c_i \in \mathcal{C}_{i,\text{feasible}}} \widehat{\text{Duty}}_i(c_i).\tag{6.11}$$

If no feasible candidate exists for an affected node, the orchestrator adopts a best-effort solution for that node, namely the candidate that maximizes predicted reliability improvement while keeping the duty-cycle increase as limited as possible.

TDMA-oriented recommendation. The TDMA-oriented recommender is activated only under bursty traffic conditions. Unlike the CP-oriented branch, this recommender selects a common slot-allocation budget for all participating nodes once the network switches to TDMA mode. It uses recent burstiness-aware observations and evaluates candidate values of the global slot-allocation parameter *max_slot*. In this case, the recommendation models estimate the expected run-level performance under each candidate allocation:

$$\begin{aligned}\widehat{\text{PDR}}_i &= h_{\text{pdr}}(\mathbf{s}_{i,t}^{(N)}, \text{max_slot}, b_{i,t}), \\ \widehat{\text{Duty}}_i &= h_{\text{duty}}(\mathbf{s}_{i,t}^{(N)}, \text{max_slot}, b_{i,t}).\end{aligned}\tag{6.12}$$

The selected TDMA slot-allocation budget is the candidate with the lowest predicted duty-cycle cost among those satisfying the reliability target for the relevant nodes. Otherwise, it resorts to a best-effort selection.

In summary, offline learning is the foundation of the proposed control framework, combining prediction and recommendation models trained from experimental data. The prediction models provide a short-term view of future network behavior by forecasting next-

window reliability and duty cycle, which supports proactive detection of degradation. The recommendation models extend this capability by selecting suitable GoMacH configurations, either through duty-cycle timing adaptation in general conditions or TDMA slot allocation under bursty traffic. Together, these two model families transform historical measurements into actionable intelligence, enabling informed and adaptive runtime control.

6.2.4 Model Families and Performance Comparison

The contribution of this work does not lie in the design of a new machine-learning model, but in the integration of prediction and recommendation within our framework. Accordingly, this section evaluates a set of established regression families and retains, for each task–target pair, the model that offers the best empirical performance. The supervised learning pipeline compares several regression families because the relationship between the observed network state and the resulting performance is nonlinear, highly interactive, and variable across nodes and experimental conditions. The candidate models were selected to cover complementary inductive biases, ranging from simple parametric formulations to more flexible nonlinear learners.

Linear models: *Ridge* and *Lasso*. Linear regularized models provide useful baselines because they are computationally efficient, stable, and easy to interpret. However, they are limited to approximately linear dependencies and therefore cannot fully capture the complex interactions among MAC timing parameters, traffic load, routing dynamics, and burstiness-related effects.

Kernel and instance-based models: *SVR* and *KNN*. These methods can represent nonlinear behavior without requiring an explicit global parametric form. Their performance may nevertheless degrade in heterogeneous feature spaces. In particular, Support Vector Regression (SVR) is sensitive to feature scaling and may become computationally expensive, whereas K-Nearest Neighbors (KNN) can suffer when neighboring samples are sparse or insufficiently representative of the current operating regime.

Neural model: *MLP*. A multilayer perceptron was also included to assess whether a generic neural nonlinear approximator could improve performance on the considered tabular features. Although MLPs can model complex relationships, like all neural networks, they generally require careful tuning and larger datasets to remain robust, and their advantages are less systematic on moderately sized structured telemetry data.

Tree-based ensembles and boosting: *Extra Trees*, *GBRT*, *HGBT*, and *XGBoost*. These models are particularly well suited to the present problem because the inputs are structured tabular features with strong interactions and threshold effects. Tree-based ensembles naturally capture nonlinearities, mixed feature influences, and regime changes

Table 6.2: Cross-validation MAE (%) of the prediction and recommendation models.

Model	Prediction				Recommendation			
	Non-bursty		Bursty		CP-oriented		TDMA-oriented	
	PDR	DC	PDR	DC	PDR	DC	PDR	DC
XGB	8.46	1.93	8.19	1.81	6.11	1.60	4.67	1.43
HGBT	8.42	1.93	8.21	1.84	6.15	1.58	4.71	1.63
GBRT	8.50	1.92	9.01	1.84	6.33	1.65	5.07	1.46
Lasso	8.74	2.08	10.45	2.17	6.53	1.78	5.45	1.71
ET	8.41	1.95	8.82	1.82	6.10	1.58	4.70	1.37
Ridge	8.68	1.93	9.49	2.03	6.31	1.61	5.14	1.52
SVR	9.58	3.03	8.75	2.66	7.15	2.75	5.86	2.50
KNN	9.67	3.80	10.46	3.23	6.96	3.07	5.69	4.06
MLP	9.05	1.96	9.00	2.13	6.93	1.80	5.69	2.08

while remaining effective on moderate-sized datasets. They also offer useful post-hoc interpretability through feature-importance analysis.

The final model is selected independently for each task–target pair by minimizing the Mean Absolute Error (MAE) under group-wise cross-validation:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|, \quad (6.13)$$

where N denotes the number of evaluated samples, y_i the measured target, and $\hat{y}_i$ its prediction.

Cross-validation is required because the telemetry samples collected within the same experiment are not independent: they share the same MAC configuration, routing convergence conditions, traffic setting, and environmental context. To avoid overly optimistic estimates, each experiment is treated as a distinct group. Let G_{train} and G_{test} denote the sets of experiment identifiers assigned to the training and test folds, respectively. The split is constrained as

$$G_{\text{train}} \cap G_{\text{test}} = \emptyset, \quad (6.14)$$

which ensures that each test fold contains only unseen experimental runs. The evaluation therefore measures generalization across runs rather than memorization of run-specific temporal trajectories.

Table 6.2 shows that boosting-based tree ensembles provide the strongest overall performance across the considered tasks. Extra Trees (ET) is selected for non-bursty PDR prediction, Contention Period (CP) recommendation PDR, and TDMA-slot recommendation duty cycle; Gradient Boosting Regression Trees (GBRT) for non-bursty duty-cycle prediction; Extreme Gradient Boosting (XGBoost) for both bursty prediction targets and TDMA-slot recommendation PDR; and Histogram-Based Gradient Boosting Trees (HGBT) for CP recommendation duty cycle. Overall, these results indicate that tree-based ensembles constitute the most reliable model family for deployment, although simpler regularized models can still remain competitive for specific targets. With offline training completed

and the models selected, the next step is to deploy them online for network monitoring and MAC-layer reconfiguration.

6.2.5 Online Closed-Loop Management

The online orchestrator integrates monitoring, learning, and reconfiguration at runtime. It continuously collects measurements from the nodes and constructs an updated state for each new observation window. This state is then provided to the trained prediction model in order to estimate the PDR over the next H windows. For each node i , the orchestrator computes the mean predicted PDR over this horizon and generates a proactive trigger when $\widehat{\text{PDR}}_{i,t}^{(H)} < \text{PDR}_{\text{target}}$ for p (*patience*) consecutive checks. Requiring p consecutive violations improves control robustness by preventing reactions to short-lived fluctuations or isolated degradations. Once this condition is met, the node is considered affected and the orchestrator invokes the recommendation stage to determine a new configuration expected to restore the target reliability in a forecast-driven manner. The selected configuration is then applied to the relevant nodes.

To avoid oscillatory behavior, each node is subject to a c (*cooldown*) period after reconfiguration during which further triggers are blocked. This mechanism ensures that the orchestrator has sufficient time to observe the effect of a previously applied action before issuing another one.

In addition to prediction-driven reconfiguration, the online orchestrator includes a burst-protection mode that activates TDMA when contention-based access is no longer suitable for the current traffic profile. The orchestrator enters the TDMA mode when the burstiness exceeds a threshold B_{th} for P_{on} consecutive windows on at least a fraction R_{on} of the monitored nodes. Conversely, it exits TDMA mode when the burstiness remains below B_{th} for P_{off} consecutive windows on at least a fraction R_{off} of the monitored nodes.

When TDMA mode is activated, the recommender additionally selects a common value of max_slot for the participating nodes according to the recent measurements and burstiness conditions, so as to satisfy the reliability target while limiting the duty-cycle cost.

This switching behavior can be summarized as

$$u_{t+1}^{\text{tdma}} = \begin{cases} 1, & \text{if } u_t^{\text{tdma}} = 0 \text{ and the TDMA-on condition holds,} \\ 0, & \text{if } u_t^{\text{tdma}} = 1 \text{ and the TDMA-off condition holds,} \\ u_t^{\text{tdma}}, & \text{otherwise,} \end{cases} \quad (6.15)$$

where $u_t^{\text{tdma}} \in \{0, 1\}$ denotes the current MAC operating mode.

Overall, the proposed control framework combines node-specific timing reconfiguration under contention-based operation with network-wide TDMA activation and a shared slot-allocation decision under bursty traffic. This design enables the network to react

Table 6.3: Traffic profiles used in the evaluation.

Profile	Initial Config.	Phase-1	Phase-2	Phase-3
Profile-1	$CP = 10$ ms $CP_{\max} = 30$ ms $SF = 400$ ms	Periodic (interval: 500 ms)		
Profile-2		Periodic (interval: 500 ms)	Bursty (30 pkts every 20 s)	Periodic (interval: 500 ms)
Profile-3		Periodic (interval: 1000 ms)	Bursty (15 pkts every 30 s)	Periodic (interval: 1000 ms)

proactively to predicted degradations while adapting its MAC operating mode and slot allocation to evolving traffic conditions.

6.3 Evaluation

The framework is evaluated through unseen experiments on the SLICES-FR, Grenoble site, whose node placement and radio environment differ from those used for training, allowing us to assess generalization to new topologies and conditions.

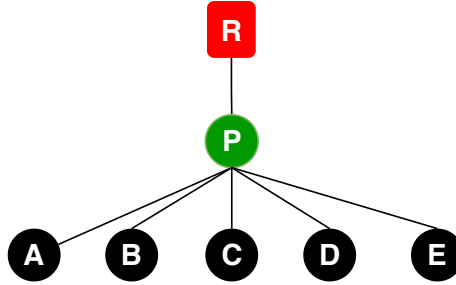


Figure 6.4: Controlled topology used to evaluate the framework

First, the controlled topology of Figure 6.4 is used, composed of a root (R), an intermediate parent (P), and five leaves (A – E). This topology is used to enable interpretable comparisons across scenarios while preserving fixed structural roles in the routing hierarchy. Detailed analysis focuses on R , P , and one representative leaf (A), which together capture the behavior of nodes subject to different forwarding responsibilities. The evaluation is then extended to a larger heterogeneous deployment.

For the evaluation, three traffic profiles are used across the experiments, as described in Table 6.3. After a 3-minute warm-up period to allow routing convergence, each profile consists of three phases that may exhibit different traffic patterns: low-rate periodic traffic, in which 1-byte UDP packets are generated at a given interval, and bursty traffic, in which a group of 1-byte UDP packets is generated at a given period. A 3-minute no-traffic interval is inserted between consecutive phases. In all experiments, the initial duty-cycle configuration is fixed to $CP = 10$ ms, $CP_{\max} = 30$ ms, and $SF = 400$ ms.

The evaluation focuses on the closed-loop adaptation logic itself. Communication

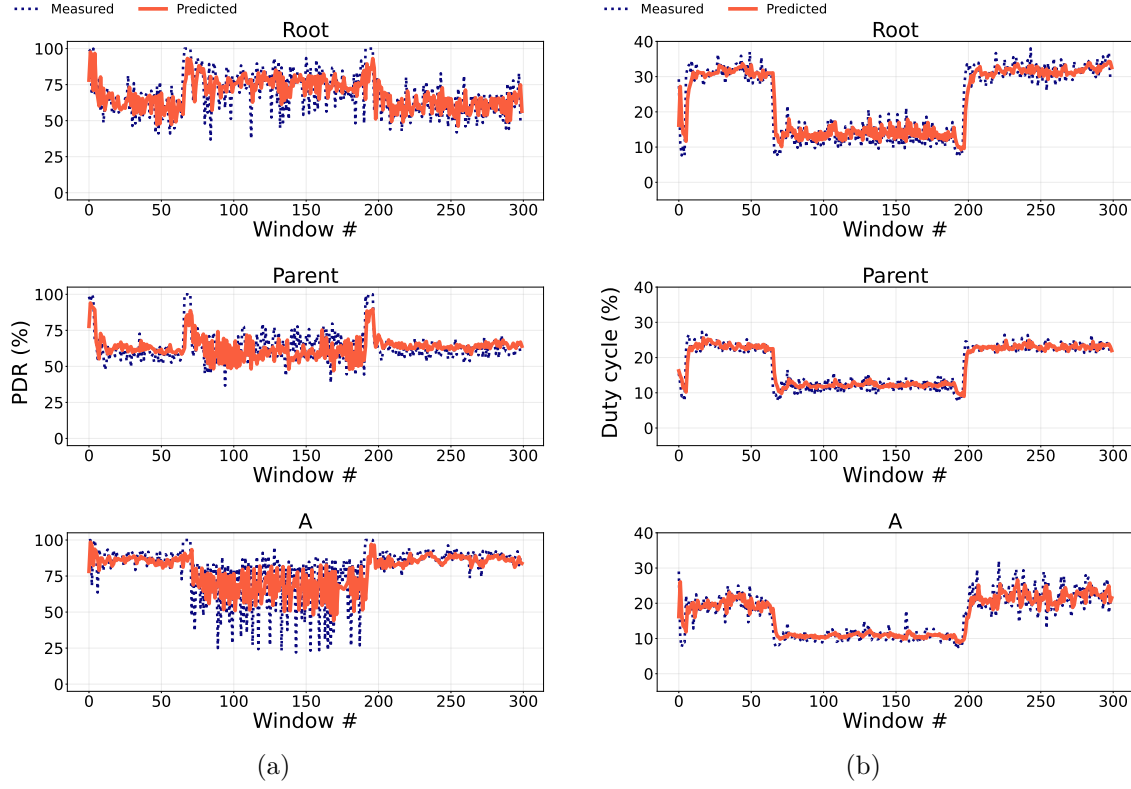


Figure 6.5: Measured versus predicted trajectories for representative nodes: (a) PDR and (b) duty cycle.

between the nodes and the network orchestrator is assumed to be reliable and is therefore considered outside the scope of this study. In practice, the network orchestrator collects telemetry through the SLICES-FR serial aggregation service and issues runtime reconfiguration commands accordingly. While this work does not directly rely on the full FINE-STACK implementation for practical and experimental convenience, it builds on its core concept of a reconfigurable protocol stack, where network functions can be dynamically updated at runtime.

To promote reproducibility and support further developments, open-source access is provided to the dataset, implementation, and evaluation results¹.

6.3.1 Prediction Model Performance

The forecasting model is first evaluated over the controlled topology *with TDMA disabled*. Profile-2 defined in Table 6.3 is used in a 2.5-hour experiment, with 30 minutes for Phase-1, 60 minutes for Phase-2, and the remaining time for Phase-3. Telemetry is collected every $\Delta = 30$ s. During rollout, the predictor is calibrated every k windows; between calibrations, it relies on its own previously predicted state.

Figure 6.5 reports the case $k = 1$, i.e., calibration at every window. The predicted

¹<https://github.com/ahmahmod/Traffic-aware-mac-control>

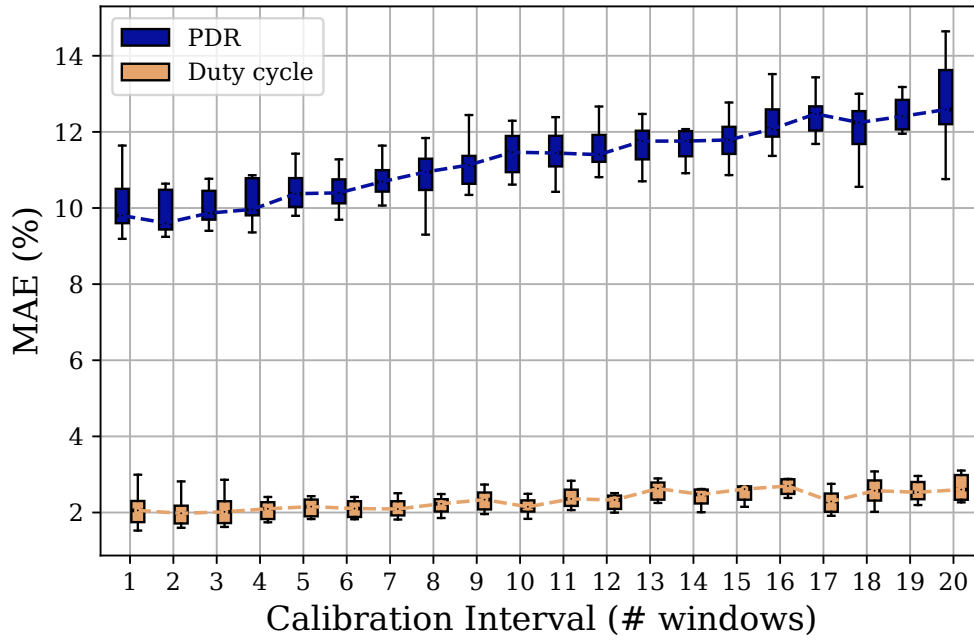


Figure 6.6: Impact of the calibration interval k on forecasting error. The error increases as k becomes larger, but remains within an acceptable range.

curves reproduce both the average level and the short-term variations. The bursty phase, particularly for the leaf A , occasionally produces deep and sharp PDR drops that are more difficult to predict. Nevertheless, the model still captures the overall degradation pattern and the changes in averages. This is sufficient for control because the recommender relies on the average forecast over the next H windows rather than on an isolated point estimate.

Duty-cycle forecasts offer complementary information. Here, duty cycle is used as a proxy for power consumption, since SLICES-FR does not provide a radio-specific power measurement tool. The model reproduces the phase-dependent change in radio activity and also captures the lower duty-cycle baseline observed in the bursty phase. This lower duty cycle does not reflect higher efficiency; rather, the configured contention period is insufficient to absorb burst arrivals when TDMA is disabled, so many packets are dropped before transmission and radio activity decreases. Capturing this joint PDR/duty behavior is important because it allows the orchestrator to distinguish true efficiency gains from degraded service.

In practice, frequent telemetry collection may be costly in LLWNs in terms of both power consumption and bandwidth usage. For this reason, this section also studies how the accuracy evolves as the calibration interval increases. Figure 6.6 quantifies the effect of increasing the calibration interval from $k = 1$ window (15 s) to $k = 20$ windows (300 s). As expected, the error grows with k because the predictor relies longer on self-generated states before receiving a measurement-based correction. Nevertheless, the increase remains moderate. For PDR, the MAE rises from about 10% at short intervals to roughly 12.5%

at $k = 20$. Duty cycle is easier to predict because it reflects aggregated radio activity over the window and is strongly constrained by the MAC timing configuration, whereas PDR depends on more stochastic packet-level events such as collisions, retransmission failures, queue drops, and burst-induced contention. As a result, duty-cycle trajectories are smoother and less sensitive to abrupt transient losses, leading to a median MAE that increases only from about 2% to 2.7%. The results indicate that the predictor remains useful even with relatively sparse calibration, which is important in LLWNs where traffic monitoring must remain limited.

Overall, the sweep shows that the proposed predictor preserves useful forecasting accuracy even for relatively large calibration intervals, which is particularly valuable in LLWNs because it enables a practical trade-off between prediction fidelity and telemetry overhead. Smaller values of k provide the best responsiveness and accuracy, while larger values reduce calibration traffic and monitoring cost while still maintaining sufficiently accurate forecasts for proactive control decisions.

6.3.2 Forecasting-based Recommendation Performance

To isolate the respective effects of recommendation and forecasting, and to highlight the importance of integrating both for proactive response, this section runs 45-minute experiments with Profile-1 (Table 6.3) generating low-rate periodic data for the entire duration. The PDR target is set to 70%.

Figure 6.7 compares: (i) no recommendation and no forecasting; (ii) recommendation without forecasting, where the orchestrator reacts to measured degradation; and (iii) recommendation with forecasting, where the orchestrator reacts to predicted degradation. The observation horizon is set to $N = 5$ and patience $p = 3$. In the reactive case, the recommender is triggered when the measured PDR remains below the target over the observation horizon for p consecutive checks.

Figure 6.7(a) shows that, under the static initial configuration with a periodic duty cycle of 400 ms, leaf A remains relatively reliable and can sustain one packet every 500 ms. In contrast, the parent and the root remain well below the target because their forwarding burden, which includes both their own traffic and that of their descendants, is much higher. In addition to receiving upstream traffic, the root must continuously exchange routing information to maintain the topology, which further increases its load.

When recommendation is enabled without forecasting (Figure 6.7(b)), the orchestrator eventually reconfigures the affected parent and root by reducing the duty-cycle period to 100 ms, thereby increasing the node wake-up rate and allowing both nodes to recover to a regime above the target. However, before reconfiguration is triggered, the root and the parent remain below the target for a substantial period, namely 252.1 s and 249.0 s, respectively, because the decision remains purely reactive.

The benefit of forecasting is evident in Figure 6.7(c). In this case, the orchestrator

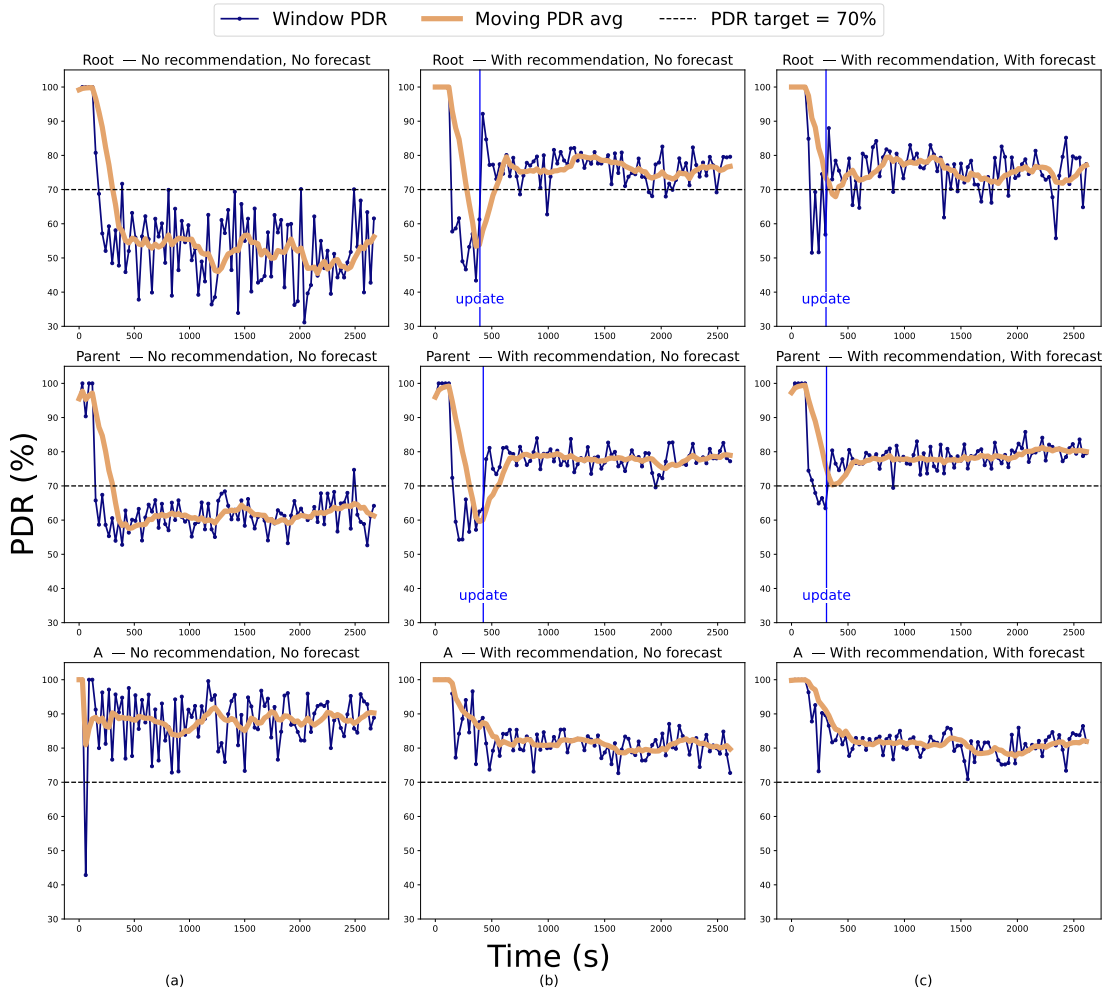


Figure 6.7: PDR comparison under three scenarios: (a) no recommendation and no forecasting, (b) recommendation without forecasting, and (c) recommendation with forecasting.

anticipates degradation and updates the configuration before the moving-average PDR falls significantly below the target. The root remains below the target for only 62.4 s, while the parent avoids any sub-target period entirely, corresponding to reductions in degradation time of 75.2% and 100%, respectively. The few transient windows that still remain below the target are mainly due to the patience mechanism, since the orchestrator must still wait for $p = 3$ consecutive checks before applying the reconfiguration. As a result, the parent and the root avoid the prolonged sub-target regime observed in the purely reactive case. These results show that prediction improves the timing of control actions, thereby reducing both the duration and severity of performance degradation.

Overall, the results demonstrate that dynamic reconfiguration can achieve the target PDR in low-rate scenarios by adapting duty-cycle timing, without requiring a permanently active and costly TDMA service.

6.3.3 Adaptability to Data Profiles

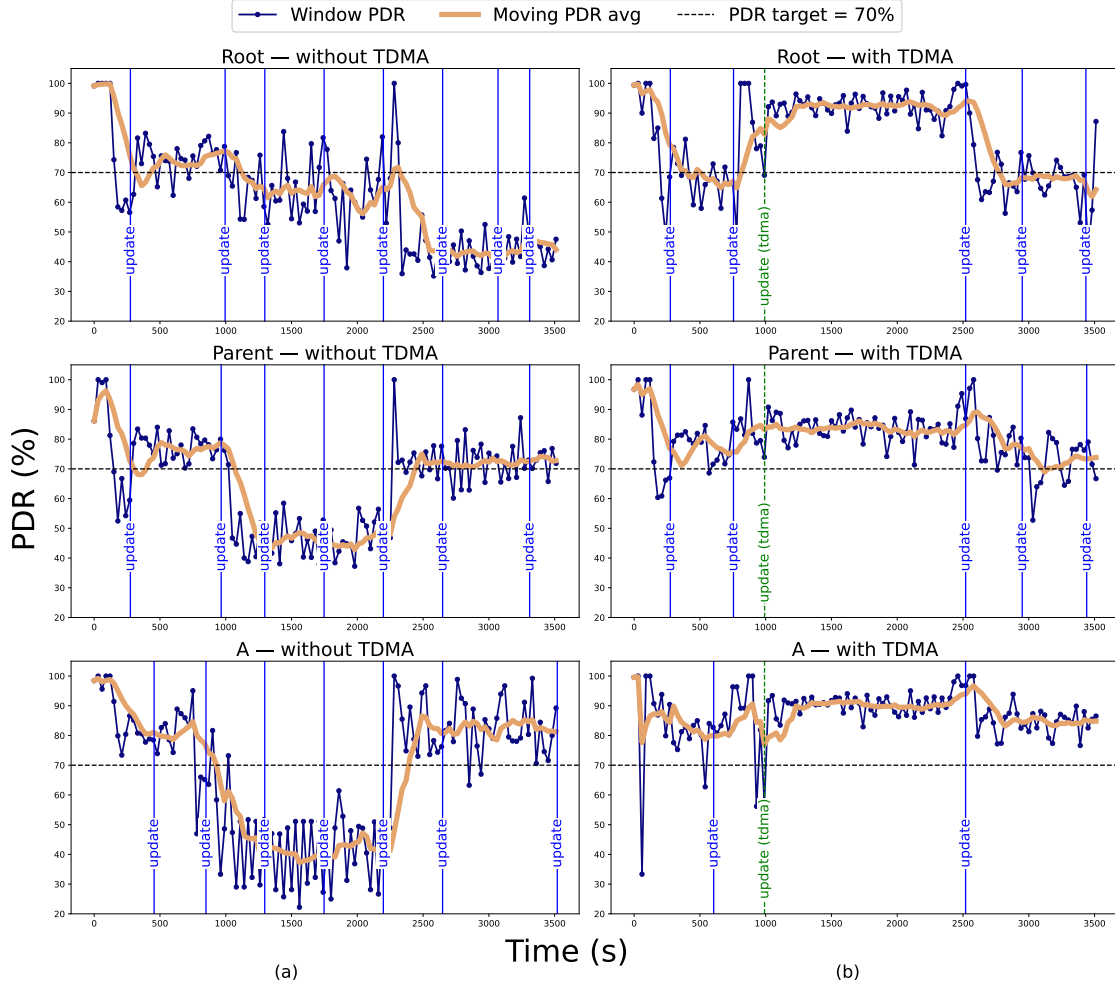


Figure 6.8: PDR comparison under mixed traffic profile: (a) without TDMA and (b) with dynamic TDMA.

This section evaluates adaptability under one-hour experiments with a mixed traffic pattern defined by Profile-2 (Table 6.3) in the controlled topology. Phase-1 lasts 10 minutes, Phase-2 lasts 30 minutes, and Phase-3 occupies the remaining time. The PDR target is set to 70%. Figure 6.8 compares two cases: (i) TDMA unavailable, such that the orchestrator can only tune the duty-cycle timing; and (ii) TDMA available and activated when burstiness becomes high, namely when at least 20% of nodes satisfy $b_{i,t} \geq 0.5$. In the figures, solid vertical markers indicate duty-cycle timing reconfigurations, whereas dashed markers indicate TDMA-enabled reconfigurations.

Without TDMA (Figure 6.8(a)), the orchestrator applies duty-cycle timing adaptation, and the configuration ($CP = 10$ ms, $CP_{\max} = 30$ ms, and $SF = 100$ ms) is sufficient during the low-rate periodic phases. However, the bursty phase generates a large number of packets that are injected almost simultaneously over a short interval, which creates sustained queue buildup, repeated channel conflicts, retransmissions, and queue drops at a

Table 6.4: Comparison of performance under No TDMA, Always-on TDMA, and Dynamic TDMA.

Metric	no TDMA	always TDMA	dynamic TDMA
PDR (%)	66.4	90.9	85.3
Queue loss (%)	21.8	5.3	5.5
Failure (%)	11.7	3.7	9.2
Duty-cycle (%)	18.7	36.5	25.5

rate that exceeds the ability of duty-cycled contention-based medium access to absorb them. As a result, all nodes fall well below the 70% target, despite the orchestrator continuing to apply best-effort contention-parameter updates.

When dynamic TDMA is available (Figure 6.8(b)), the orchestrator applies duty-cycle timing adaptation during the low-rate periodic phase, but it switches to TDMA during the bursty phase and selects a suitable allocated number of slots $max_slot = 30$. Reliability then recovers rapidly to a regime close to or above the target because scheduled access reduces contention and serves queued burst packets more deterministically. Once traffic returns to periodic operation, TDMA is disabled and the previous configuration is restored. Later reconfigurations may be applied either with the same parameters to include newly affected nodes or with a completely new configuration.

To highlight the importance of dynamically reconfiguring the contention-based duty cycle timing and enabling TDMA only when contention-based access becomes insufficient, Table 6.4 compares the PDR and duty cycle of three scenarios: TDMA permanently disabled, TDMA permanently enabled, and our framework, in which TDMA is activated dynamically only when needed. Without TDMA, the network cannot achieve the 70% target because the bursty phase generates a substantial queue loss that cannot be handled by reconfiguring only contention-based duty-cycle access. Always-on TDMA achieves the highest PDR and the lowest queue loss, but at a substantially higher duty-cycle cost, almost twice that of the No TDMA scenario.

In contrast, our dynamic-TDMA framework achieves the target by enabling TDMA only when necessary: it reaches queue-loss levels close to those of the always-on TDMA case while reducing the duty-cycle cost by 30.13% relative to permanent TDMA. These results show that TDMA should not remain permanently disabled or permanently enabled; rather, it should be activated selectively during bursty periods and released when contention-based operation becomes sufficient again. This is justified by the synchronization overhead of TDMA, which may become comparable to the effective data traffic in low-rate scenarios, resulting in longer radio activity and higher power consumption.

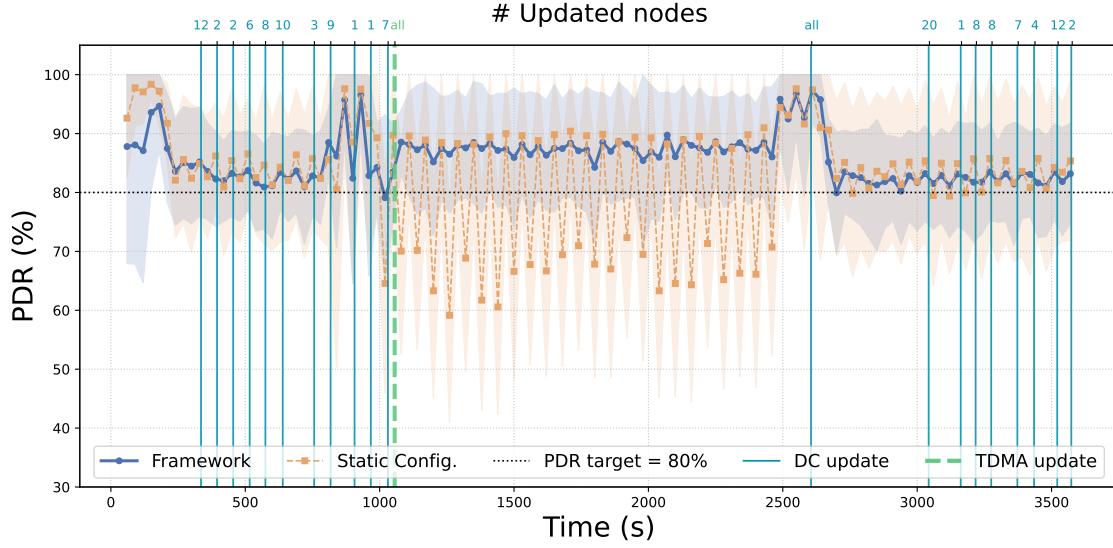


Figure 6.9: Global average PDR in the 50-node heterogeneous topology under both closed-loop dynamic control and static configuration.

6.3.4 Scalability over a Large Heterogeneous Topology

This section evaluates the scalability of the proposed framework on a larger multi-hop deployment with greater topology heterogeneity, forwarding asymmetry, and different QoS levels. The largest connected 50-node subset at the Grenoble site is used, where nodes form routes to the root over 1–4 hops. The network runs for one hour under the mixed traffic Profile-3 (Table 6.3). Phase-1 lasts 10 minutes, Phase-2 lasts 30 minutes, and Phase-3 covers the remaining time. The PDR target is set to 80% to validate adaptability to different QoS levels. Two scenarios are compared: (i) dynamic configuration using the proposed framework and (ii) a static configuration without TDMA and runtime reconfiguration. This section reports only PDR results in this experiment, as duty-cycle values exhibit strong spatial and temporal heterogeneity across nodes in this irregular multi-hop topology, which makes aggregate statistics such as mean values and confidence intervals non-informative.

Figure 6.9 reports the global average PDR with its standard deviation, together with the runtime reconfiguration actions applied during dynamic operation. At the beginning of the initial periodic phase, several nodes trigger duty-cycle timing updates to accommodate the generated traffic. After this initial adjustment, reconfiguration becomes more selective and mainly targets forwarding nodes, allowing the network to maintain the target PDR with lower variability than in the static scenario. When the bursty phase begins, the orchestrator issues a global TDMA-enabled update with a common slot-allocation budget $max_slot = 100$, which keeps the average PDR above the target. By contrast, the static configuration suffers a severe degradation during the bursty phase. After the bursty phase, TDMA is disabled again, and only occasional duty-cycle timing updates are applied. This behavior illustrates the two complementary adaptation mechanisms at scale: contention-parameter tuning under moderate load to limit energy cost, and TDMA switching under

burst-induced stress to preserve reliability. Quantitatively, the static configuration spends 31.7% of the experiment below the target PDR, whereas the proposed framework reduces this duration to 14.6%.

Taken together, these results show that the proposed framework remains effective in a larger and more heterogeneous multi-hop setting and adapts its decisions to both evolving traffic conditions and topology-dependent forwarding load.

6.4 Conclusion

This chapter presented the control loop responsible for collecting metrics, predicting network degradation, and computing new configurations, thereby providing the missing building blocks required for FINE-STACK to achieve a complete network stack reconfiguration architecture. It uses per-node telemetry to capture communication quality, topology-dependent forwarding burden, and traffic burstiness, then exploits learning-based models to forecast short-horizon PDR and duty-cycle evolution and to select suitable MAC-layer configurations at runtime. In this way, the proposed approach supports contention-based duty-cycle timing adaptation under moderate traffic and TDMA-oriented protection under bursty traffic.

The experimental results on the SLICES-FR testbed showed that the proposed framework remains effective on unseen deployments with different topological and radio conditions. The forecasting results demonstrated that the predictor captures the main evolution of both PDR and duty cycle across periodic and bursty phases while remaining useful even when calibration becomes less frequent. The online results further showed that integrating forecasting with recommendation improves the timing of runtime actions and reduces the duration of sub-target operation compared with purely reactive control. Under mixed traffic, the framework adapted its MAC operating mode according to the traffic regime: contention-based timing adaptation was sufficient during low-rate periodic phases, whereas TDMA activation became necessary during bursty phases to restore reliability.

This chapter focused on MAC-layer reconfiguration because of its central role in balancing the capacity-energy trade-off. It also emphasized the integrated architecture of real-time measurement collection, forecasting, recommendation, and runtime reconfiguration, rather than the design of the machine-learning models themselves. Future work may extend the configuration space beyond the MAC layer to other layers, such as routing, in order to achieve full-stack recommendation and cross-layer optimization. In addition, investigating online learning models that continuously retrain after deployment could improve long-term adaptability under evolving network conditions.

Conclusion and Perspectives

Chapter contents

7.1	Thesis Contributions	108
7.2	Limitations and Perspectives	110
7.2.1	Modular Network Stack	110
7.2.2	Update Management	111
7.2.3	Update Automation	113
7.3	Concluding Perspective	113

This thesis aimed to advance LLWNs toward practical network programmability by narrowing the gap between flexibility and efficiency and by making frequent, semantic, and automated runtime reconfiguration feasible on highly constrained IoT devices. To this end, it proposed a network framework that represents a shift from hardware-centric to software-driven networking, paving the way for fully programmable wireless networks.

7.1 Thesis Contributions

This thesis provides three progressive contributions, as follows:

Modular Network Stack Design. Chapter 4 presented the first contribution, which introduced a modular network-stack design for constrained LLWN devices. Instead of treating the communication stack as a monolithic firmware component, the proposed architecture decomposes protocol functionality into lightweight virtual functions (VFs). Each VF implements a specific network function and can be independently installed, updated, replaced, or reused. This design enables fine-grained programmability at the level of network functions rather than at the level of complete firmware images or coarse

protocol modules. This contribution shows that selective runtime updates can be made practical on constrained devices. By updating only the modified function, the proposed architecture significantly reduces update size and update time compared with full-image OTA updates. It also avoids unnecessary replacement of unchanged parts of the system and reduces the amount of traffic injected into the LLWN during maintenance operations.

The contribution was validated progressively through the implementation of concrete protocol functions using the VF-based design. First, UDP was decomposed into lightweight functions for packet transmission and reception, showing that transport-layer functionality can be modularized and updated selectively. The approach was then extended to RPL, where the routing protocol was decomposed into several VFs responsible for distinct control plane operations, such as processing routing messages and managing routing-state transitions. These implementations demonstrate that VF-based programmability is not limited to simple application-level extensions, but can also be applied to core network-stack functions.

Hybrid Control Plane Deployment. The second contribution, presented in Chapter 5, augmented the modular network-stack design with flexible control plane deployments, especially hybrid deployments. This contribution showed that efficient updates are not only a matter of reducing binary size. Runtime updates of network protocols must also account for the meaning of the modified function and the state associated with it. To address this issue, this contribution introduced state-preserving orchestration through hybrid control plane deployments.

This idea was evaluated and compared with fully distributed and centralized control plane models. The distributed model preserves local autonomy and resilience, but may suffer from slow convergence and local suboptimality [90]. The centralized model enables global optimization and fast rule installation, but depends strongly on the reliability of the multi-hop backhaul [18]. The hybrid model combines the advantages of both approaches by keeping local control plane logic on the devices while allowing the orchestrator to inject global knowledge, restore state, and accelerate post-update recovery.

The results showed that hybrid deployment was able to achieve near-instant convergence after updates that would typically require reconvergence in distributed deployments. This was achieved by saving the network state globally and restoring it after the update, thereby avoiding the need for lengthy local control plane exchanges. In addition, the hybrid model demonstrated resilience under intermittent conditions when central optimization was unavailable by relying on local control planes to maintain connectivity and avoid service interruptions.

Self-Adaptive and Proactive Network Reconfiguration. Chapter 6 presented the third contribution, which focused on the automation of network updates. This contribution addressed the performance-optimization problem under changing traffic regimes and

heterogeneous forwarding roles by proposing a foundation for an AI-based orchestration framework for the MAC layer in LLWNs. The main goal was to balance performance and power efficiency in multi-hop LLWN topologies, where some forwarding nodes carry more traffic than other nodes and therefore experience different operating conditions. The proposed closed-loop framework combines runtime monitoring, short-horizon forecasting, recommendation, and online reconfiguration. It collects per-node telemetry to characterize communication quality, forwarding burden, traffic burstiness, and current MAC behavior. Learning-based models then forecast near-future performance and power-consumption evolution. Based on these forecasts, the orchestrator recommends suitable MAC-layer reconfigurations, including duty-cycle timing adaptation under moderate traffic and TDMA-oriented protection under bursty traffic.

The evaluation showed that forecasting improves the timing of runtime decisions compared with purely reactive control. Instead of waiting for performance degradation to become severe, the orchestrator can anticipate near-future drops in performance and trigger reconfiguration earlier. The results also showed that the framework adapts its control strategy to the traffic regime. During periodic or moderate traffic phases, contention-based timing adaptation is generally sufficient and avoids unnecessary duty-cycle costs that can result from running TDMA continuously. During bursty phases, TDMA activation becomes necessary to restore reliability. This contribution therefore completes the thesis by adding intelligence to the orchestration layer, transforming programmable updates into an automated control loop.

7.2 Limitations and Perspectives

Revisiting the contributions of this thesis, which are distributed between the LLWN devices and the orchestrator as illustrated in Figure 1.2, several limitations remain and open promising directions for future research for each aspect of the proposed framework.

7.2.1 Modular Network Stack

Network-dedicated virtualization technique. This thesis adopted Femto Container [88] as the virtualization technique used to host virtual functions, although it was not originally designed for network-stack execution. This choice introduced several practical limitations during the implementation of UDP and RPL as VFs. First, each VF had to interact with internal operating-system structures through explicitly defined interfaces, because direct access to data structures, packet buffers, timers, routing tables, and MAC- or network-layer state is restricted. Second, protocol structures often had to be redefined or duplicated across different VFs, increasing code size and creating a risk of inconsistency with the native stack. Third, the separation between VFs prevents some compiler optimizations that are available in monolithic implementations, such as global

optimization across shared data structures. As a result, these constraints may increase memory usage, processing overhead, and timing overhead.

Future work should therefore investigate virtualization techniques specifically designed for network functions in constrained devices. Such a network-dedicated virtualization layer should provide stable access to common packet, timer, neighbor, routing, queue, and radio abstractions; shared protocol headers and data structures to avoid duplication; and, when possible, safe access to packet buffers. It should also include network-aware verification mechanisms capable of checking not only memory safety, but also compatibility with timing constraints and update dependencies. These improvements would reduce execution overhead, code size, and development complexity, while making VF-based network-stack programming more systematic and efficient.

Network-stack granularity and protocol scope. The limitations of the adopted virtualization technique also raise a broader scientific question: how should protocol functionality be decomposed into VFs? The thesis validated the approach with UDP and RPL, which represent transport- and network-layer protocols where packet processing, control-message handling, and state management can be separated into relatively well-defined functions. In these cases, the VF-based design remains manageable and demonstrates the feasibility of modular network-stack programmability. However, this does not imply that the same decomposition strategy can be applied uniformly across all protocol layers. Very coarse-grained VFs reduce flexibility and make updates closer to classical module replacement, whereas overly fine-grained VFs may increase interaction overhead, state-sharing complexity, timing overhead, and management cost.

This trade-off becomes even more critical at lower layers. MAC-layer functions interact directly with duty cycling, queues, retransmissions, contention periods, radio wake-up schedules, and sometimes TDMA slot boundaries; therefore, their VF-based implementation must preserve strict timing behavior and avoid excessive execution overhead. The validation of the UDP and RPL protocols covers important transport and routing behaviors, but they do not exhaust the full diversity of LLWN protocol stacks. Future work should extend the VF-based approach to additional protocols and layers. It should therefore define a systematic methodology for selecting VF boundaries according to update frequency, state dependencies, timing sensitivity, expected reuse across protocols, and degree of hardware dependence. Such a methodology would make it possible to characterize, in a formal and reproducible way, the trade-off between programmability, resource overhead, update size, and runtime performance, while also identifying which lower-layer functions can be safely virtualized and which should remain native.

7.2.2 Update Management

Robust orchestration and update strategy. The proposed update mechanism uses a coordinated install-and-activate procedure, but it assumes a reliable backhaul channel with

the orchestrator. In practice, however, update robustness depends strongly on how the management plane is deployed. If update traffic shares the same wireless resources as application and control/management traffic, LLWNs may experience delayed update delivery, intermittent connectivity, temporary partitions, and update failures. Future work should therefore investigate robust update strategies under unstable network conditions, including retransmission tuning, staged rollouts, opportunistic retries, and delayed activation.

A complementary direction is to study dedicated management-plane resources, such as reserved time slots, prioritized management traffic, or separate channels for update dissemination. Such mechanisms could improve update reliability and reduce interference with application traffic, but they also consume scarce communication resources and may reduce the capacity available for normal data forwarding. Therefore, the design of the management plane should explicitly balance update reliability, data-plane performance, and energy cost.

Another important challenge concerns partial updates and network consistency. If only a subset of nodes successfully installs a new configuration, the network may become partitioned or temporarily contain incompatible protocol behaviors. For example, some nodes may switch to a new routing mode or MAC configuration while their neighbors continue to operate with the old one. Future work should therefore define partition-aware activation policies, version-compatibility rules, safe fallback modes, and consistency checks before activation. Such mechanisms would strengthen the operational reliability of programmable LLWNs in long-term deployments.

Coordination between local and global control planes. One limitation of the proposed hybrid control plane deployment is the possibility of conflicts between global and local decisions. Since both the orchestrator and the local control plane may influence the same forwarding, routing, or scheduling state, inconsistent decisions may arise if their responsibilities are not clearly coordinated. For example, the orchestrator may install a globally optimized rule based on its network-wide view, while a local control plane protocol may later modify this rule according to local link-quality variations or neighborhood conditions. This can create an oscillatory overwrite effect, where global and local decisions repeatedly override each other. Although this thesis introduced state restoration after updates, a complete conflict-resolution framework between global and local control plane decisions remains an open research direction.

Future work should therefore investigate closer coordination between the two control planes. One possible direction is to define authority and priority rules that specify which decisions can be modified locally and under which conditions. For instance, local control could be prevented from overriding globally installed decisions unless the global rule becomes obsolete or connectivity with the orchestrator is interrupted. Such coordination could be supported through rule lifetimes, versioning, fallback policies, and explicit ownership of network state. This would allow the hybrid model to preserve the benefits of global

optimization while maintaining the responsiveness and resilience of local control in an efficient way.

7.2.3 Update Automation

The third contribution uses learning-based forecasting and recommendation to support MAC-layer adaptation. However, the action space can be extended beyond the current timing and TDMA-oriented configurations. Future work should consider larger datasets with a greater diversity of configurations, traffic profiles, topology conditions, and performance metrics. At the MAC layer, reconfiguration could include richer scheduling policies, contention parameters, slot-allocation strategies, and duty-cycle mechanisms. Beyond the MAC layer, the configuration space could also be extended to other layers of the network stack, especially routing. For example, the recommender could jointly select MAC parameters and routing configurations according to predicted forwarding burden, link quality, traffic regime, and reliability requirements. This would move the framework from MAC-layer adaptation toward full-stack recommendation and cross-layer optimization.

Such an extension is particularly relevant in LLWNs because performance degradation is rarely caused by a single layer in isolation. Routing decisions influence forwarding load and congestion at intermediate nodes, while MAC-layer configurations determine whether this load can be served efficiently. A cross-layer recommender could therefore decide whether a performance issue should be addressed by adapting MAC timing, enabling TDMA-oriented protection, changing routing behavior, redistributing forwarding load, or combining several actions. This would provide a more complete automation framework in which runtime updates are selected according to the global state of the network rather than according to a single protocol or layer.

Developing a dedicated machine-learning model for decision-making was beyond the scope of this contribution. Future work should also investigate learning models that improve after deployment. The current approach relies on models trained offline on experimental datasets, which may limit long-term adaptability when the network evolves beyond the conditions observed during training. Online learning, continual retraining, or periodic model adaptation could allow the framework to incorporate new measurements collected during operation and progressively adapt to changing topology, interference, traffic patterns, or hardware conditions. Such extensions would improve robustness when the environment differs from training conditions and would make the closed-loop framework more suitable for long-term autonomous LLWN operation.

7.3 Concluding Perspective

Nikola Tesla once anticipated that wireless communication would come to mankind “*in its full meaning like a hurricane*” [127]. More than a century later, this vision has partly

materialized through the pervasive deployment of wireless technologies and, in particular, through the rapid expansion of constrained IoT networks in everyday life. Yet, despite their growing societal and technological importance, the evolution of these networks remains largely constrained by their rigid and static design.

Beyond the specific context of low-power and lossy wireless networks, this thesis contributes to a broader transformation in computer networking: the transition from fixed protocol implementations toward programmable, composable, and adaptive network functions. By decomposing protocol behavior into lightweight functional blocks that can be installed, replaced, combined, and orchestrated at runtime, the proposed approach introduces a new degree of flexibility into constrained IoT environments. This evolution echoes the transformation already observed in 5G and cloud-native infrastructures, where network functions are increasingly software-based, modular, virtualized, and dynamically managed. In this sense, the thesis opens a path toward a new generation of LLWN architectures in which even highly constrained devices are no longer bound to static protocol stacks, but can instead evolve continuously according to application requirements, network conditions, and long-term sustainability objectives.

As in any research work, the contributions presented in this thesis are subject to limitations arising from simplifying assumptions, implementation constraints, and experimental conditions. Several issues remain open, as discussed in the previous section, and further work is required to move from the proposed prototypes toward a fully mature and general-purpose architecture for programmable constrained networks. Nevertheless, this thesis establishes important foundations for programmable and adaptive LLWNs by demonstrating the feasibility of runtime protocol decomposition and intelligent network adaptation. By making these building blocks available as open source and by identifying open research directions, this work aims to encourage further contributions from the research community to this emerging paradigm.

Finally, I hope that the results presented in this thesis will support future research efforts and contribute to the advancement of low-power and lossy wireless networks and, more broadly, to the design of adaptive computer networks.

Bibliography

- [1] Ala Al-Fuqaha, Mohsen Guizani, Mehdi Mohammadi, Mohammed Aledhari, and Moussa Ayyash. “Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications”. In: *IEEE Communications Surveys & Tutorials* 17.4 (2015), pp. 2347–2376. DOI: 10.1109/COMST.2015.2444095.
- [2] Daniele Miorandi, Sabrina Sicari, Francesco De Pellegrini, and Imrich Chlamtac. “Internet of things: Vision, applications and research challenges”. In: *Ad Hoc Networks* 10.7 (2012), pp. 1497–1516.
- [3] Carsten Bormann, Mehmet Ersue, and Ari Keränen. *Terminology for Constrained-Node Networks*. RFC 7228. May 2014. DOI: 10.17487/RFC7228. URL: <https://www.rfc-editor.org/info/rfc7228>.
- [4] R. Porkodi and V. Bhuvaneswari. “The Internet of Things (IoT) Applications and Communication Enabling Technology Standards: An Overview”. In: *2014 International Conference on Intelligent Computing Applications*. 2014, pp. 324–329. DOI: 10.1109/ICICA.2014.73.
- [5] Aditya Gaur, Bryan Scotney, Gerard Parr, and Sally McClean. “Smart City Architecture and its Applications Based on IoT”. In: *Procedia computer science* 52 (2015), pp. 1089–1094.
- [6] Satyajit Sinha. *State of IoT 2025: Number of connected IoT devices growing 14% to 21.1 billion globally*. IoT Analytics. Oct. 28, 2025. URL: <https://iot-analytics.com/number-connected-iot-devices/> (visited on 06/07/2026).
- [7] Michael Chui, Mark Collins, and Mark Patel. *IoT value set to accelerate through 2030: Where and how to capture it*. McKinsey & Company. Nov. 9, 2021. URL: <https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/iot-value-set-to-accelerate-through-2030-where-and-how-to-capture-it> (visited on 06/08/2026).
- [8] Global Market Insights. *Wireless Sensor Network Market Size & Share 2025–2034*. Global Market Insights. Feb. 2025. URL: <https://www.gminsights.com/industry-analysis/wireless-sensor-network-market> (visited on 06/07/2026).

- [9] Maria Trigka and Elias Dritsas. “Wireless Sensor Networks: From Fundamentals and Applications to Innovations and Future Trends”. In: *IEEE Access* 13 (2025), pp. 96365–96399. DOI: 10.1109/ACCESS.2025.3572328.
- [10] Afonso Oliveira and Teresa Vazão. “Low-Power and Lossy Networks under Mobility: A Survey”. In: *Computer Networks* 107 (2016), pp. 339–352. DOI: 10.1016/j.comnet.2016.03.018.
- [11] Jan Bauwens, Peter Ruckebusch, Spilios Giannoulis, Ingrid Moerman, and Eli De Poorter. “Over-the-Air Software Updates in the Internet of Things: An Overview of Key Principles”. In: *IEEE Communications Magazine* 58.2 (2020), pp. 35–41. DOI: 10.1109/MCOM.001.1900125.
- [12] Seung Yeob Shin, Shiva Nejati, Mehrdad Sabetzadeh, Lionel C Briand, Chetan Arora, and Frank Zimmer. “Dynamic adaptation of software-defined networks for IoT systems: A search-based approach”. In: *Proceedings of the IEEE/ACM 15th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. 2020, pp. 137–148.
- [13] Koen Zandberg, Kaspar Schleiser, Francisco Acosta, Hannes Tschofenig, and Emmanuel Baccelli. “Secure Firmware Updates for Constrained IoT Devices Using Open Standards: A Reality Check”. In: *IEEE Access* 7 (2019), pp. 71907–71920. DOI: 10.1109/ACCESS.2019.2919760.
- [14] Peter Ruckebusch, Spilios Giannoulis, Ingrid Moerman, Jeroen Hoebeke, and Eli De Poorter. “Modelling the energy consumption for over-the-air software updates in LPWAN networks: SigFox, LoRa and IEEE 802.15. 4g”. In: *Internet of Things* 3 (2018), pp. 104–119.
- [15] Hauke Petersen, Martine Lenders, Matthias Wählisch, Oliver Hahm, and Emmanuel Baccelli. “Old Wine in New Skins? Revisiting the Software Architecture for IP Network Stacks on Constrained IoT Devices”. In: *Proceedings of the 2015 Workshop on IoT Challenges in Mobile and Industrial Systems*. New York, NY, USA: Association for Computing Machinery, 2015, pp. 31–35. DOI: 10.1145/2753476.2753485.
- [16] Iliar Rabet, Shunmuga Priyan Selvaraju, Hossein Fotouhi, Mário Alves, Maryam Vahabi, Ali Balador, and Mats Björkman. “SDMob: SDN-Based Mobility Management for IoT Networks”. In: *Journal of Sensor and Actuator Networks* 11.1 (2022), p. 8.
- [17] Michael Baddeley, Reza Nejabati, George Oikonomou, Mahesh Sooriyabandara, and Dimitra Simeonidou. “Evolving SDN for Low-Power IoT Networks”. In: *2018 4th IEEE Conference on Network Softwarization and Workshops (NetSoft)*. 2018, pp. 71–79. DOI: 10.1109/NETSOFT.2018.8460125.
- [18] Farzad Veisi, Julien Montavont, and Fabrice Theoleyre. “Link Quality Estimation in Wireless Software Defined Network with a Reliable Control Plane”. In: *2023 IEEE 9th International Conference on Network Softwarization (NetSoft)*. 2023, pp. 10–18. DOI: 10.1109/NetSoft57336.2023.10175475.

- [19] Giuseppe Anastasi, Marco Conti, Mario Di Francesco, and Andrea Passarella. “Energy conservation in wireless sensor networks: A survey”. In: *Ad Hoc Networks* 7.3 (2009), pp. 537–568.
- [20] Ricardo C. Carrano, Diego Passos, Luiz C. S. Magalhaes, and Celio V. N. Albuquerque. “Survey and Taxonomy of Duty Cycling Mechanisms in Wireless Sensor Networks”. In: *IEEE Communications Surveys & Tutorials* 16.1 (2014), pp. 181–194. DOI: 10.1109/SURV.2013.052213.00116.
- [21] Ilker Demirkol, Cem Ersoy, Fatih Alagöz, and Hakan Delic. “The impact of a realistic packet traffic model on the performance of surveillance wireless sensor networks”. In: *Computer Networks* 53.3 (2009), pp. 382–399.
- [22] Carlos M. García Algora, Vitalio Alfonso Reguera, Nikos Deligiannis, and Kris Steenhaut. “Review and Classification of Multichannel MAC Protocols for Low-Power and Lossy Networks”. In: *IEEE Access* 5 (2017), pp. 19536–19561. DOI: 10.1109/ACCESS.2017.2748178.
- [23] Jonathan W. Hui and David E. Culler. “Extending IP to Low-Power, Wireless Personal Area Networks”. In: *IEEE Internet Computing* 12.4 (2008), pp. 37–45. DOI: 10.1109/MIC.2008.79.
- [24] Hanin Almutairi and Ning Zhang. “A Survey on Routing Solutions for Low-Power and Lossy Networks: Toward a Reliable Path-Finding Approach”. In: *Network* 4.1 (2024), pp. 1–32. DOI: 10.3390/network4010001.
- [25] Chi-Anh La, Liviu-Octavian Varga, Martin Heusse, and Andrzej Duda. “Energy-efficient multi-hop broadcasting in low power and lossy networks”. In: *Proceedings of the 17th ACM international conference on Modeling, analysis and simulation of wireless and mobile systems*. 2014, pp. 41–50.
- [26] Sergio Barrachina-Muñoz, Boris Bellalta, Toni Adame, and Albert Bel. *Multi-Hop Communication in the Uplink for LPWANs*. 2017. arXiv: 1611.08703 [cs.NI].
- [27] Laila Al-Qaisi, Suhaidi Hassan, and Nur Haryani Binti Zakaria. “Secure Routing Protocol for Low Power and Lossy Networks Against Rank Attack: A Systematic Review”. In: *International Journal of Advanced Computer Science and Applications* 13.5 (2022), pp. 330–339.
- [28] Adrian Pekar, Jozef Mocnej, Winston K. G. Seah, and Iveta Zolotova. “Application Domain-Based Overview of IoT Network Traffic Characteristics”. In: *ACM Computing Surveys* 53.4 (2020), pp. 1–33. DOI: 10.1145/3399669.
- [29] Arunan Sivanathan, Daniel Sherratt, Hassan Habibi Gharakheili, Adam Radford, Chamith Wijenayake, Arun Vishwanath, and Vijay Sivaraman. “Characterizing and Classifying IoT Traffic in Smart Cities and Campuses”. In: *Proceedings of the IEEE Conference on Computer Communications Workshops*. IEEE, 2017.

- [30] Arunan Sivanathan, Hassan Habibi Gharakheili, Franco Loi, Adam Radford, Chamith Wijenayake, Arun Vishwanath, and Vijay Sivaraman. “Classifying IoT Devices in Smart Environments Using Network Traffic Characteristics”. In: *IEEE Transactions on Mobile Computing* 18.8 (2019), pp. 1745–1759. DOI: 10.1109/TMC.2018.2866249.
- [31] Benjamin Finley, Jaume Benseny, Alexey Vesselkov, and Jaspreet Walia. “How Does Enterprise IoT Traffic Evolve? Real-World Evidence from a Finnish Operator”. In: *Internet of Things* 12 (2020), p. 100294. DOI: 10.1016/j.iot.2020.100294.
- [32] Basim K. J. Al-Shammari, Nadia Al-Aboody, and Hamed S. Al-Raweshidy. “IoT Traffic Management and Integration in the QoS Supported Network”. In: *IEEE Internet of Things Journal* 5.1 (2018), pp. 352–370.
- [33] Giacomo Tanganelli, Carlo Vallati, and Enzo Mingozzi. “Ensuring Quality of Service in the Internet of Things”. In: *New Advances in the Internet of Things*. Springer International Publishing, 2018, pp. 139–163. DOI: 10.1007/978-3-319-58190-3_9.
- [34] Yufei Li and Wanqing Tu. “Traffic Modelling for IoT Networks: A Survey”. In: *Proceedings of the ACM International Conference*. 2020. DOI: 10.1145/3418981.3418986.
- [35] JeongGil Ko, Andreas Terzis, Stephen Dawson-Haggerty, David E. Culler, Jonathan W. Hui, and Philip Levis. “Connecting Low-Power and Lossy Networks to the Internet”. In: *IEEE Communications Magazine* 49.4 (2011), pp. 96–101.
- [36] Vinay Kumar and Sudarshan Tiwari. “Routing in IPv6 over Low-Power Wireless Personal Area Networks (6LoWPAN): A Survey”. In: *Journal of Computer Networks and Communications* 2012 (2012), pp. 1–10. DOI: 10.1155/2012/316839.
- [37] José V. V. Sobral, Joel J. P. C. Rodrigues, Ricardo A. L. Rabêlo, Jalal Al-Muhtadi, and Valery Korotaev. “Routing Protocols for Low Power and Lossy Networks in Internet of Things Applications”. In: *Sensors* 19.9 (2019), p. 2144. DOI: 10.3390/s19092144.
- [38] *IEEE Standard for Low-Rate Wireless Networks*. IEEE, 2020. DOI: 10.1109/IEEESTD.2020.9144691.
- [39] Gabriel Montenegro, Nandakishore Kushalnagar, Jonathan W. Hui, and David E. Culler. *Transmission of IPv6 Packets over IEEE 802.15.4 Networks*. RFC 4944. 2007. DOI: 10.17487/RFC4944.
- [40] Jonathan W. Hui and Pascal Thubert. *Compression Format for IPv6 Datagrams over IEEE 802.15.4-Based Networks*. RFC 6282. 2011. DOI: 10.17487/RFC6282.
- [41] Olfa Gaddour and Anis Koubâa. “RPL in a nutshell: A survey”. In: *Computer Networks* 56.14 (2012), pp. 3163–3178.

- [42] Emilio Ancillotti, Raffaele Bruno, and Marco Conti. “Reliable Data Delivery with the IETF Routing Protocol for Low-Power and Lossy Networks”. In: *IEEE Transactions on Industrial Informatics* 10.3 (2014), pp. 1864–1877. DOI: 10.1109/TII.2014.2332117.
- [43] Jon Postel. *User Datagram Protocol*. RFC 768. 1980. DOI: 10.17487/RFC0768.
- [44] Carsten Bormann, Angelo P. Castellani, and Zach Shelby. “CoAP: An Application Protocol for Billions of Tiny Internet Nodes”. In: *IEEE Internet Computing* 16.2 (2012), pp. 62–67.
- [45] Wenfeng Xia, Yonggang Wen, Chuan Heng Foh, Dusit Niyato, and Haiyong Xie. “A Survey on Software-Defined Networking”. In: *IEEE Communications Surveys & Tutorials* 17.1 (2015), pp. 27–51. DOI: 10.1109/COMST.2014.2330903.
- [46] Yong Li and Min Chen. “Software-Defined Network Function Virtualization: A Survey”. In: *IEEE Access* 3 (2015), pp. 2542–2553. DOI: 10.1109/ACCESS.2015.2499271.
- [47] Tie Luo, Hwee-Pink Tan, and Tony Q. S. Quek. “Sensor OpenFlow: Enabling Software-Defined Wireless Sensor Networks”. In: *IEEE Communications Letters* 16.11 (2012), pp. 1896–1899. DOI: 10.1109/LCOMM.2012.092812.121712.
- [48] Open Networking Foundation. *Software-Defined Networking: The New Norm for Networks*. Tech. rep. Open Networking Foundation, 2012.
- [49] Abdullah Al Hayajneh, Md Zakirul Alam Bhuiyan, and Ian McAndrew. “Improving Internet of Things (IoT) Security with Software-Defined Networking (SDN)”. In: *Computers* 9.1 (2020), p. 8. DOI: 10.3390/computers9010008.
- [50] Iqbal Alam, Kashif Sharif, Fan Li, Zohaib Latif, M. M. Karim, Sujit Biswas, Boubakr Nour, and Yu Wang. “A Survey of Network Virtualization Techniques for Internet of Things Using SDN and NFV”. In: *ACM Computing Surveys* 53.2 (2020), pp. 1–40. DOI: 10.1145/3379444.
- [51] Mehdi Amirinasab Nasab, Shahaboddin Shamshirband, Anthony Theodore Chronopoulos, Amir Mosavi, and Narjes Nabipour. “Energy-Efficient Method for Wireless Sensor Networks Low-Power Radio Operation in Internet of Things”. In: *Electronics* 9.2 (2020), p. 320. DOI: 10.3390/electronics9020320.
- [52] Julius Degesys, Ian Rose, Ankit Patel, and Radhika Nagpal. “DESYNC: Self-Organizing Desynchronization and TDMA on Wireless Sensor Networks”. In: *Proceedings of the 6th International Conference on Information Processing in Sensor Networks*. IPSN ’07. ACM, 2007, pp. 11–20. DOI: 10.1145/1236360.1236363.
- [53] Chuanying Zhai, Zhuo Zou, Qiang Chen, Lida Xu, Li-Rong Zheng, and Hannu Tenhunen. “Delay-Aware and Reliability-Aware Contention-Free MF-TDMA Protocol for Automated RFID Monitoring in Industrial IoT”. In: *Journal of Industrial Information Integration* 3 (2016), pp. 8–19. DOI: 10.1016/j.jii.2016.06.002.

- [54] Surjya Ray, Ilker Demirkol, and Wendi Heinzelman. “ATMA: Advertisement-Based TDMA Protocol for Bursty Traffic in Wireless Sensor Networks”. In: *Proceedings of the IEEE Global Telecommunications Conference*. GLOBECOM. 2010, pp. 1–5. DOI: 10.1109/GLOCOM.2010.5683930.
- [55] Yan Zhang, Haopeng Huang, Qingqing Huang, and Yan Han. “6TiSCH IIoT Network: A Review”. In: *Computer Networks* 254 (2024), p. 110759. DOI: 10.1016/j.comnet.2024.110759.
- [56] Abdeldjalil Tabouche, Badis Djamaa, and Mustapha Reda Senouci. “Traffic-Aware Reliable Scheduling in TSCH Networks for Industry 4.0: A Systematic Mapping Review”. In: *IEEE Communications Surveys & Tutorials* 25.4 (2023), pp. 2834–2861. DOI: 10.1109/COMST.2023.3302157.
- [57] Mohamed Sofiane Batta, Zibouda Aliouat, and Saad Harous. “A Distributed Weight-Based TDMA Scheduling Algorithm for Latency Improvement in IoT”. In: *Proceedings of the IEEE 10th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference*. UEMCON. 2019, pp. 768–774. DOI: 10.1109/UEMCON47517.2019.8993097.
- [58] Ilker Demirkol, Cem Ersoy, and Fatih Alagöz. “MAC Protocols for Wireless Sensor Networks: A Survey”. In: *IEEE Communications Magazine* 44.4 (2006), pp. 115–121. DOI: 10.1109/MCOM.2006.1632658.
- [59] Wei Ye, John Heidemann, and Deborah Estrin. “An Energy-Efficient MAC Protocol for Wireless Sensor Networks”. In: *Proceedings of the 21st Annual Joint Conference of the IEEE Computer and Communications Societies*. Vol. 3. INFOCOM. IEEE, 2002, pp. 1567–1576. DOI: 10.1109/INFCOM.2002.1019408.
- [60] Tijs van Dam and Koen Langendoen. “An Adaptive Energy-Efficient MAC Protocol for Wireless Sensor Networks”. In: *Proceedings of the 1st International Conference on Embedded Networked Sensor Systems*. SenSys. ACM, 2003, pp. 171–180. DOI: 10.1145/958491.958512.
- [61] Joseph Polastre, Jason Hill, and David Culler. “Versatile Low Power Media Access for Wireless Sensor Networks”. In: *Proceedings of the 2nd International Conference on Embedded Networked Sensor Systems*. SenSys. ACM, 2004, pp. 95–107. DOI: 10.1145/1031495.1031508.
- [62] Michael Buettner, Gary V. Yee, Eric Anderson, and Richard Han. “X-MAC: A Short Preamble MAC Protocol for Duty-Cycled Wireless Sensor Networks”. In: *Proceedings of the 4th International Conference on Embedded Networked Sensor Systems*. SenSys. ACM, 2006, pp. 307–320.
- [63] Yanjun Sun, Omer Gurewitz, and David B. Johnson. “RI-MAC: A Receiver-Initiated Asynchronous Duty Cycle MAC Protocol for Dynamic Traffic Loads in Wireless Sensor Networks”. In: *Proceedings of the 6th ACM Conference on Embedded Networked Sensor Systems*. SenSys. ACM, 2008, pp. 1–14. DOI: 10.1145/1460412.1460414.

- [64] Adam Dunkels. *The ContikiMAC Radio Duty Cycling Protocol*. 2011.
- [65] Adam Kozłowski and Janusz Sosnowski. “Energy Efficiency Trade-Off Between Duty-Cycling and Wake-Up Radio Techniques in IoT Networks”. In: *Wireless Personal Communications* 107 (2019), pp. 1951–1971. DOI: 10.1007/s11277-019-06368-0.
- [66] Saad El Jaouhari and Eric Bouvet. “Secure firmware Over-The-Air updates for IoT: Survey, challenges, and discussions”. In: *Internet of Things* 18 (2022), p. 100508.
- [67] Konstantinos Arakadakis, Pavlos Charalampidis, Antonis Makrogiannakis, and Alexandros Fragkiadakis. “Firmware Over-the-air Programming Techniques for IoT Networks - A Survey”. In: *ACM Comput. Surv.* 54.9 (Oct. 2021). ISSN: 0360-0300. DOI: 10.1145/3472292.
- [68] Emmanuel Baccelli, Cenk Gündoğan, Oliver Hahm, Peter Kietzmann, Martine S Lenders, Hauke Petersen, Kaspar Schleiser, Thomas C Schmidt, and Matthias Wählisch. “RIOT: An open source operating system for low-end embedded devices in the IoT”. In: *IEEE Internet of Things Journal* 5.6 (2018), pp. 4428–4440.
- [69] George Oikonomou, Simon Duquennoy, Atis Elsts, Joakim Eriksson, Yasuyuki Tanaka, and Nicolas Tsiftes. “The Contiki-NG open source operating system for next generation IoT devices”. In: *SoftwareX* 18 (2022), p. 101089.
- [70] Thomas Watteyne, Xavier Vilajosana, Branko Kerkez, Fabien Chraim, Kevin Weekly, Qin Wang, Steven Glaser, and Kris Pister. “OpenWSN: a standards-based low-power wireless development environment”. In: *Transactions on Emerging Telecommunications Technologies* 23.5 (2012), pp. 480–493.
- [71] Andrea De Simone, Giovanna Turvani, and Fabrizio Riente. “Incremental firmware update over-the-air for low-power IoT devices over LoRaWAN”. In: *Internet of Things* (2025), p. 101772.
- [72] Jayden Renee Sorensen, Syed Aftab Rashid, Hossam ElHussini, and Andrei-Marian Dan. “Evaluating Differential Firmware Updates for Embedded IoT Device Fleets”. In: *2025 IEEE 21st International Conference on Factory Communication Systems (WFCS)*. IEEE. 2025, pp. 1–8.
- [73] Evgeny Gavrin, Sung-Jae Lee, Ruben Ayrapetyan, and Andrey Shitov. “Ultra lightweight JavaScript engine for internet of things”. In: *Companion Proceedings of the 2015 ACM SIGPLAN International Conference on Systems, Programming, Languages and Applications: Software for Humanity*. 2015, pp. 19–20.
- [74] Emmanuel Baccelli, Joerg Doerr, Shinji Kikuchi, Francisco Acosta Padilla, Kaspar Schleiser, and Ian Thomas. “Scripting Over-The-Air: Towards Containers on Low-end Devices in the Internet of Things”. In: *2018 IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops)*. 2018, pp. 504–507. DOI: 10.1109/PERCOMW.2018.8480277.

- [75] Dong Li, Bin Huang, Li Cui, and Zhiwei Xu. “WebletScript: A lightweight distributed JavaScript engine for Internet of Things”. In: *2018 IEEE Global Communications Conference (GLOBECOM)*. IEEE. 2018, pp. 1–6.
- [76] Adam Dunkels, Thiemo Voigt, et al. “Contiki-a lightweight and flexible operating system for tiny networked sensors”. In: *29th annual IEEE international conference on local computer networks*. IEEE Computer Society. 2004, pp. 455–462.
- [77] Waqaas Munawar, Muhammad Hamad Alizai, Olaf Landsiedel, and Klaus Wehrle. “Dynamic tinyos: Modular and transparent incremental code-updates for sensor networks”. In: *2010 IEEE International Conference on Communications*. IEEE. 2010, pp. 1–6.
- [78] Chih-Chieh Han, Ram Kumar, Roy Shea, Eddie Kohler, and Mani Srivastava. “A dynamic operating system for sensor nodes”. In: *Proceedings of the 3rd international conference on Mobile systems, applications, and services*. 2005, pp. 163–176.
- [79] Yu-Ting Chen, Ting-Chou Chien, and Pai H Chou. “Enix: a lightweight dynamic operating system for tightly constrained wireless sensor platforms”. In: *Proceedings of the 8th ACM Conference on Embedded Networked Sensor Systems*. 2010, pp. 183–196.
- [80] Amir Taherkordi, Frederic Loiret, Romain Rouvoy, and Frank Eliassen. “Optimizing sensor network reprogramming via in situ reconfigurable components”. In: *ACM Transactions on Sensor Networks (TOSN)* 9.2 (2013), pp. 1–33.
- [81] Barry Porter, Utz Roedig, and Geoff Coulson. “Type-safe updating for modular WSN software”. In: *2011 International Conference on Distributed Computing in Sensor Systems and Workshops (DCOSS)*. IEEE. 2011, pp. 1–8.
- [82] Peter Ruckebusch, Eli De Poorter, Carolina Fortuna, and Ingrid Moerman. “Gitar: Generic extension for internet-of-things architectures enabling dynamic updates of network and application modules”. In: *Ad Hoc Networks* 36 (2016), pp. 127–151.
- [83] Philip Levis and David Culler. “Maté: A tiny virtual machine for sensor networks”. In: *ACM Sigplan Notices* 37.10 (2002), pp. 85–95.
- [84] Niels Brouwers, Koen Langendoen, and Peter Corke. “Darjeeling, a feature-rich VM for the resource poor”. In: *Proceedings of the 7th ACM Conference on Embedded Networked Sensor Systems*. 2009, pp. 169–182.
- [85] Eduardo Ribeiro, André Restivo, Hugo Sereno Ferreira, and João Pedro Dias. “WASMICO: Micro-containers in microcontrollers with WebAssembly”. In: *Journal of Systems and Software* 214 (2024), p. 112081.
- [86] Koen Zandberg and Emmanuel Baccelli. “Minimal Virtual Machines on IoT Microcontrollers: The Case of Berkeley Packet Filters with rBPF”. In: *2020 9th IFIP International Conference on Performance Evaluation and Modeling in Wireless Networks (PEMWN)*. 2020, pp. 1–6. DOI: 10.23919/PEMWN50727.2020.9293081.

- [87] Szymon Kubica and Marios Kogias. “ μ BPF: Using eBPF for Microcontroller Compartmentalization”. In: *Proceedings of the ACM SIGCOMM 2024 Workshop on eBPF and Kernel Extensions*. 2024, pp. 23–29.
- [88] Koen Zandberg, Emmanuel Baccelli, Shenghao Yuan, Frédéric Besson, and Jean-Pierre Talpin. “Femto-containers: lightweight virtualization and fault isolation for small software functions on low-power IoT microcontrollers”. In: *Proceedings of the 23rd ACM/IFIP International Middleware Conference*. 2022, pp. 161–173.
- [89] L. Kleinrock and S. Lam. “Packet Switching in a Multiaccess Broadcast Channel: Performance Evaluation”. In: *IEEE Transactions on Communications* 23.4 (1975), pp. 410–423. DOI: 10.1109/TCOM.1975.1092814.
- [90] Habib Mostafaei and Michael Menth. “Software-defined wireless sensor networks: A survey”. In: *Journal of Network and Computer Applications* 119 (2018), pp. 42–56. ISSN: 1084-8045.
- [91] Leticia Djennadi, Gladys Diaz, Khaled Boussetta, and Christophe Cerin. “Exploring SDN architectures for IoT over Low-power and Lossy Networks (LLNs): A survey”. In: *Computer Networks* 270 (2025), p. 111494. ISSN: 1389-1286. DOI: 10.1016/j.comnet.2025.111494.
- [92] Laura Galluccio, Sebastiano Milardo, Giacomo Morabito, and Sergio Palazzo. “SDN-WISE: Design, prototyping and experimentation of a stateful SDN solution for Wireless Sensor networks”. In: *2015 IEEE Conference on Computer Communications (INFOCOM)*. IEEE. 2015, pp. 513–521.
- [93] Raheleh Samadi, Amin Nazari, and Jochen Seitz. “Intelligent Energy-Aware Routing Protocol in Mobile IoT Networks Based on SDN”. In: *IEEE Transactions on Green Communications and Networking* 7.4 (2023), pp. 2093–2103. DOI: 10.1109/TGCN.2023.3296272.
- [94] Michael Baddeley, Reza Nejabati, George Oikonomou, Sedat Gormus, Mahesh Sooriyabandara, and Dimitra Simeonidou. “Isolating SDN control traffic with layer-2 slicing in 6TiSCH industrial IoT networks”. In: *2017 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*. IEEE. 2017, pp. 247–251.
- [95] Farzad Veisi, Julien Montavont, and Fabrice Théoleyre. “Enabling Centralized Scheduling Using Software Defined Networking in Industrial Wireless Sensor Networks”. In: *IEEE Internet of Things Journal* 10.23 (2023), pp. 20675–20685. DOI: 10.1109/JIOT.2023.3302994.
- [96] Federico Orozco-Santos, Víctor Sempere-Payá, Javier Silvestre-Blanes, and Jose Vera-Pérez. “Scalability Enhancement on Software Defined Industrial Wireless Sensor Networks Over TSCH”. In: *IEEE Access* 10 (2022), pp. 107137–107151. DOI: 10.1109/ACCESS.2022.3212728.

- [97] Sajad Khorsandroo, Adrián Gallego Sánchez, Ali Saman Tosun, José M Arco, and Roberto Doriguzzi-Corin. “Hybrid SDN evolution: A comprehensive survey of the state-of-the-art”. In: *Computer Networks* 192 (2021), p. 107981.
- [98] Stefano Vissicchio, Luca Cittadini, Olivier Bonaventure, Geoffrey G. Xie, and Laurent Vanbever. “On the co-existence of distributed and centralized routing control-planes”. In: *2015 IEEE Conference on Computer Communications (INFOCOM)*. 2015, pp. 469–477. DOI: 10.1109/INFOCOM.2015.7218413.
- [99] Olivier Tilmans and Stefano Vissicchio. “IGP-as-a-Backup for Robust SDN Networks”. In: *10th International Conference on Network and Service Management (CNSM) and Workshop*. IEEE. 2014, pp. 127–135.
- [100] Tao Feng and Jun Bi. “OpenRouteFlow: Enable legacy router as a software-defined routing service for hybrid SDN”. In: *2015 24th International Conference on Computer Communication and Networks (ICCCN)*. IEEE. 2015, pp. 1–8.
- [101] Mehran Abolhasan, Justin Lipman, Wei Ni, and Brett Hagelstein. “Software-Defined Wireless Networking: Centralized, Distributed, or Hybrid?” In: *IEEE Network* 29.4 (2015), pp. 32–38.
- [102] Malik Najmus Siraj, Zaheer Ahmed, Muhammad Kashif Hanif, Muhammad Hasanain Chaudary, Shoab Ahmed Khan, and Nadeem Javaid. “A Hybrid Routing Protocol for Wireless Distributed Networks”. In: *IEEE Access* 6 (2018), pp. 67244–67260. DOI: 10.1109/ACCESS.2018.2875952.
- [103] Rui Silva, David Santos, Flávio Meneses, Daniel Corujo, and Rui L. Aguiar. “A Hybrid SDN Solution for Mobile Networks”. In: *Computer Networks* 190 (2021), p. 107958. DOI: 10.1016/j.comnet.2021.107958.
- [104] Ran Duo, Celimuge Wu, Tsutomu Yoshinaga, Jiefang Zhang, and Yusheng Ji. “SDN-Based Handover Scheme in Cellular/IEEE 802.11p Hybrid Vehicular Networks”. In: *Sensors* 20.4 (2020), p. 1082. DOI: 10.3390/s20041082.
- [105] Rong Chen, Han Ji, Xiping Wu, and Cheng-Xiang Wang. “A Mininet-based SD-WLAN emulator for hybrid LiFi and WiFi networks with performance analysis”. In: *2025 13th International Conference on Intelligent Computing and Wireless Optical Communications (ICWOC)*. IEEE. 2025, pp. 240–245.
- [106] Michele Bonanni, Francesco Chiti, and Laura Pierucci. “Hybrid Dual Stack Control Plane Design for Resilient Software-Defined WSANs”. In: *IEEE Internet of Things Journal* 12.18 (2025), pp. 37853–37862. DOI: 10.1109/JIOT.2025.3584375.
- [107] Gayoung Kim, Jin-Gu Kang, and Minjoong Rim. “Dynamic duty-cycle MAC protocol for IoT environments and wireless sensor networks”. In: *Energies* 12.21 (2019), p. 4069.

- [108] Niayesh Gharaei. “A Novel Adaptive TSCH-MAC Protocol for Industrial IoT: Enhancing Efficiency via Topology, Trust, and Data Diversity”. In: *IEEE Transactions on Green Communications and Networking* 10 (2026), pp. 2641–2650.
- [109] Thair A Al-Janabi and Hamed S Al-Raweshidy. “An energy efficient hybrid MAC protocol with dynamic sleep-based scheduling for high density IoT networks”. In: *IEEE Internet of Things Journal* 6.2 (2019), pp. 2273–2287.
- [110] Damilola D Olatinwo, Adnan M Abu-Mahfouz, Gerhard P Hancke, and Hermanus C Myburgh. “Energy-efficient multichannel hybrid MAC protocol for IoT-enabled WBAN systems”. In: *IEEE Sensors Journal* 23.22 (2023), pp. 27967–27983.
- [111] Shuguo Zhuo and Ye-Qiong Song. “GoMacH: A traffic adaptive multi-channel MAC protocol for IoT”. In: *2017 IEEE 42nd Conference on Local Computer Networks (LCN)*. IEEE. 2017, pp. 489–497.
- [112] Muhammad Ateeq, Farruh Ishmanov, Muhammad Khalil Afzal, and Muhammad Naeem. “Multi-parametric analysis of reliability and energy consumption in IoT: A deep learning approach”. In: *Sensors* 19.2 (2019), p. 309.
- [113] Muhammad Ateeq, Farruh Ishmanov, Muhammad Khalil Afzal, and Muhammad Naeem. “Predicting delay in IoT using deep learning: A multiparametric approach”. In: *IEEE Access* 7 (2019), pp. 62022–62031.
- [114] Aroosa Hameed, John Violos, Nina Santi, Aris Leivadeas, and Nathalie Mitton. “A machine learning regression approach for throughput estimation in an IoT environment”. In: *2021 IEEE International Conferences on Internet of Things (iThings)*. IEEE. 2021, pp. 29–36.
- [115] Muhammad Ateeq, Muhammad Khalil Afzal, Muhammad Naeem, Muhammad Shafiq, and Jin-Ghoo Choi. “Deep learning-based multiparametric predictions for IoT”. In: *Sustainability* 12.18 (2020), p. 7752.
- [116] Samir Si-Mohammed and Fabrice Theoleyre. “Per link data-driven network replication towards self-adaptive digital twins”. In: *2025 International Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems (MSWiM)*. IEEE. 2025, pp. 84–93.
- [117] Mostafa Hussien, Islam ATF Taj-Eddin, Mohammed FA Ahmed, Ali Ranjha, Kim Khoa Nguyen, and Mohamed Cheriet. “Evolution of MAC protocols in the machine learning decade: A comprehensive survey”. In: *arXiv preprint arXiv:2302.13876* (2023).
- [118] Andre Gomes, Daniel F Macedo, and Luiz FM Vieira. “Automatic MAC protocol selection in wireless networks based on reinforcement learning”. In: *Computer Communications* 149 (2020), pp. 312–323.

- [119] Arslan Musaddiq, Zulqar Nain, Yazdan Ahmad Qadri, Rashid Ali, and Sung Won Kim. “Reinforcement Learning-Enabled Cross-Layer Optimization for Low-Power and Lossy Networks under Heterogeneous Traffic Patterns”. In: *Sensors* 20.15 (2020), p. 4158. DOI: 10.3390/s20154158.
- [120] Chien-Min Wu, Yen-Chun Kao, Kai-Fu Chang, Cheng-Tai Tsai, and Cheng-Chun Hou. “A Q-Learning-Based Adaptive MAC Protocol for Internet of Things Networks”. In: *IEEE Access* 9 (2021), pp. 128905–128918. DOI: 10.1109/ACCESS.2021.3103718.
- [121] Nasser S Albalawi, Yazeed Alzahrani, Nada Alsalmi, Yogesh Patidar, and Manoj Tolani. “Energy-efficient priority encoding strategies using machine learning based hybrid MAC protocol for wireless sensor networks”. In: *Scientific Reports* 15.1 (2025), p. 45054.
- [122] Sohail Sarang, Goran M Stojanović, Micheal Driberg, Stevan Stankovski, Kishore Bingi, and Varun Jeoti. “Machine learning prediction based adaptive duty cycle MAC protocol for solar energy harvesting wireless sensor networks”. In: *IEEE Access* 11 (2023), pp. 17536–17554.
- [123] Brendan Moran, Hannes Tschofenig, David Brown, and Milosch Meriac. *A Firmware Update Architecture for Internet of Things*. RFC 9019. Apr. 2021.
- [124] Atis Elsts, Xenofon Fafoutis, George Oikonomou, Robert Piechocki, and Ian Craddock. “TSCH networks for health IoT: Design, evaluation, and trials in the wild”. In: *ACM Transactions on Internet of Things* 1.2 (2020), pp. 1–27.
- [125] Jiliang Wang, Zhichao Cao, Xufei Mao, Xiang-Yang Li, and Yunhao Liu. “Towards Energy Efficient Duty-Cycled Networks: Analysis, Implications and Improvement”. In: *IEEE Transactions on Computers* 65.1 (2016), pp. 270–280. DOI: 10.1109/TC.2015.2417558.
- [126] Manuel Mazo and Paulo Tabuada. “Decentralized Event-Triggered Control Over Wireless Sensor/Actuator Networks”. In: *IEEE Transactions on Automatic Control* 56.10 (2011), pp. 2456–2461. DOI: 10.1109/TAC.2011.2164036.
- [127] The New York Times. *Nikola Tesla Sees a Wireless Vision*. Wikisource. Originally published in The New York Times, October 3, 1915. Accessed: 2026-06-25. Oct. 1915. URL: https://en.wikisource.org/wiki/The_New_York_Times/1915/10/03/Nikola_Tesla_Sees_a_Wireless_Vision.